INFORMATIONAL MEETING ON HARDWARE DESCRIPTION LANGUAGES (HDL)

SPEAKERS:

Giuliano Cardinali

Emanuel Conti

ORGANIZED BY:

PROFESSIONISTI PER L'INNOVAZIONE DIGITALE

— ASSOCIAZIONE — PID

https://www.pidrsm.sm

https://www.facebook.com/professionistiperinnovazionedigitale

PART ONE
DIGITAL LOGIC NETWORKS

HISTORICAL NOTES

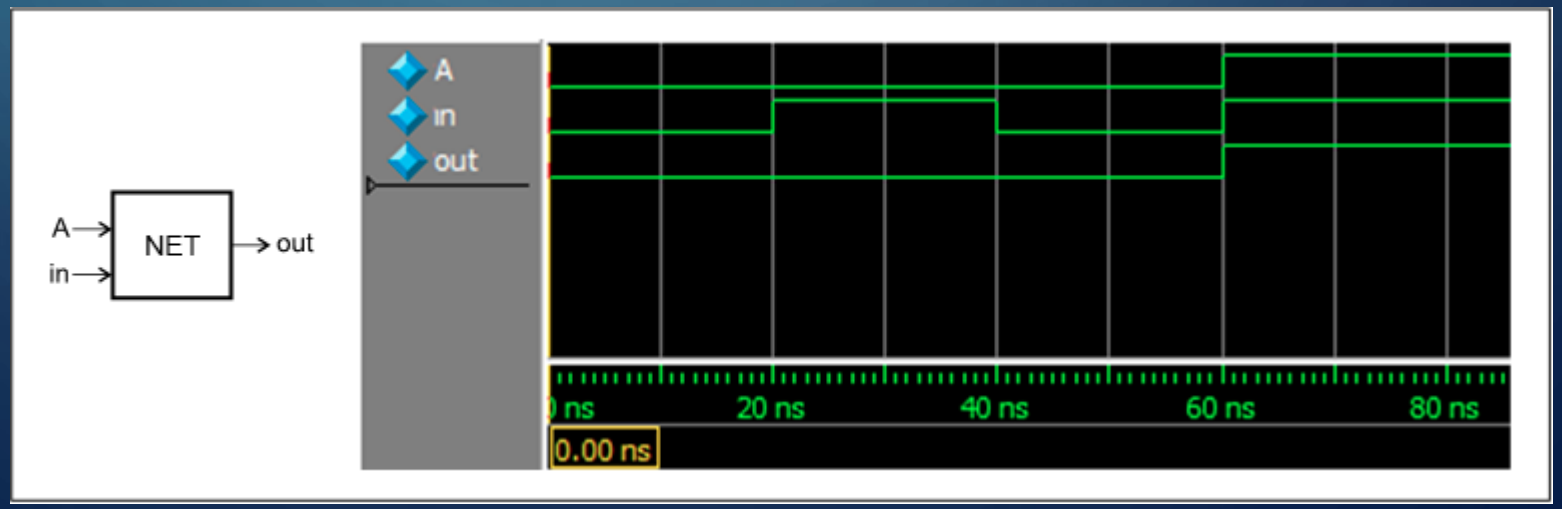
- Binary system: Leibniz (1705).
- Boolean algebra: Boole (1847).
- Boolean algebra applied to electrical circuits: thanks to Peirce (1886), to the work of several nineteenth-century philosophers and mathematicians, and following the technological developments of the early twentieth century, the first digital logic circuits were built in 1924 by Walther Bothe.
- Claude Shannon, the father of Information Theory, showed in 1938 that Boole's two-valued logic and Peirce's work were applicable to electrical circuits (relays) and electronic circuits (vacuum tubes), contributing strongly to the creation of digital circuits described by means of "Logic Network Theory".

LOGIC NETWORKS

- Logic networks process information. They are the foundation of many electronic devices and the heart of modern digital computers.
- Boolean algebra is used to describe their structure and behavior.
- The values handled are normally symbols representing true and false (by convention 1 and 0). The variables take the values above and are therefore called binary.

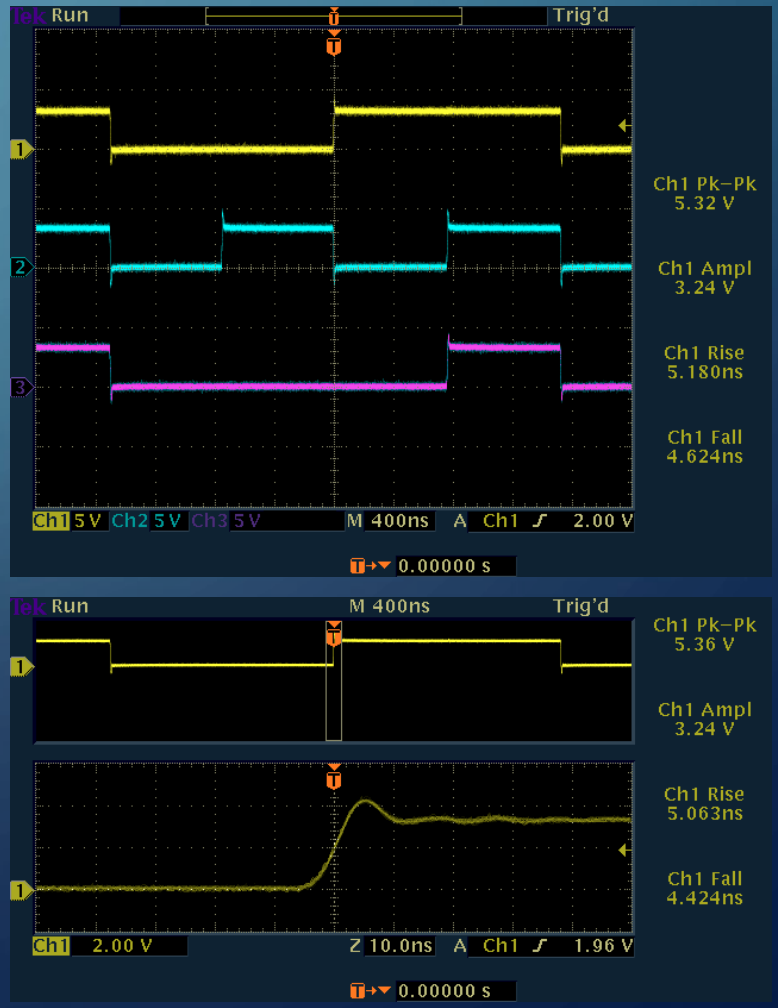
SIGNALS IN THE CONTEXT OF LOGIC NETWORKS

- In electronic digital circuits it is useful to regard binary variables as signals: voltages or currents that evolve over time.
- When represented ideally, they show discontinuities in the transitions between the two values.



REAL SIGNALS AND LOGIC NETWORKS

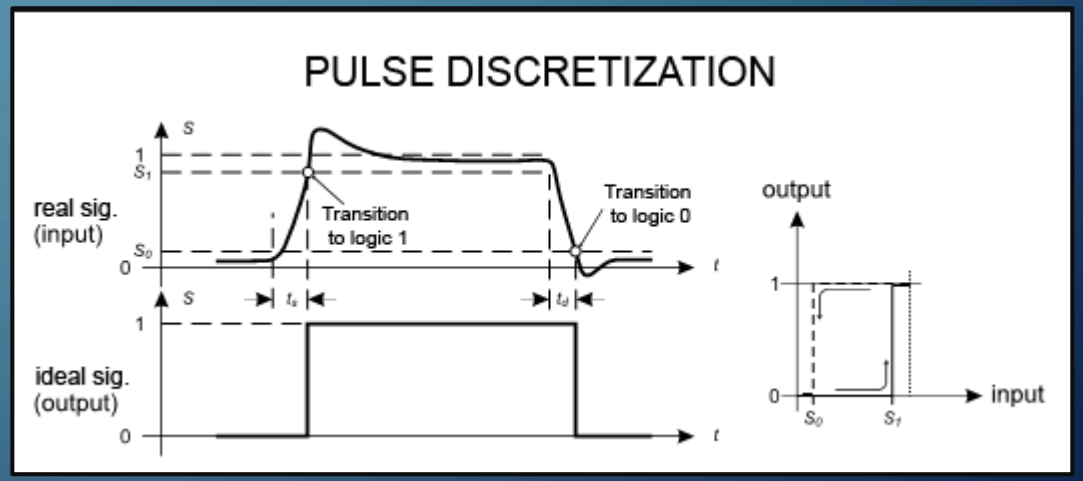
- In the real world, the function that represents the evolution of values over time is continuous.
- Signals can be degraded by physical factors or phenomena, either internal or external to the logic network itself, which may compromise their integrity.



SIGNAL INTEGRITY: THEY CAN DEGRADE DUE TO…

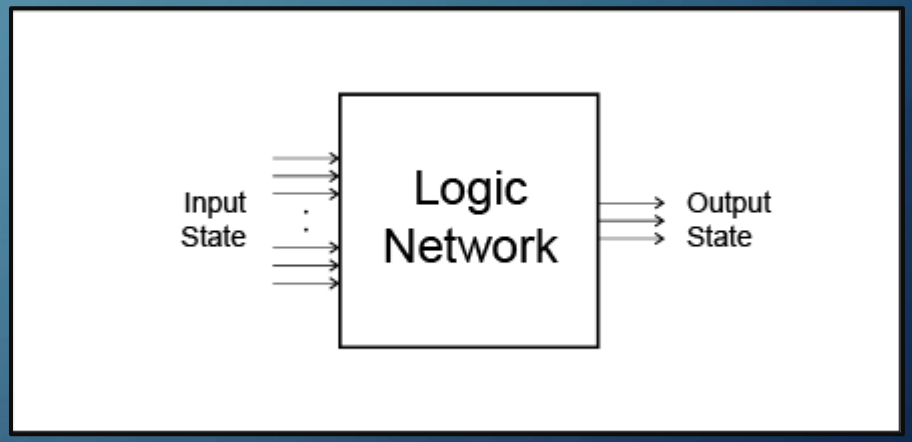
- Attenuation due to (thermal) energy dissipation in devices and electrical connections.
- Overshoot/Undershoot, i.e. signals may take values that "exceed" the absolute permissible limits because of second-order effects due to the reactive components distributed in the circuit (parasitic inductances and capacitances).
- In the distributed-parameter regime, propagation delays for unavoidable physical reasons (non-negligible propagation times along lines).
- Reflections between the two ends of a transmission line due to impedance mismatches between the signal source, the load and the line itself.
- Electrical noise of various kinds: intrinsic thermal noise such as the Johnson effect, or induced noise (from inductive/capacitive coupling or external electromagnetic fields).

SIGNAL INTERPRETATION IN LOGIC NETWORKS



- To mitigate errors in interpreting signals as binary variables, two thresholds (conventional levels) S_0 and S_1 are usually defined. When the input signal exceeds threshold S_1 the output takes the value 1. For the output to return to 0, the input must fall below threshold S_0. The hysteresis defined by the thresholds and by the output state yields an output free of uncertainty (spikes) in the presence of small amounts of noise or disturbance.
- The discretization performed by the thresholds on disturbed or degraded signals still produces jitter (time fluctuations at value changes). The thresholds also cause a pulse to be slightly distorted in duration and delayed in time. In practice, discretizing a signal never produces ideal outputs like the one in the figure.

LOGIC NETWORKS AS BLACKBOXES



- Logic networks can be regarded as "black boxes" that may have an input state and an output state.
- Grouping several related signals (binary variables) can be seen as a "vector" representing a so-called "state"; a state may in turn represent several values at once and thus correspond to familiar "objects", such as numbers or characters. A single binary variable can itself be considered a state.
- Observable states are used to study them and therefore, when possible, to describe how they work. They may also have internal, unobservable states that are nevertheless necessary for their operation.

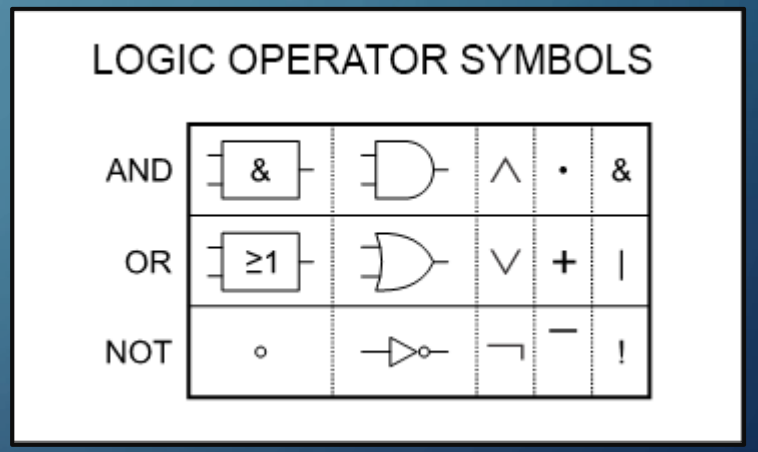
LOGIC DESIGN (1)

- Logic design aims at building digital devices that process input signals and produce output signals. The process of defining the structure of a logic network starting from its specifications is called "Synthesis". The opposite operation is called "Analysis".
- There are several methodologies for Synthesis, from "manual" methods to automatic procedures. Nowadays manual synthesis is performed very rarely.
- To design a logic network, the verbal specifications must be translated into formal specifications, using ideal signal diagrams, truth tables and Boolean equations for so-called "combinational networks". For so-called "sequential" networks, flow tables, state diagrams or so-called ASM charts will generally be needed.

LOGIC DESIGN (2)

- The specifications will determine a possible architecture for the system to be designed.
- With the specifications in hand, the technologies to adopt can be chosen based on the architecture and on purely technical or commercial requirements.
- Technologies can influence the type of development tools needed; in turn, depending on availability and cost, they may redefine technologies and architecture.

LOGIC NETWORKS: FUNDAMENTAL OPERATORS (1)



- Using the three fundamental operators (negation, intersection and union) to "combine" binary variables, any desired expression can be composed.
- In the context of logic networks, expressions always represent predicates. Predicates, in turn, can be defined as expressions whose result is only true or false. The only exception is the "simulation" or analysis scenario, where logic is extended to three values (three-valued logic), so that a predicate may fail to provide a certain answer given undefined inputs.

LOGIC NETWORKS: FUNDAMENTAL OPERATORS (2)

A function $y = f(x_1, \ldots, x_n)$ is a rule $\{0,1\}^n \to \{0,1\}$ that maps y to each combination of $x_1, \ldots, x_n$.

Functions can be constructed by enumeration in tables.

x	$f_1(x)$
0	1
1	0

$x_1\ x_2$	$f_2(x_1, x_2)$
0 0	0
0 1	1
1 0	1
1 1	1

$x_1\ x_2$	$f_3(x_1, x_2)$
0 0	0
0 1	0
1 0	0
1 1	1

These functions can also be described by means of Boolean equations, using the three fundamental operators corresponding to negation (complementation or NOT, symbols $\bar{}$ or $\neg$), sum (OR, symbol + or |), and product (AND, symbol $\cdot$ or &):

$$f_1(x) = \bar{x}, \qquad f_2(x_1, x_2) = x_1 + x_2, \qquad f_3(x_1, x_2) = x_1 \cdot x_2.$$

ALGEBRAIC PROPERTIES (1)

Any function can be represented by an algebraic expression built using these operators and parentheses where needed.
The operators $\bar{}$, $+$ and $\cdot$ have properties fully determined by the functions f_1, f_2 e f_3 introduced earlier.
The duality principle of Boolean algebra allows these properties to be stated in pairs simply by interchanging the operators $+$ and $\cdot$ and the symbols 0 and 1.

P_1)	$x + 0 = x$	P_1')	$x \cdot 1 = x$
P_2)	$x + 1 = 1$	P_2')	$x \cdot 0 = 0$
P_3)	$x + x = x$	P_3')	$x \cdot x = x$
P_4)	$x + \bar{x} = 1$	P_4')	$x \cdot \bar{x} = 0$
P_5)	$x_1 + x_2 = x_2 + x_1$	P_5')	$x_1 \cdot x_2 = x_2 \cdot x_1$
P_6)	$(x_1 + x_2) + x_3 = x_1 + (x_2 + x_3)$	P_6')	$(x_1 \cdot x_2) \cdot x_3 = x_1 \cdot (x_2 \cdot x_3)$
P_7)	$(x_1 \cdot x_2) + (x_1 \cdot x_3) = x_1 \cdot (x_2 + x_3)$	P_7')	$(x_1 + x_2) \cdot (x_1 + x_3) = x_1 + (x_2 \cdot x_3)$

Finally, we can define the "self-dual" property as follows:

P_8) $\quad \bar{\bar{x}} = x$

ALGEBRAIC PROPERTIES (2)

Note that properties P_6 and P_6' (associativity), which show that the grouping of a sequence of $+$ and $\cdot$ erations is immaterial, allow these operators to be defined for an arbitrary number of variables. Properties P_2 and P_2' also justify the existence of operators with n variables. For example, property P_2 can represent a function $x_1 + \ldots + x_n$ that equals 0 only if all the variables x_i equal 0. Dually, property P_2' describes a function $x_1 \cdot \ldots \cdot x_n$ that equals 1 only if all the variables x_i equal 1.

Note also that property P_7' (distributivity) does not hold in ordinary algebra.

Other properties useful for algebraic simplification (and other purposes) include:

$P_9)$ $\quad x_1 \cdot x_2 + x_1 \cdot \overline{x}_2 = x_1$

$P_9')$ $\quad (x_1 + x_2) \cdot (x_1 + \overline{x}_2) = x_1$

$P_{10})$ $\quad x_1 + x_1 \cdot x_2 = x_1$

$P_{10}')$ $\quad x_1 \cdot (x_1 + x_2) = x_1$

Finally, we state De Morgan's theorem in its two forms:

$P_{11})$ $\quad \overline{x_1 + x_2 + \ldots + x_n} = \overline{x}_1 \cdot \overline{x}_2 \cdot \ldots \cdot \overline{x}_n$

$P_{11}')$ $\quad \overline{x_1 \cdot x_2 \cdot \ldots \cdot x_n} = \overline{x}_1 + \overline{x}_2 + \ldots + \overline{x}_n$

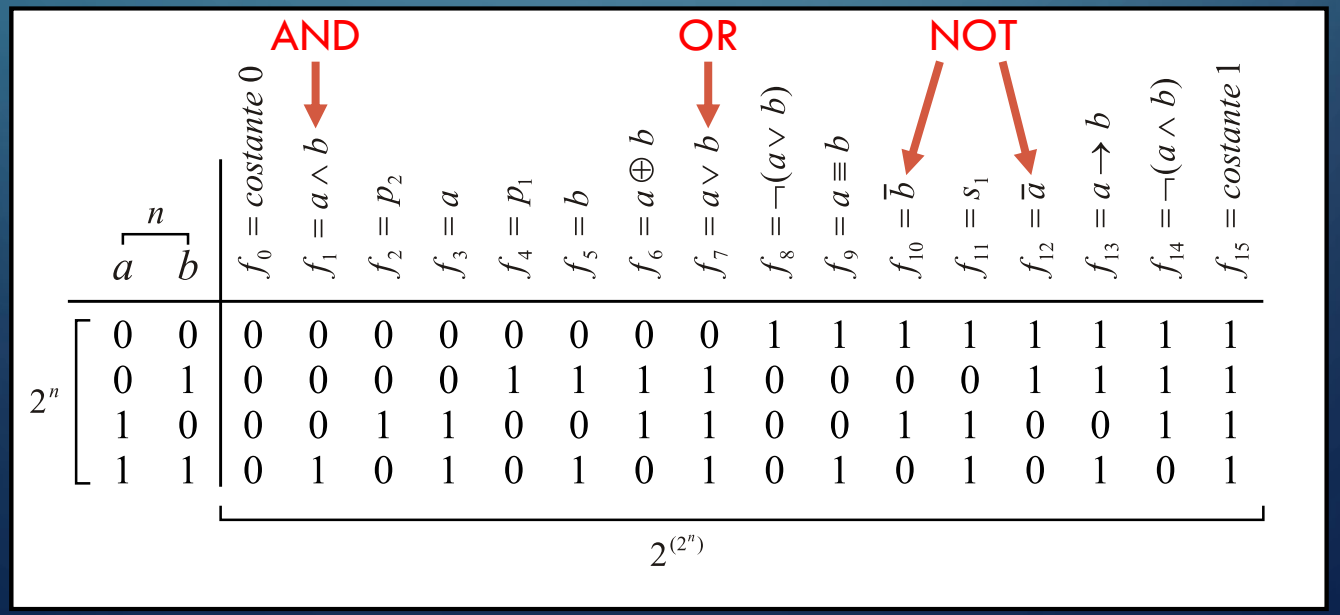
ENUMERATION OF FUNCTIONS (1)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For *n*=2 they are the following:

	n																
	a	b	$f_0 = costante\ 0$	$f_1 = a \wedge b$	$f_2 = p_2$	$f_3 = a$	$f_4 = p_1$	$f_5 = b$	$f_6 = a \oplus b$	$f_7 = a \vee b$	$f_8 = \neg(a \vee b)$	$f_9 = a \equiv b$	$f_{10} = \bar{b}$	$f_{11} = s_1$	$f_{12} = \bar{a}$	$f_{13} = a \rightarrow b$	$f_{14} = \neg(a \wedge b)$	$f_{15} = costante\ 1$
2^n	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
			$2^{(2^n)}$															

ENUMERATION OF FUNCTIONS (2)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For $n=2$ they are the following:



ENUMERATION OF FUNCTIONS (3)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For *n*=2 they are the following:

	n																
	a	b	$f_0 = costante\ 0$	$f_1 = a \wedge b$	$f_2 = p_2$	$f_3 = a$	$f_4 = p_1$	$f_5 = b$	$f_6 = a \oplus b$	$f_7 = a \vee b$	$f_8 = \neg(a \vee b)$ NOR	$f_9 = a \equiv b$	$f_{10} = \bar{b}$	$f_{11} = s_1$	$f_{12} = \bar{a}$	$f_{13} = a \rightarrow b$	$f_{14} = \neg(a \wedge b)$ NAND	$f_{15} = costante\ 1$
2^n	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
			$2^{(2^n)}$															

ENUMERATION OF FUNCTIONS (4)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For $n=2$ they are the following:

XOR

XNOR (EQ)

	n																
	a	b	$f_0 = costante\ 0$	$f_1 = a \wedge b$	$f_2 = p_2$	$f_3 = a$	$f_4 = p_1$	$f_5 = b$	$f_6 = a \oplus b$	$f_7 = a \vee b$	$f_8 = \neg(a \vee b)$	$f_9 = a \equiv b$	$f_{10} = \bar{b}$	$f_{11} = s_1$	$f_{12} = \bar{a}$	$f_{13} = a \rightarrow b$	$f_{14} = \neg(a \wedge b)$	$f_{15} = costante\ 1$
2^n	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
			$2^{(2^n)}$															

ENUMERATION OF FUNCTIONS (5)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For *n*=2 they are the following:

IMPLICATION

	n																
	a	b	$f_0 = costante\ 0$	$f_1 = a \wedge b$	$f_2 = p_2$	$f_3 = a$	$f_4 = p_1$	$f_5 = b$	$f_6 = a \oplus b$	$f_7 = a \vee b$	$f_8 = \neg(a \vee b)$	$f_9 = a \equiv b$	$f_{10} = \bar{b}$	$f_{11} = s_1$	$f_{12} = \bar{a}$	$f_{13} = a \rightarrow b$	$f_{14} = \neg(a \wedge b)$	$f_{15} = costante\ 1$
2^n	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
	0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
	1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
			$2^{(2^n)}$															

ENUMERATION OF FUNCTIONS (6)

There are $2^{(2^n)}$ possible functions that can be obtained from the combination of *n* binary variables. For *n*=2 they are the following:

CONTRADICTION

TAUTOLOGY

n																	
a	b	$f_0 = constante\ 0$	$f_1 = a \wedge b$	$f_2 = p_2$	$f_3 = a$	$f_4 = p_1$	$f_5 = b$	$f_6 = a \oplus b$	$f_7 = a \vee b$	$f_8 = \neg(a \vee b)$	$f_9 = a \equiv b$	$f_{10} = \bar{b}$	$f_{11} = s_1$	$f_{12} = \bar{a}$	$f_{13} = a \rightarrow b$	$f_{14} = \neg(a \wedge b)$	$f_{15} = constante\ 1$
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

2^n

$2^{(2^n)}$

STRUCTURE OF LOGIC NETWORKS

- **Combinational Networks**
  Comparable to "pure" functions, the output state depends solely on the values of the input state (one-to-one or onto function).
- **Sequential Networks**
  The outputs depend on the "history" of the input states up to the instant of observation considered. They are defined in terms of combinational networks equipped with feedback on themselves or with the addition of memory elements.

MANUAL SYNTHESIS OF COMBINATIONAL NETWORKS (1)

- With the formal definitions of the network's operation given in truth tables or as expressions, transformations can be applied to produce structures suitable for practical implementation: that is, this process involves their rearrangement, for example to adapt them to the technology to be used.
- Reorganizing the specifications also entails seeking the lowest cost in terms of network size, consistent with the required performance. Here the network is minimized while always keeping the desired performance in mind.

MANUAL SYNTHESIS OF COMBINATIONAL NETWORKS (2)

- Manually designing a logic network involves direct algebraic manipulation of the equations, "graphical" techniques on suitably rearranged truth tables, such as Veitch-Karnaugh maps, or a mix of these and other techniques.
- A relatively complex logic network may consist of several subnetworks that can be handled separately.
- The "logic minimizer" *Espresso* (or other similar tools) can be considered an aid to manual design.

SYNTHESIS AND MINIMIZATION OF COMBINATIONAL NETWORKS

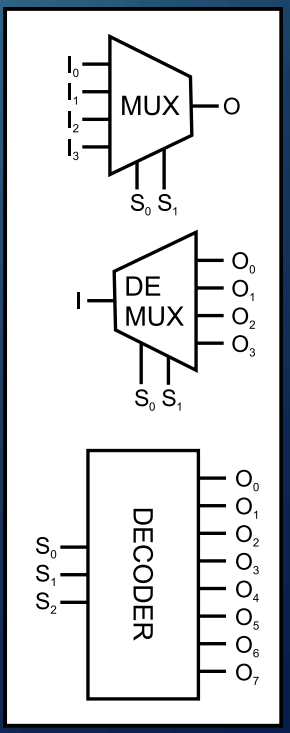
- Definition of the network with truth tables or Boolean equations (or equivalent)
- Optional minimization:
  - Manual minimization of combinational networks
    - With Boolean algebra (algebraic manipulation, tedious and error-prone).
    - With Veitch-Karnaugh maps (limited to six variables).
    - With Reed-Müller maps (using the XOR operator).
  - Automatic minimization (by computer)
    - With the Quine-McCluskey tabular method (prohibitive with a large number of variables).
    - With the *Espresso* heuristic minimizer or similar tools.

FROM EQUATIONS TO CIRCUITS

Expressions are manipulated in order to exploit predefined logic blocks, such as AND, OR and NOT gates, gates with intrinsic functional completeness such as NAND, NOR and XOR, or more complex functions such as DECODER, MUX, DEMUX, HALF-ADDER, FULL-ADDER, and others. These functions are embedded in low-density chips (SSI) and known as "discrete logic".

INPUT ➡	0	1
NOT	1	0

INPUT ➡	00	01	10	11
AND	0	0	0	1
OR	0	1	1	1
NAND	1	1	1	0
NOR	1	0	0	0
XOR	0	1	1	0
XNOR	1	0	0	1



CATEGORIES OF COMBINATIONAL LOGIC

- **Combinational logic network in canonical (or normal) form**
  It has two forms:
  - Disjunctive normal form (sum of products, or SOP)
  - Conjunctive normal form (product of sums, or POS)

  NB: "product" means conjunction (AND) and "sum" means disjunction (OR)

  A network in canonical form is characterized by two levels of logic (NOTs are not counted): this means that an input signal passes through two operators before reaching the output.

- **Multilevel combinational logic network**
  This is a network in which at least one input signal passes through more than two operators before reaching the output. It is generally a more compact form than the previous one, but usually exhibits longer propagation times. With this category of networks it is also possible to build circuits based on a single type of operator, provided it is functionally complete (that is, the operator includes the negation of at least one variable); examples of such operators are NAND, NOR and XOR.

COMBINATIONAL LOGIC NETWORKS (CANONICAL FORMS)

- These are networks made up of two levels of logic.
- They define the function by joining the intersections of several signals or their negations (sums of products or sums of minterms) or by intersecting the unions of several signals or their negations (products of sums or products of maxterms).

$f(a,b,c)$

	a b c	z	P_0 P_3 P_6 P_7	S_1 S_2 S_4 S_5
T_0	0 0 0	1	1 0 0 0	1 1 1 1
T_1	0 0 1	0	0 0 0 0	0 1 1 1
T_2	0 1 0	0	0 0 0 0	1 0 1 1
T_3	0 1 1	1	0 1 0 0	1 1 1 1
T_4	1 0 0	0	0 0 0 0	1 1 0 1
T_5	1 0 1	0	0 0 0 0	1 1 1 0
T_6	1 1 0	1	0 0 1 0	1 1 1 1
T_7	1 1 1	1	0 0 0 1	1 1 1 1

- The Products P_n (minterms) identify the input combinations (the terms T_n) for which the function equals 1. The Sums S_n (maxterms) identify the terms for which the function equals 0.

DESIGN EXAMPLE

- The function z described by the table at the side can be written as a Boolean expression using one of the two canonical forms. In this example the SOP (Sum of Products) form was used.
- From the tabular description of 'z', the terms where the function equals 1 (the minterms only) are selected; in the predicate z = f(a, b, c) the terms for the products P_0, P_3, P_6 and P_7 are listed, taking care to negate the arguments (a, b or c) that are zero for a given product P_n.
- By applying the algebraic properties seen earlier, a reduced or alternative form of the original expression can be obtained (note that the last version of function z in the box at the bottom right is not in canonical form, i.e. we will see later that it represents logic with more than two levels).

	a b c	z	P_0 P_3 P_6 P_7	S_1 S_2 S_4 S_5
T_0	0 0 0	1	1 0 0 0	1 1 1 1
T_1	0 0 1	0	0 0 0 0	0 1 1 1
T_2	0 1 0	0	0 0 0 0	1 0 1 1
T_3	0 1 1	1	0 1 0 0	1 1 1 1
T_4	1 0 0	0	0 0 0 0	1 1 0 1
T_5	1 0 1	0	0 0 0 0	1 1 1 0
T_6	1 1 0	1	0 0 1 0	1 1 1 1
T_7	1 1 1	1	0 0 0 1	1 1 1 1

Selection of product terms

$$z = \sum(P_0, P_3, P_6, P_7)$$

$$z = \bar{a} \cdot \bar{b} \cdot \bar{c} + \bar{a} \cdot b \cdot c + a \cdot b \cdot \bar{c} + a \cdot b \cdot c$$

Algebraic minimization

$$z = \bar{a} \cdot \bar{b} \cdot \bar{c} + \cancel{\bar{a}} \cdot b \cdot c + \cancel{a} \cdot b \cdot c + a \cdot b \cdot \cancel{\bar{c}} + a \cdot b \cdot \cancel{c}$$

$$z = \bar{a} \cdot \bar{b} \cdot \bar{c} + b \cdot c + b \cdot c + a \cdot b + a \cdot b$$

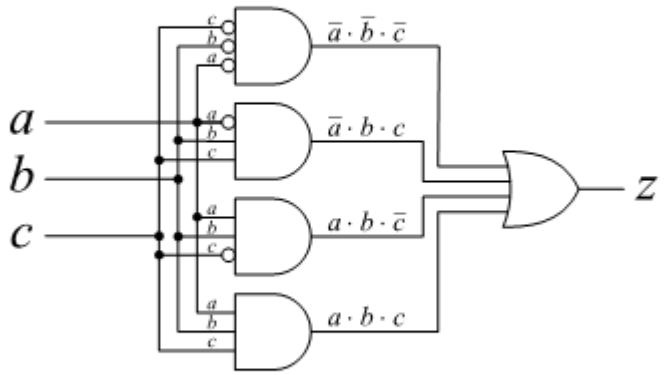
$$z = \bar{a} \cdot \bar{b} \cdot \bar{c} + a \cdot b + b \cdot c$$

$$z = \bar{a} \cdot \bar{b} \cdot \bar{c} + b \cdot (a + c)$$

IMPLEMENTATION WITH LOGIC GATES

- A non-minimized canonical SOP version of function *z* can be built using only logic gates corresponding to the negation, intersection and union operators.
- Note that some terms going into the AND gates, all of which have three inputs, pass through a negation operator (shown as an empty circle).
- Canonical forms are ideal for building reliable logic circuits and naturally fit the structures of the simplest programmable logic devices, allowing direct technology mapping.

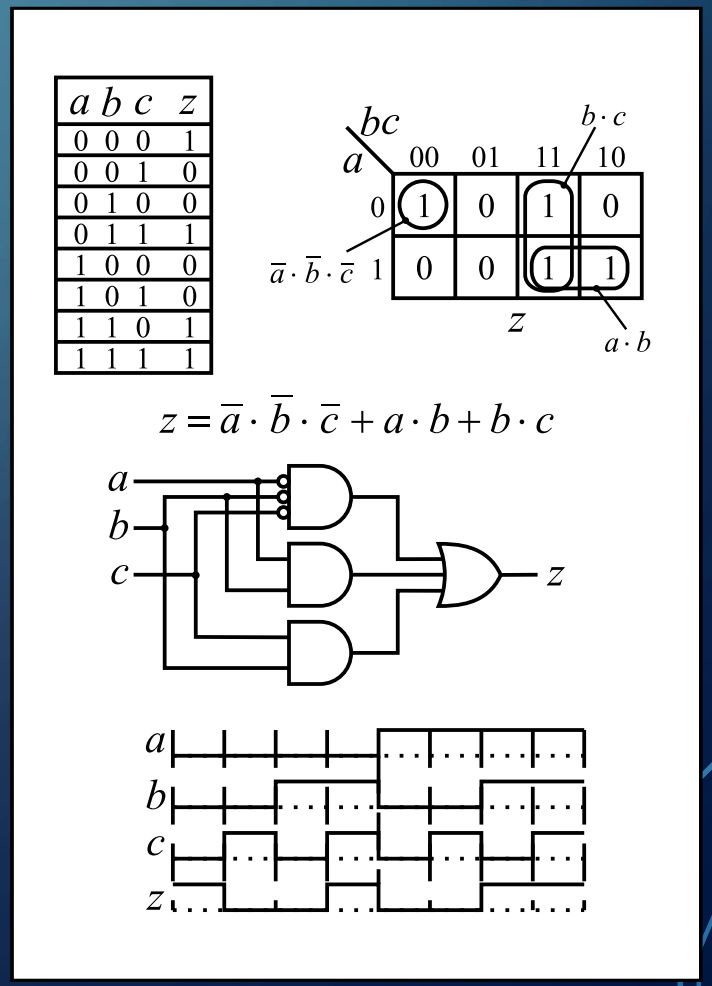
Function 'z' in unminimized canonical SOP form, implemented using logic gates



$$z = \bar{a}\cdot\bar{b}\cdot\bar{c} + \bar{a}\cdot b\cdot c + a\cdot b\cdot\bar{c} + a\cdot b\cdot c$$

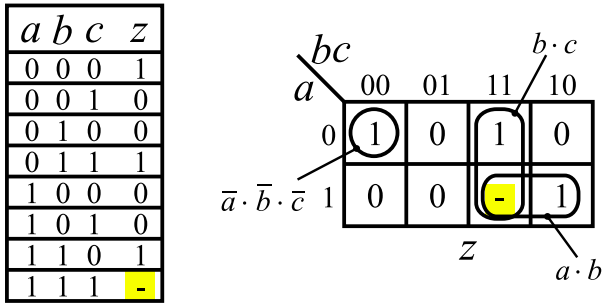
MINIMIZATION WITH THE GRAPHICAL METHOD (1)

- Besides the algebraic manipulation seen earlier, an alternative way to minimize a function is to use Veitch-Karnaugh maps, mainly known as Karnaugh maps or K-Maps. They exploit properties P_9 or P'_9 seen earlier, highlighting the so-called logical adjacency of two or more *minterms* (SOP) or of two or more *maxterms* (POS).
- For SOP forms, K-Maps are used to group all the *minterms* that can geometrically form squares or rectangles whose sides have a height or width equal to a power of two cells. For POS forms the maxterms are grouped in the same way. Grouping terms into regular areas with the characteristics above is called a "cover". The larger a cover, the fewer variables are involved and therefore the "smaller" the operator required.

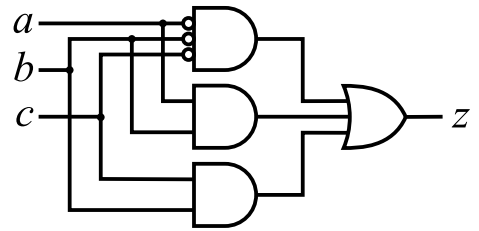


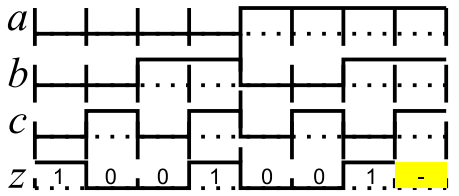
MINIMIZATION WITH THE GRAPHICAL METHOD (2)

- In some cases the tables representing the functions may be not completely specified. That is, for some terms the result is a don't-care and can be considered either zero or one depending on convenience. In the example at the side, term T_7 is considered a *minterm* (one) because it allows two overlapping covers larger than a single cell. That is, only two variables are involved for those covers instead of three, yielding a "smaller" function and therefore a simpler circuit.
- K-Maps are useful for minimizing functions of up to six variables. Beyond six variables logical adjacency is hard to perceive, so using them is no longer convenient.



$$z = \overline{a}\cdot\overline{b}\cdot\overline{c} + a\cdot b + b\cdot c$$





MINIMIZATION WITH ESPRESSO SOFTWARE (1)

- The Espresso logic minimizer can be considered an aid to manual synthesis. It has also been used as a minimizer inside CAE software. It originated at the University of California, Berkeley as an aid to minimizing combinational networks with many input variables.
- Espresso is a heuristic minimizer that bases the minimization process on a few empirical rules in addition to the properties of Boolean algebra. The result may be sub-optimal, yet it can quickly produce excellent-quality results even for networks with a large number of input variables.
- The Espresso logic minimizer is widely available online and can be found both as source code and as a *build* for various platforms (https://github.com/Gigantua/Espresso).

MINIMIZATION WITH ESPRESSO SOFTWARE (2)

- Minimization of function z.

File z_sp.pla

```
.i 3
.o 1
.p 4

.ilb a b c
.ob z

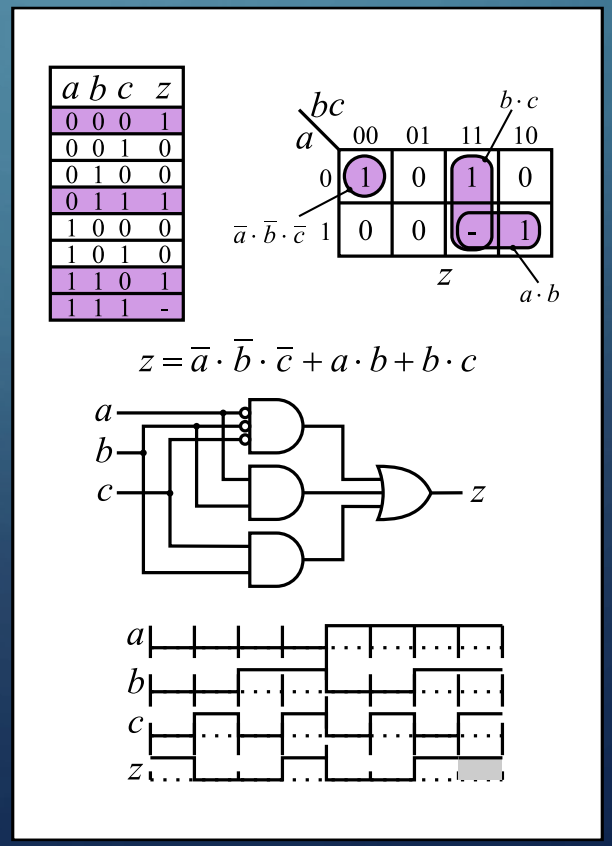
000 1
011 1
110 1
111 -
.e
```

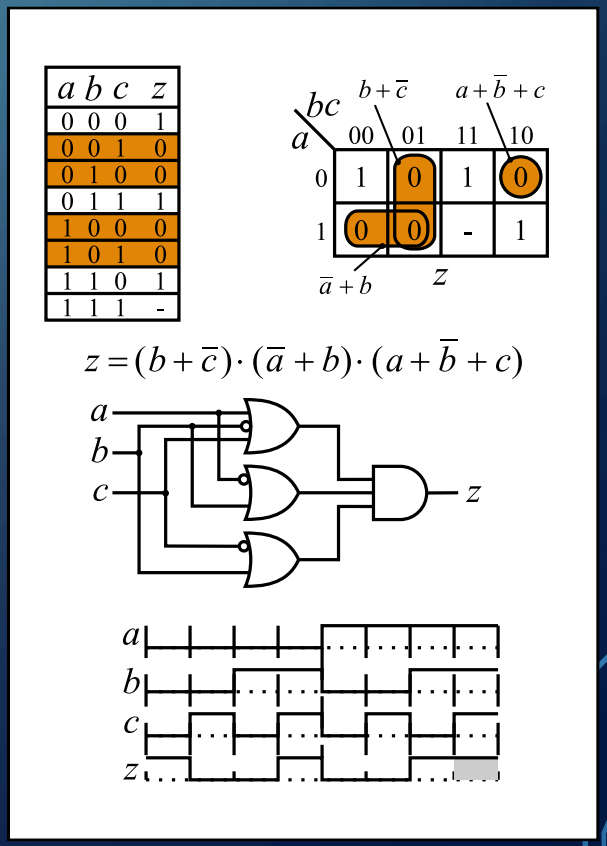
File

```
.i 3
.o 1
.p 5
.type r

.ilb a b c
.ob z

001 0
010 0
100 0
101 0
111 -
.e
```





MINIMIZATION WITH ESPRESSO SOFTWARE (3)

- Minimization of function z defined on the minterms. Minimizer output for z_sp.pla

```
.i 3
.o 1
.p 4

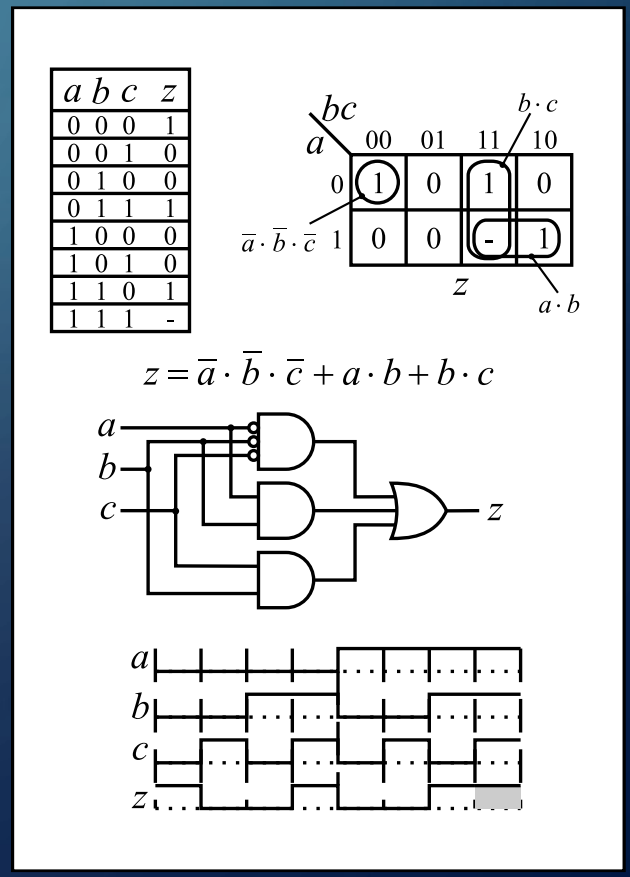
.ilb a b c
.ob z

000 1
011 1
110 1
111 -
.e
```

```
C:\Espresso>espresso z_sp.pla
.i 3
.o 1
.ilb a b c
.ob z
.p 3
000 1
11- 1
-11 1
.e

C:\Espresso>espresso -o eqntott z_sp.pla
z = (!a & !b & !c) | (a & b) | (b & c);

C:\Espresso>
```



MINIMIZATION WITH ESPRESSO SOFTWARE (4)

- Minimization of function z defined on the maxterms. Minimizer output for z_ps.pla

```
.i 3
.o 1
.p 5
.type r
.phase 0

.ilb a b c
.ob z

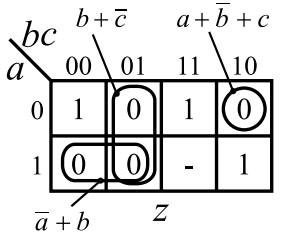
001 0
010 0
100 0
101 0
111 -
.e
```

```
C:\Espresso>espresso z_ps.pla
.i 3
.o 1
.ilb a b c
.ob z
#.phase 0
.p 3
010 1
10- 1
-01 1
.e

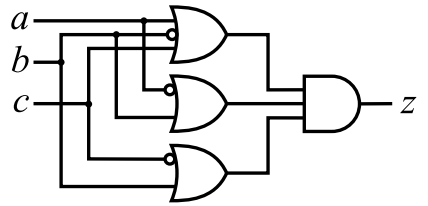
C:\Espresso>espresso -o eqntott z_ps.pla
z = (!a & b & !c) | (a & !b) | (!b & c);

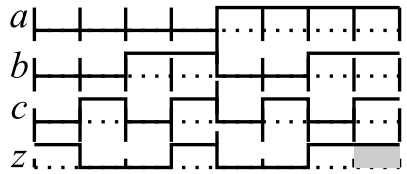
C:\Espresso>
```

a	b	c	z
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	-	-

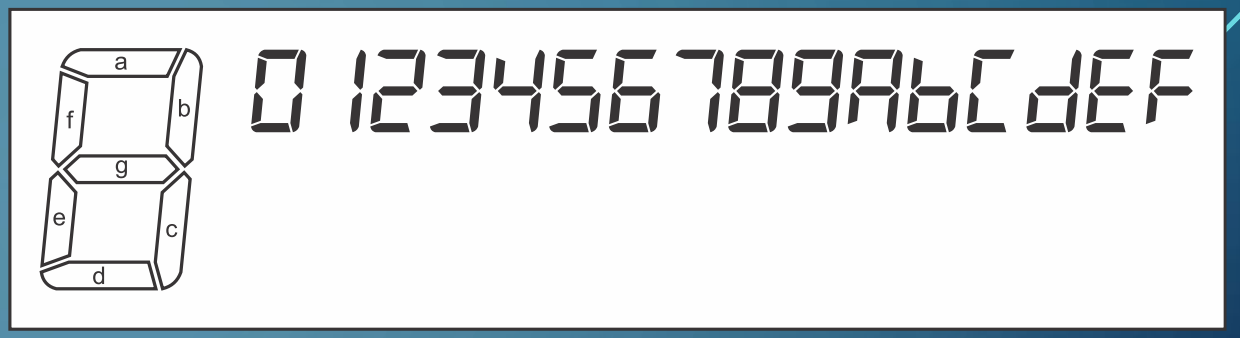


$$z = (b + \bar{c}) \cdot (\bar{a} + b) \cdot (a + \bar{b} + c)$$





WHY ESPRESSO?




```
# 7seg.pla
.i 4
.o 7
.p 16
.ilb i3 i2 i1 i0
.ob a b c d e f g
0000 1111110
0001 0110000
0010 1101101
0011 1111001
0100 0110011
0101 1011011
0110 1011111
0111 1110000
1000 1111111
1001 1110011
1010 1110111
1011 0011111
1100 1001110
1101 0111101
1110 1001111
1111 1000111
.e
```

```
C:>espresso 7seg.pla

.i 4
.o 7
.ilb i3 i2 i1 i0
.ob a b c d e f g
.p 14
1011 0001100
100- 1000000
001- 0001001
-000 0001010
0-11 1110000
0100 0110011
-00- 0110000
1101 0111101
0101 1011011
-0-0 1100100
11-0 1001110
111- 1000111
10-- 0010011
0110 1011111
.e
```

```
C:>espresso -o eqntott 7seg.pla

a = (i3 & !i2 & !i1) | (!i3 & i1 & i0) | (!i3 & i2 &! i1 & i0) | (!i2 & !i0)
  | (i3 & i2 & !i0 ) | (i3 & i2 & i1) | (!i3 & i2 & i1 & !i0);

b = (!i3 & i1 & i0) | (!i3 & i2 & !i1 & !i0) | (!i2 & !i1)
  | (i3 & i2 & !i1 & i0) | (!i2 & !i0);

c = (!i3 & i1 & i0) | (!i3 & i2 & !i1 & !i0) | (!i2 & !i1)
  | (i3 & i2 & !i1 & i0) | (!i3 & i2 & !i1 & i0) | (i3 &! i2)
  | (!i3 & i2 & i1 & !i0);

d = (i3 & !i2 &i1 & i0) | (!i3 & !i2 & i1) | (!i2 & !i1 & !i0)
  | (i3 & i2 & !i1 & i0) | (!i3 & i2 & !i1 & i0 ) | (i3&i2&!i0)
  | (!i3 & i2 & i1 & !i0);

e = (i3 & !i2 & i1 & i0) | (i3 & i2 & !i1 & i0) | (!i2 & !i0)
  | (i3 & i2 & !i0) | (i3 & i2 & i1) | (!i3 & i2 & i1 & !i0);

f = (!i2 & !i1 & !i0) | (!i3 & i2 & !i1 & !i0) | (!i3 & i2 & !i1 & i0)
  | (i3 & i2 & !i0) | (i3 & i2 & i1) | (i3 & !i2) | (!i3 & i2 & i1 & !i0);

g = (!i3 & !i2 & i1) | (!i3 & i2 & !i1 & !i0) | (i3 & i2 & !i1 & i0)
  | (!i3 & i2 & !i1 & i0) | (i3 & i2 & i1) | (i3 & !i2)
  | (!i3 & i2 & i1 & !i0);
```

ASICS (1)

- ASIC is the acronym of Application Specific Integrated Circuit.
- In the field of digital logic, these are devices designed specifically to optimize, in terms of speed and efficiency, a specific computation or processing task.
- These devices are historically developed by semiconductor companies on commission from the customer, or by the customer itself with very significant support from the manufacturer.
- Designing an ASIC is usually a long and expensive process.

ASICS (2)

- ASICs minimize the cost of the final product and are therefore justified only by very large production volumes.
- When the ASIC implements functions generic enough to be used in several products or by several customers, we speak of an ASSP (Application Specific Standard Product/Part). Because they are used in several designs, ASSPs help reduce the "non-recurring engineering costs" (NRE cost) in the development cycle.
- The impossibility of resorting every time to completely custom solutions (ASICs) or to specific standard functions (ASSPs) led semiconductor companies to develop the so-called Programmable Logic.

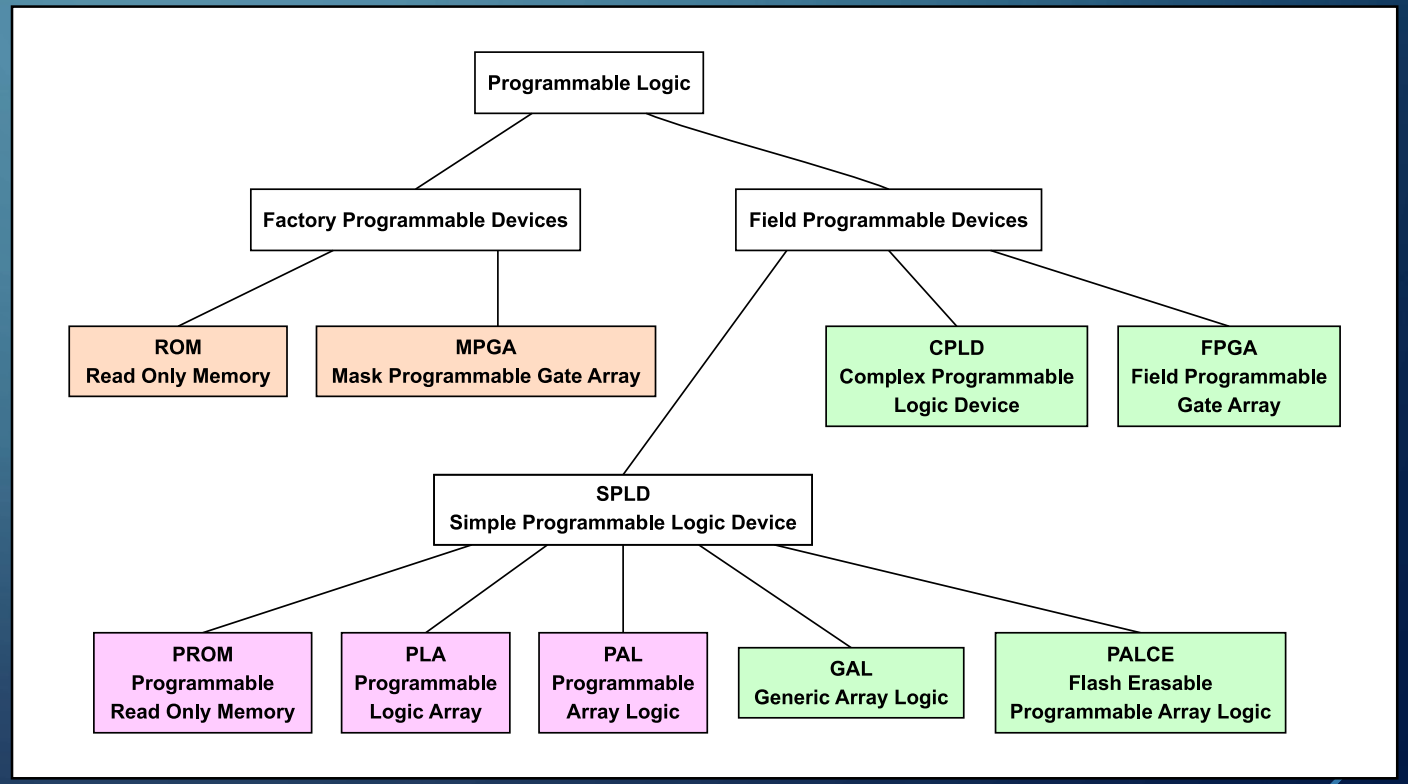
PROGRAMMABLE LOGIC (PLD)

- Programmable Logic Devices (PLDs) are devices whose internal architecture is predefined by the manufacturer.
- Compared with ASICs, PLDs have the advantage of greatly reduced development costs and times, and of not requiring large minimum quantities of parts to be ordered from the manufacturer.
- PLD technology has evolved in step with the manufacturing of other types of chips, following their evolution in terms of integration density as well.
- In the early days of the technology, PLDs existed with SSI (Small Scale Integration) levels of integration, which implemented fairly limited functionality. Ever higher densities were then adopted, such as MSI (Medium Scale Integration), LSI (Large Scale Integration) and then VLSI (Very Large Scale Integration). The latter is used in modern programmable logic. The largest modern user-programmable PLDs can easily host over 100 million simple logic functions (called gates).

TAXONOMY OF PROGRAMMABLE LOGIC (1)

Programmable logic can be divided into:

- "Field" programmable logic (FPDs, Field Programmable Devices), i.e. programmable on the production line only the first time, or programmable/updatable when already mounted on the printed circuit board (PCB).
- Logic pre-programmed by the manufacturer (Factory Programmable Devices). In general these are "Gate Arrays", i.e. devices with many logic functions that lack only the interconnections (the metallizations). They are logic to consider when the required production volumes are significant but not large enough to justify the development cost of a true ASIC. During prototyping the customer uses an FPD, then asks the manufacturer for equivalent devices pre-programmed to their specification.



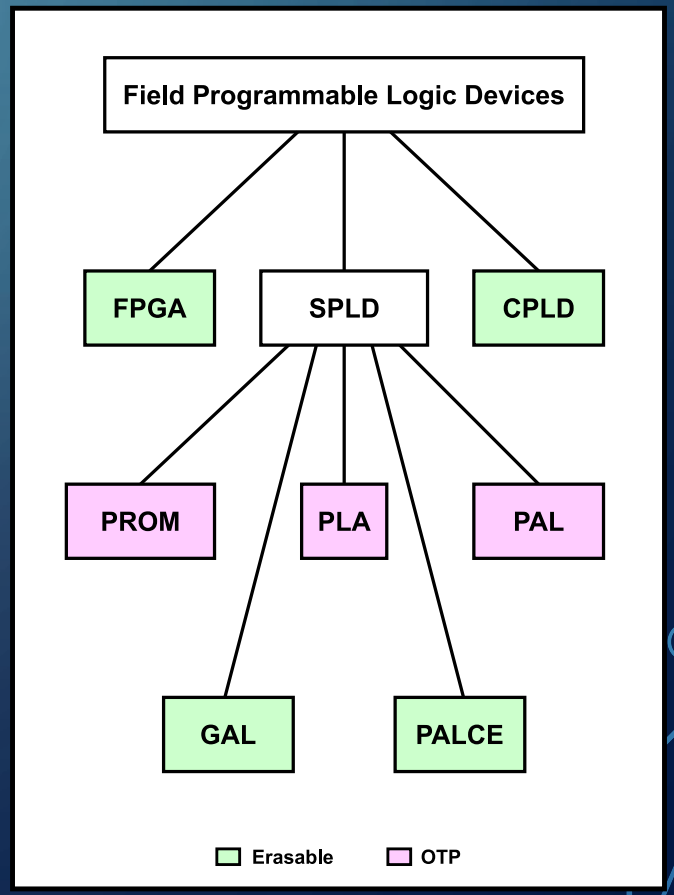
FIELD PROGRAMMABLE LOGIC (1)

SPLD: Simple Programmable Logic Devices

This group includes devices that can be programmed only once (OTP, One Time Programmable): PROMs (Programmable Read Only Memory), PLAs (Programmable Logic Array) and PALs (Programmable Array Logic). More modern, reprogrammable versions of PALs also exist: the best known are GAL and PALCE.

CPLD

That is, Complex Programmable Logic Devices. These are devices that host several interconnected macro-blocks implementing the disjunctive normal form (SP canonical form), similar to erasable PALs.

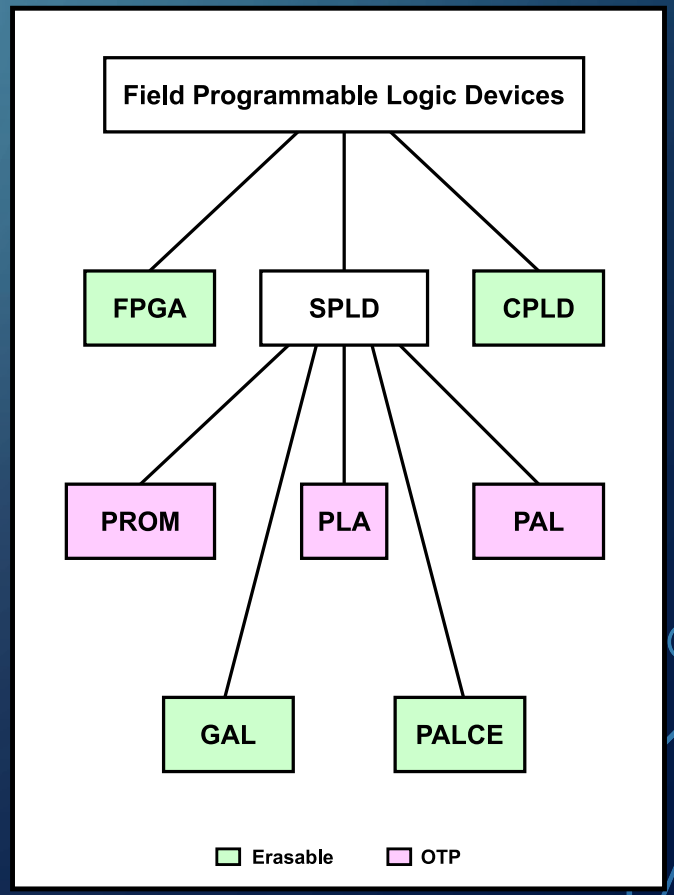


FIELD PROGRAMMABLE LOGIC (2)

FPGA: Field Programmable Gate Array

As the name itself suggests, these are programmable Gate Arrays. There are three types:

- Antifuse: the connections between metallizations are made using a tiny segment of amorphous (non-crystalline) silicon that normally exhibits a very high resistance ($> 10^6$ Ω). As soon as appropriate voltages and currents are applied, the amorphous silicon converts into conductive polycrystalline silicon.
- Static RAM: once a configuration is loaded, the RAM bits drive transistors that act as switches connecting the metallizations and configure the LUTs (LookUp Tables).
- FLASH/EPROM: like the static RAM versions, the non-volatile memory bits are used to enable the connections between the metallizations and to configure the LUTs.



CLASSIC DEVICES IMPLEMENTING THE SOP CANONICAL FORMS (SSI)

- ROM: Read Only Memory

  They have an output pattern predetermined by the manufacturer.

- PROM: Programmable Read Only Memory

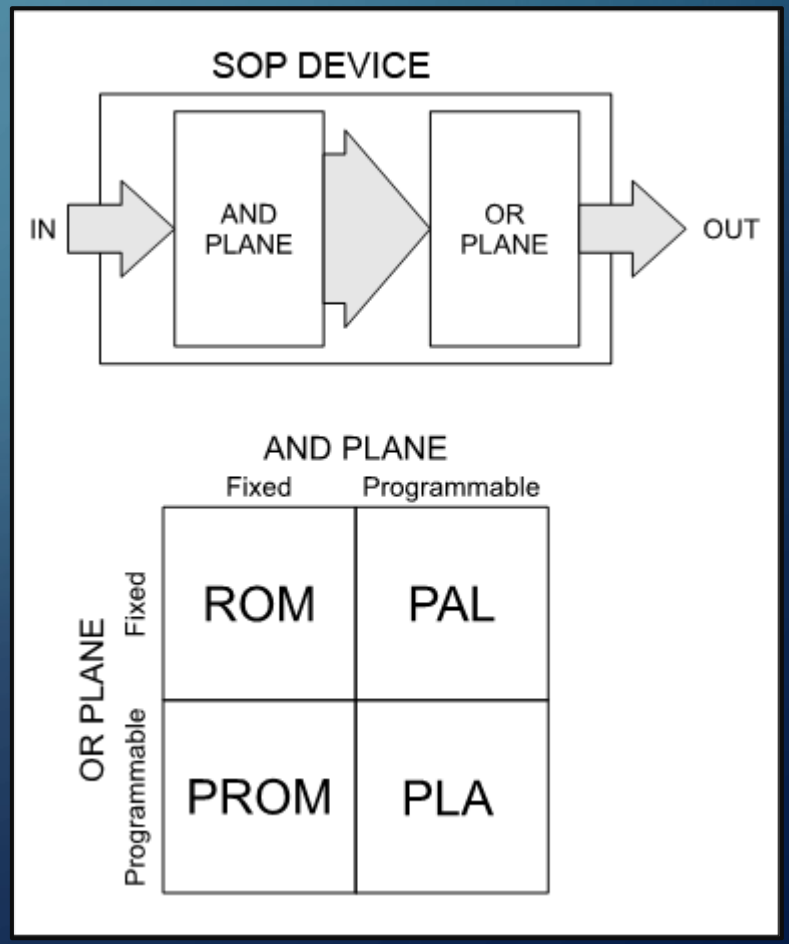
  Only the OR plane is programmable.

- PAL: Programmable Array Logic

  The AND plane is programmable and the OR plane is predetermined by the manufacturer. They implement only SP forms (sums of products or sums of minterms).

- PLA: Programmable Logic Array

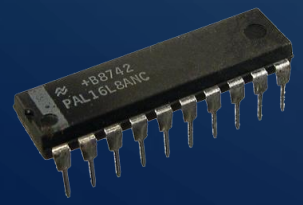
  Both planes (AND and OR) are programmable.



EXAMPLE WITH PAL16L8 (SSI)

PALs (Programmable Array Logic) are by now obsolete programmable logic devices that were invented by John Birkner and introduced by MMI back in 1978.

The PAL16L8 is a device for combinational (or asynchronous sequential) logic. Historically built in bipolar technology (with BJT transistors), it was a hugely successful device, used by the early minicomputer makers (DEC, Data General, etc.) to optimize their systems.

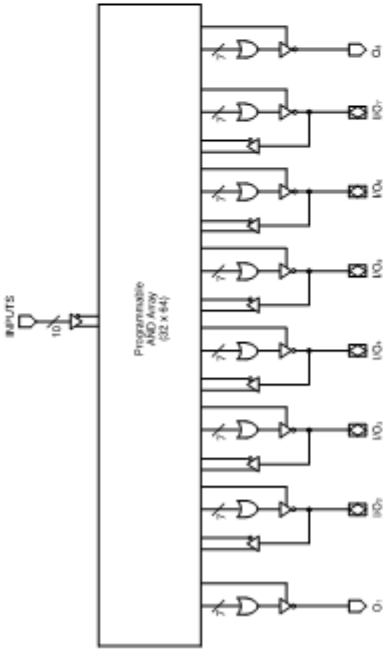


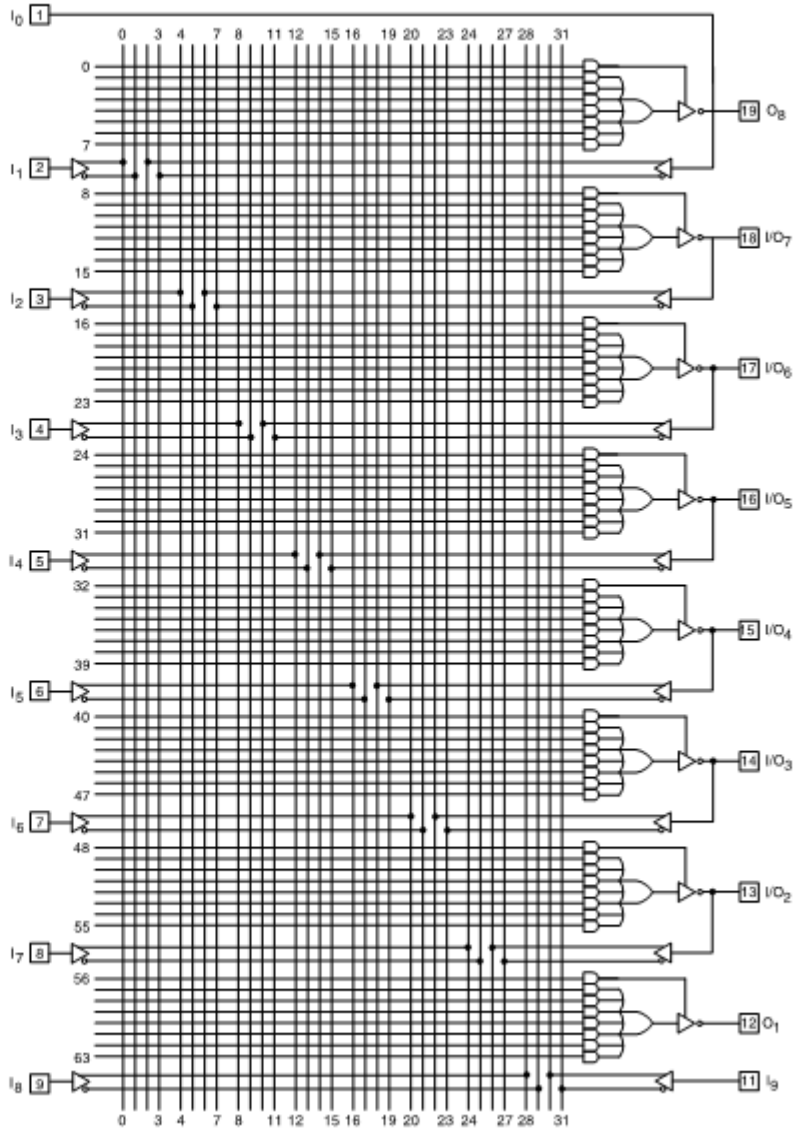
EXAMPLE OF PAL PROGRAMMABLE LOGIC



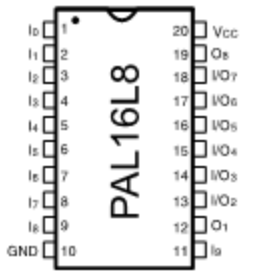
PIN DESIGNATIONS

Vcc	=	Supply Voltage
GND	=	Ground
I	=	Input
I/O	=	Input/Output
O	=	Output



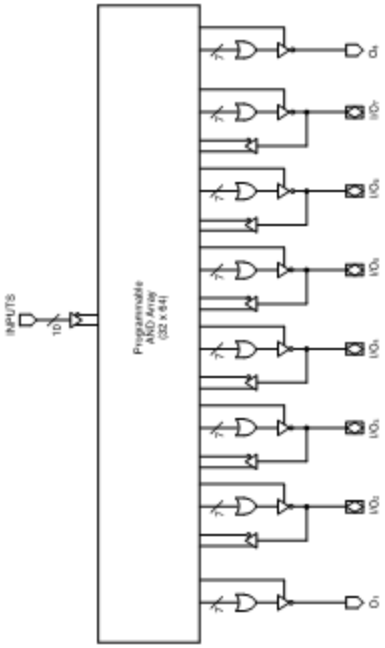


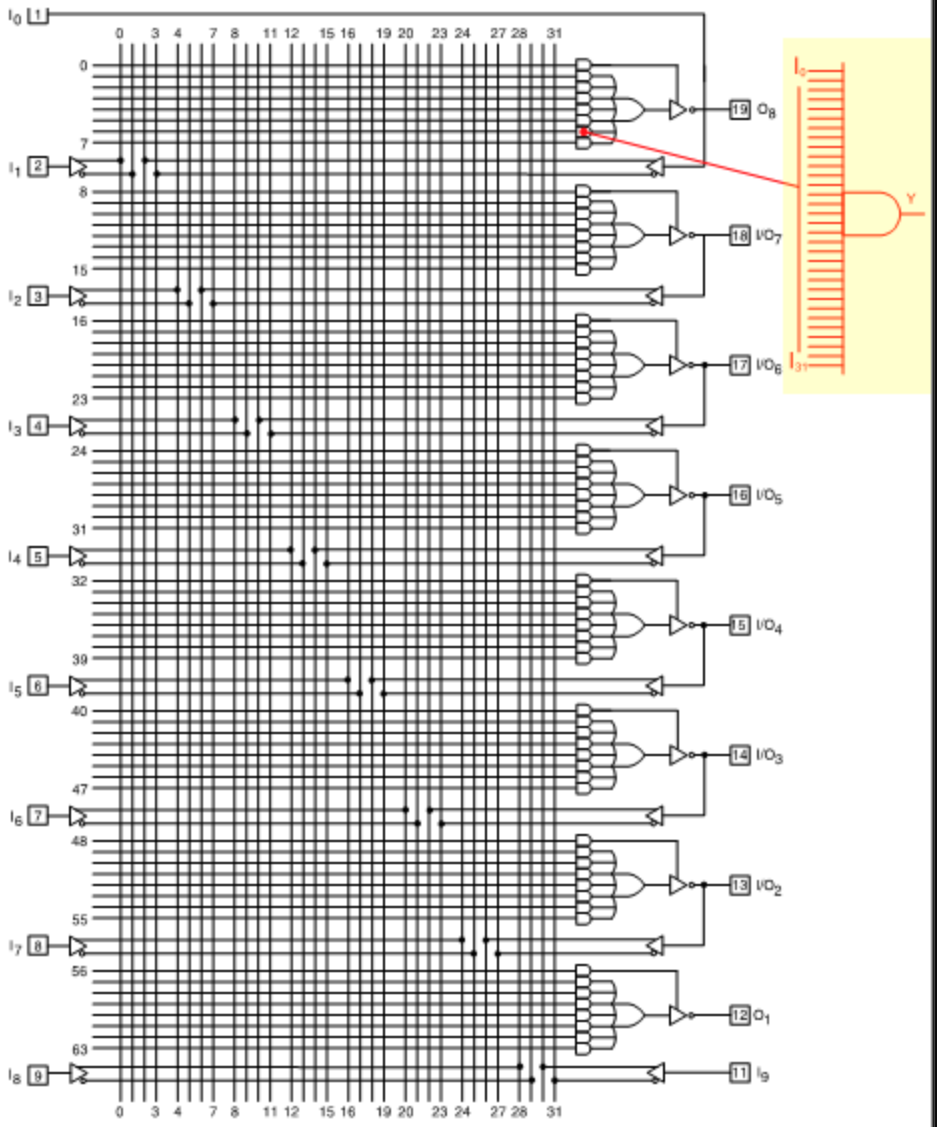
EXAMPLE OF PAL PROGRAMMABLE LOGIC



PIN DESIGNATIONS

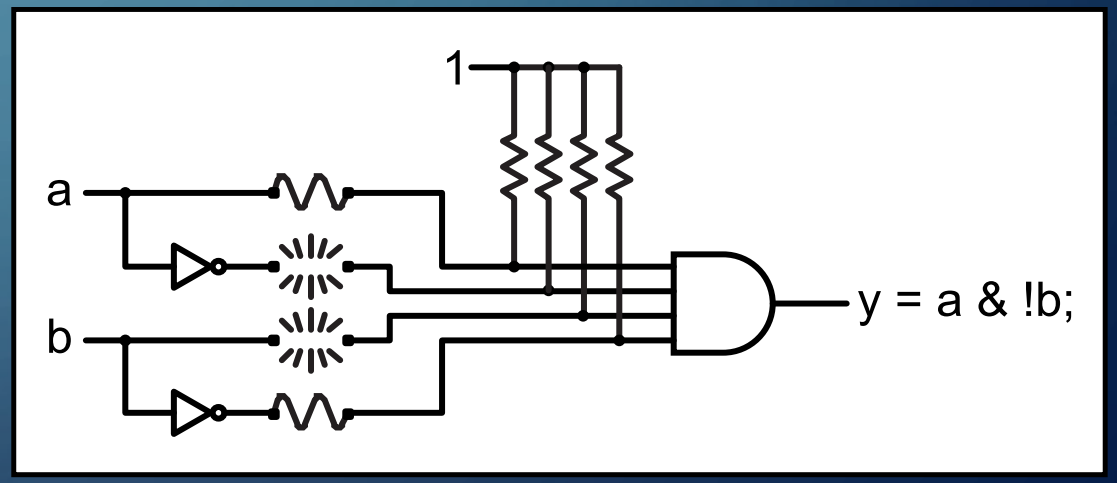
V_{CC}	=	Supply Voltage
GND	=	Ground
I	=	Input
I/O	=	Input/Output
O	=	Output





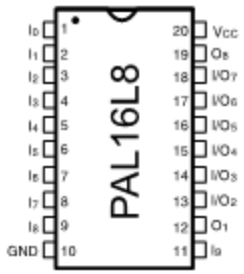
PROGRAMMING OTP SPLDS

- In devices such as PALs, PLAs and PROMs, programming is done by blowing tiny fuses. Specifically for PAL devices, there are fuses in series with each signal and its complement on the path that reaches the AND gate.
  Initially all the fuses are intact.



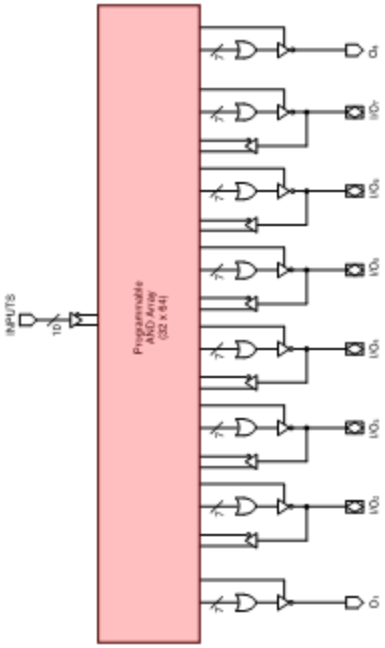
When a fuse is blown as a result of programming, the signal can no longer reach the AND gate. The input is therefore connected to 1 through a so-called pull-up resistor.
In practice, this means the AND output will depend only on the inputs whose fuses have remained intact. In the example, note that only a and $\overline{b}$ can make up the term y.

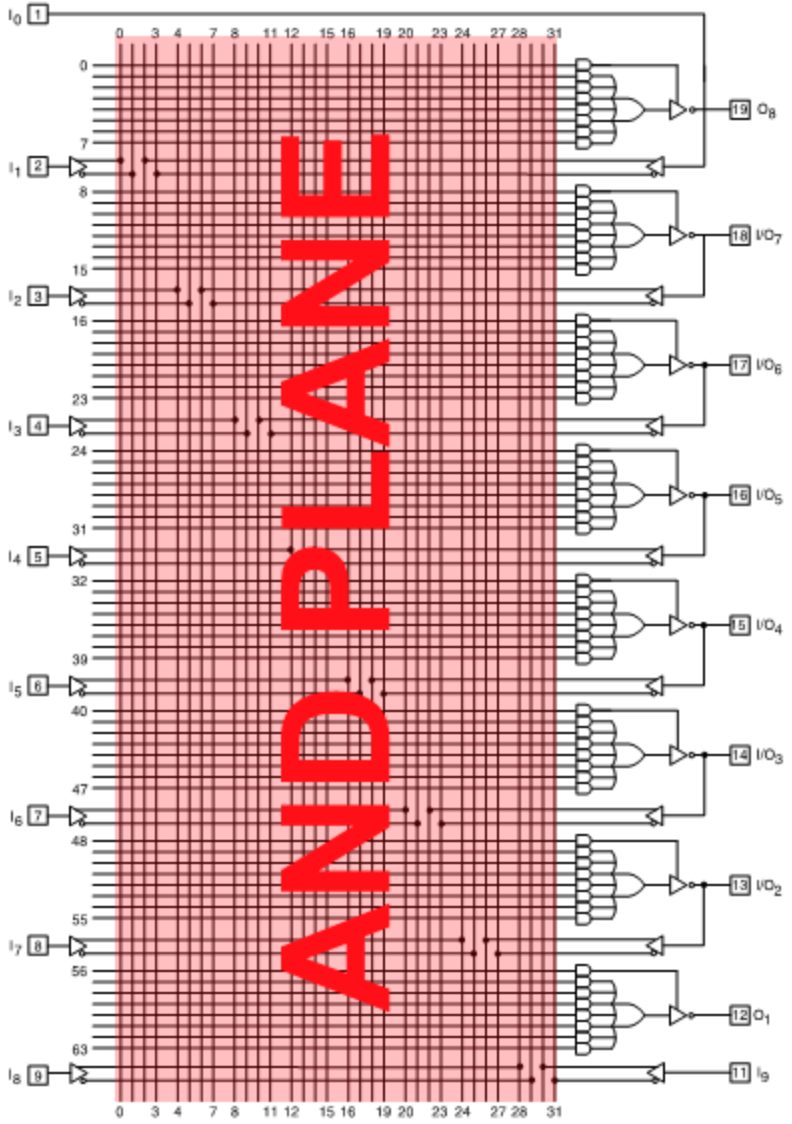
EXAMPLE OF PAL PROGRAMMABLE LOGIC



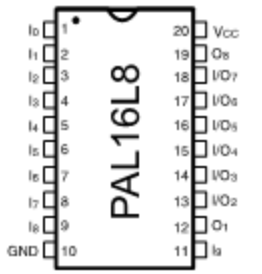
PIN DESIGNATIONS

V_{CC}	=	Supply Voltage
GND	=	Ground
I	=	Input
I/O	=	Input/Output
O	=	Output



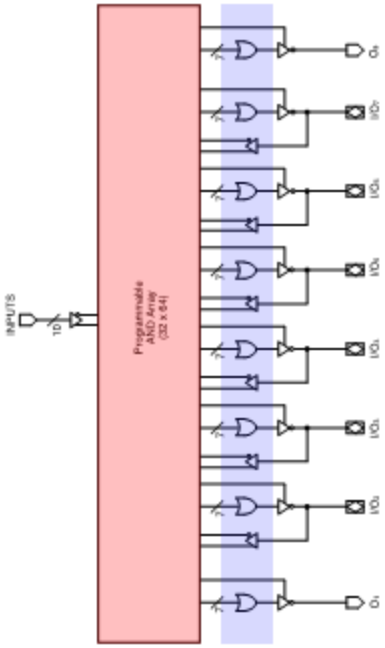


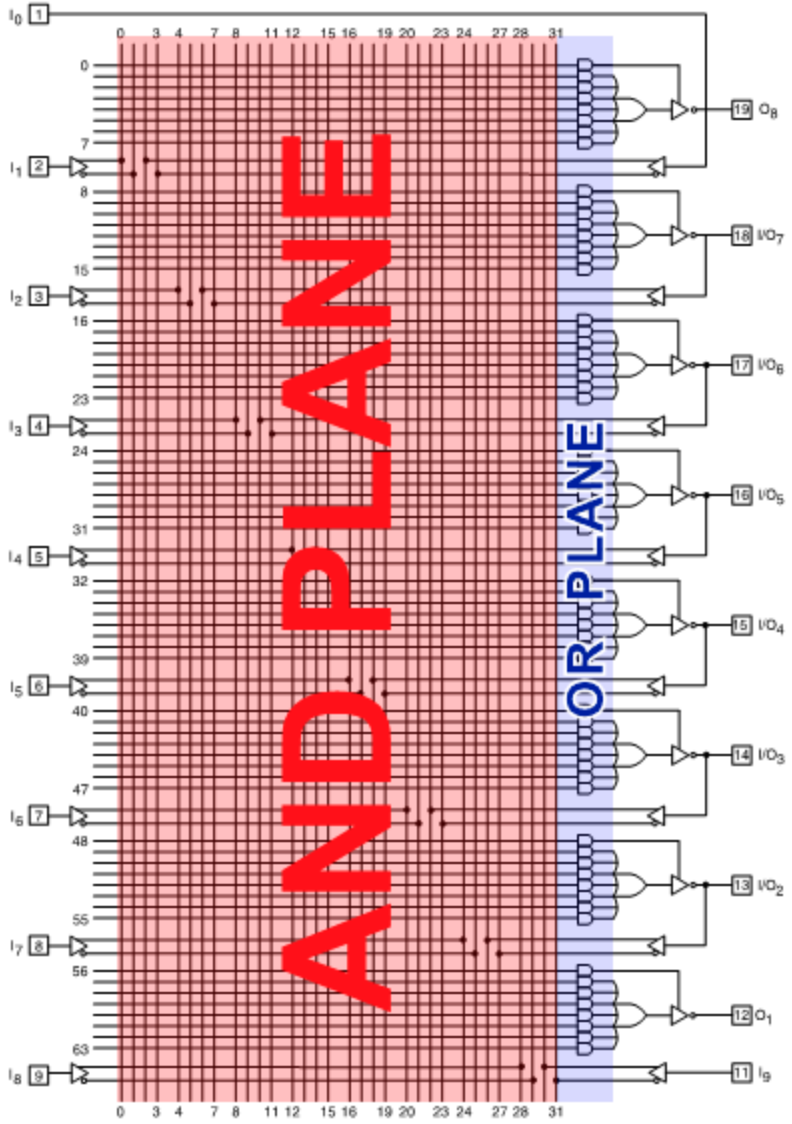
EXAMPLE OF PAL PROGRAMMABLE LOGIC



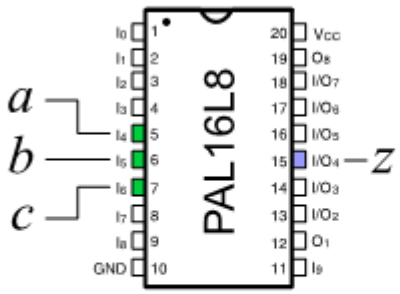
PIN DESIGNATIONS

V_{CC}	=	Supply Voltage
GND	=	Ground
I	=	Input
I/O	=	Input/Output
O	=	Output



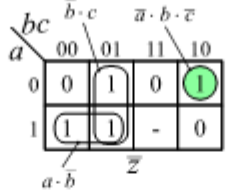


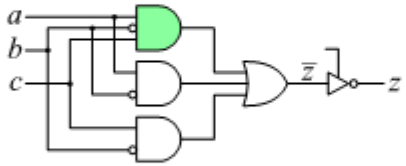
EXAMPLE OF PAL PROGRAMMABLE LOGIC



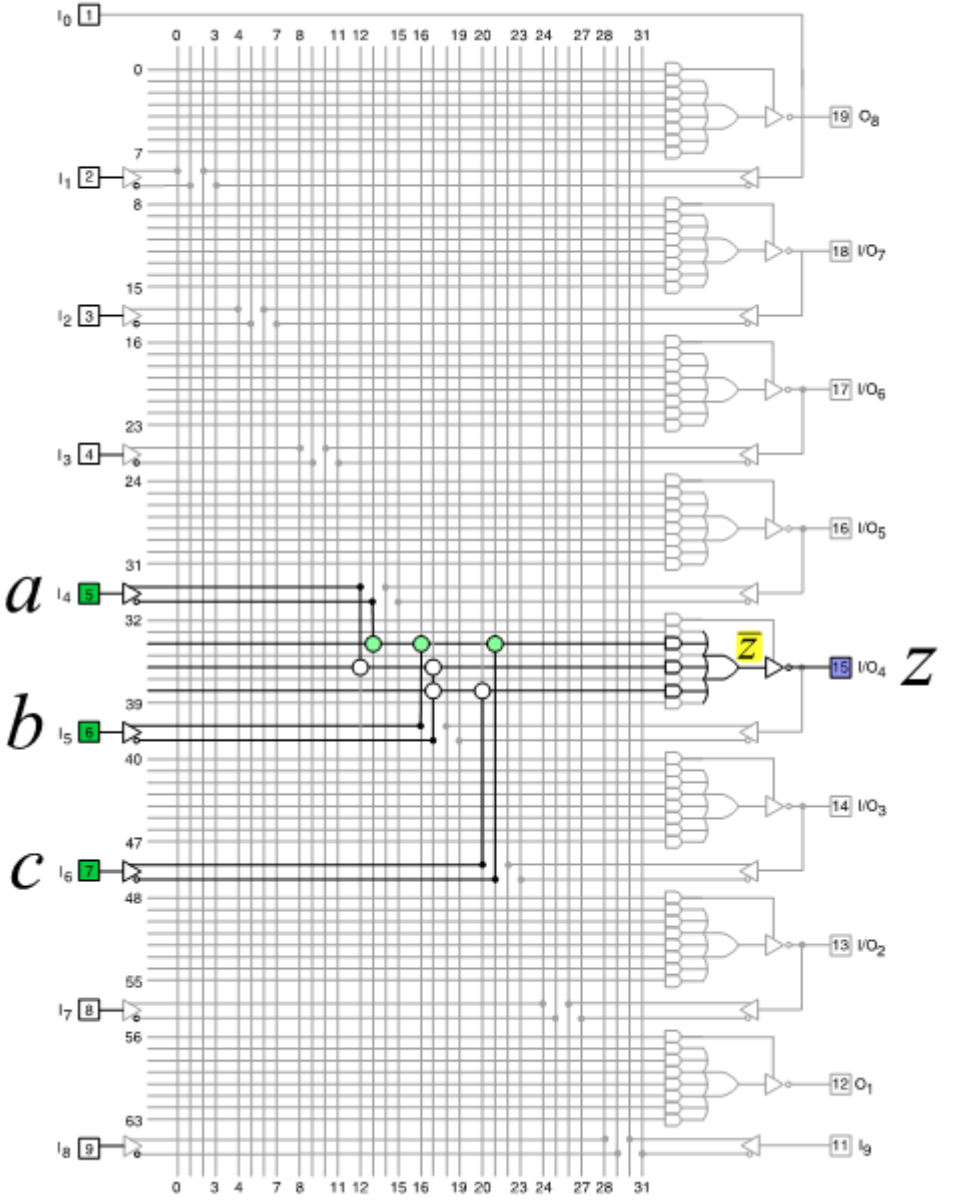
$$\overline{z} = \overline{a} \cdot b \cdot \overline{c} + a \cdot \overline{b} + \overline{b} \cdot c$$

a	b	c	$\overline{z}$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	-	-

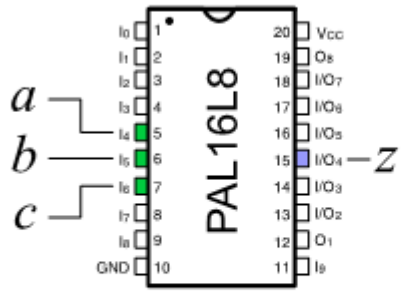






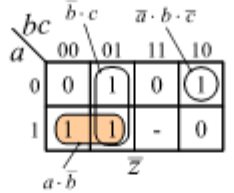


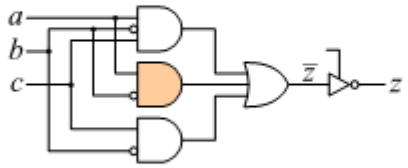
EXAMPLE OF PAL PROGRAMMABLE LOGIC



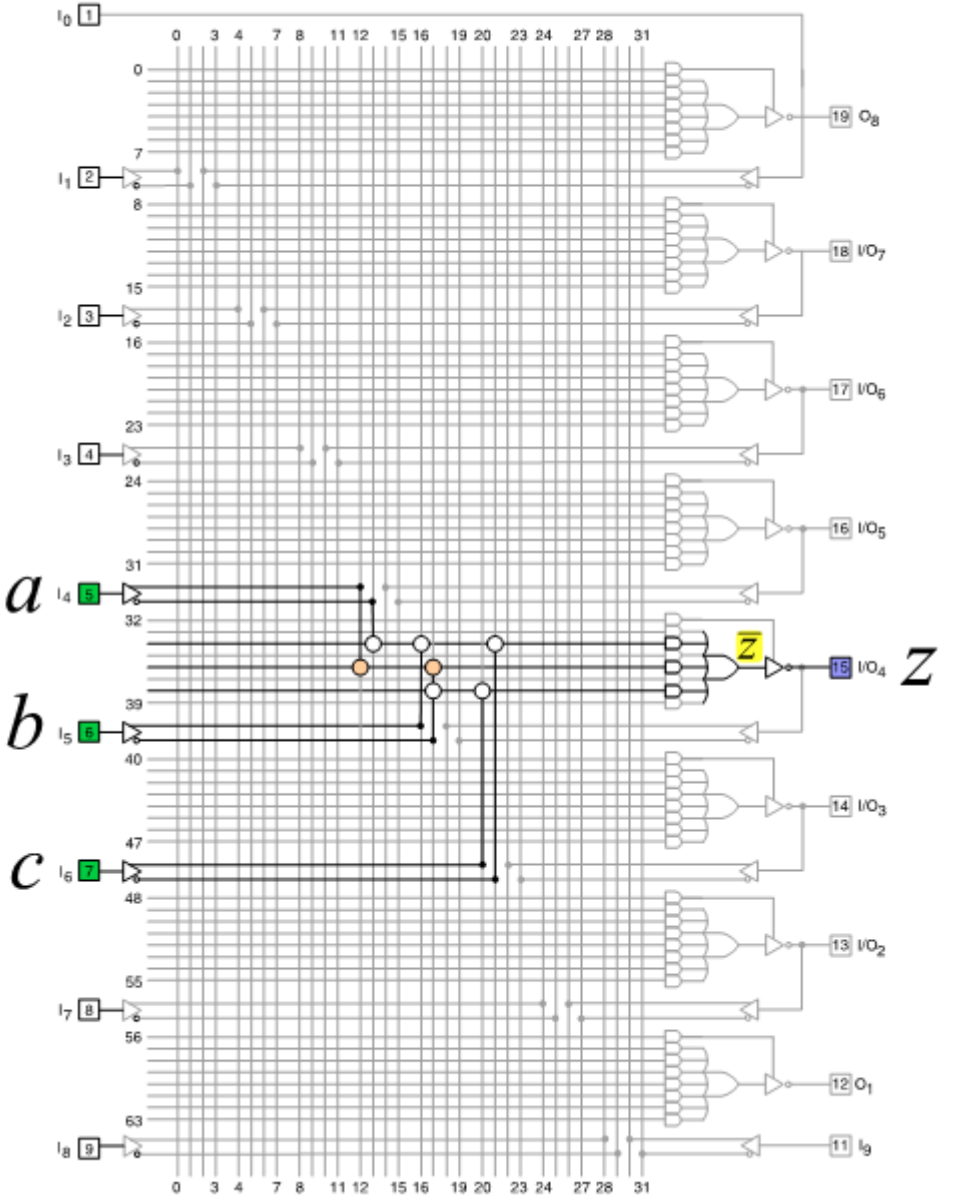
$$\overline{z} = \overline{a} \cdot b \cdot \overline{c} + a \cdot \overline{b} + \overline{b} \cdot c$$

a	b	c	$\overline{z}$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	-	-

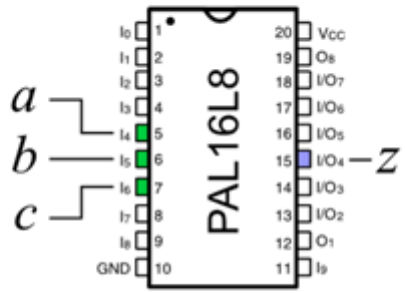






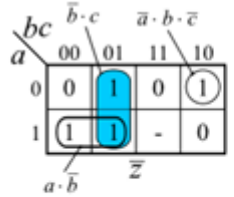


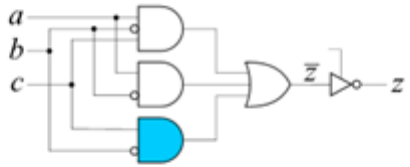
EXAMPLE OF PAL PROGRAMMABLE LOGIC

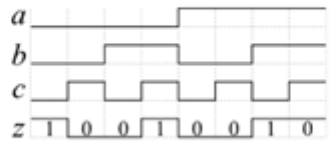


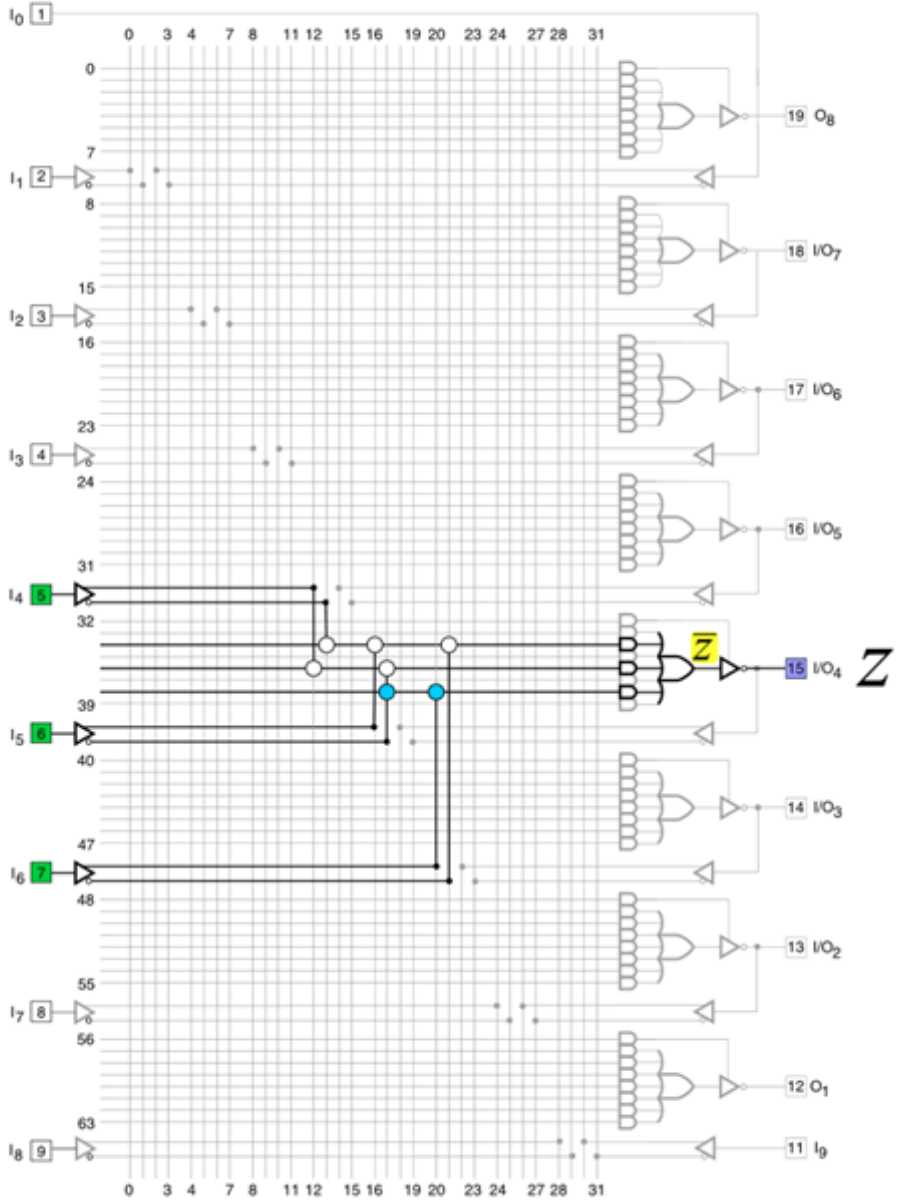
$$\overline{z} = \overline{a} \cdot b \cdot \overline{c} + a \cdot \overline{b} + \overline{b} \cdot c$$

a	b	c	$\overline{z}$
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	-



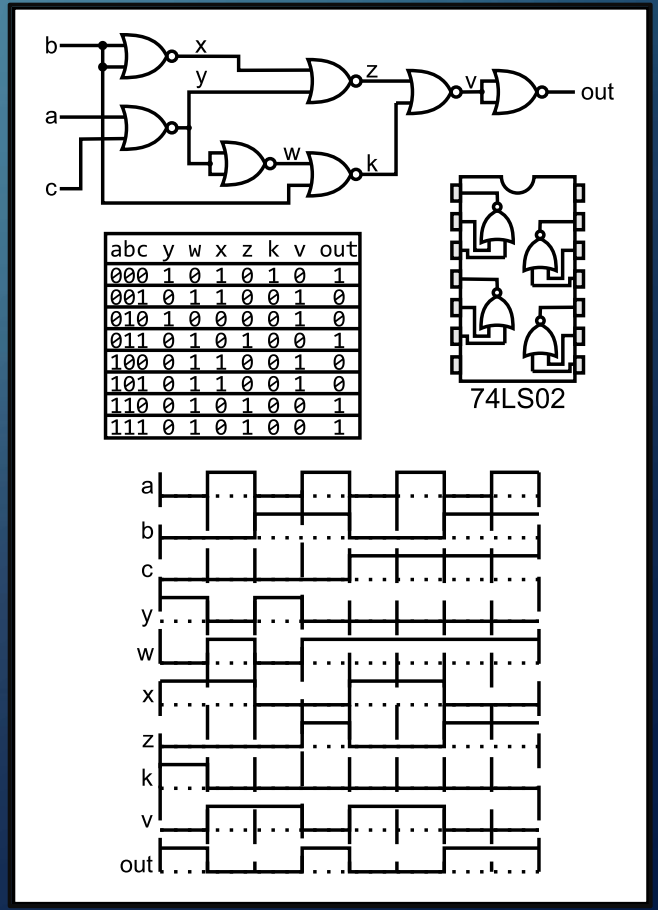






MULTILEVEL COMBINATIONAL LOGIC NETWORKS

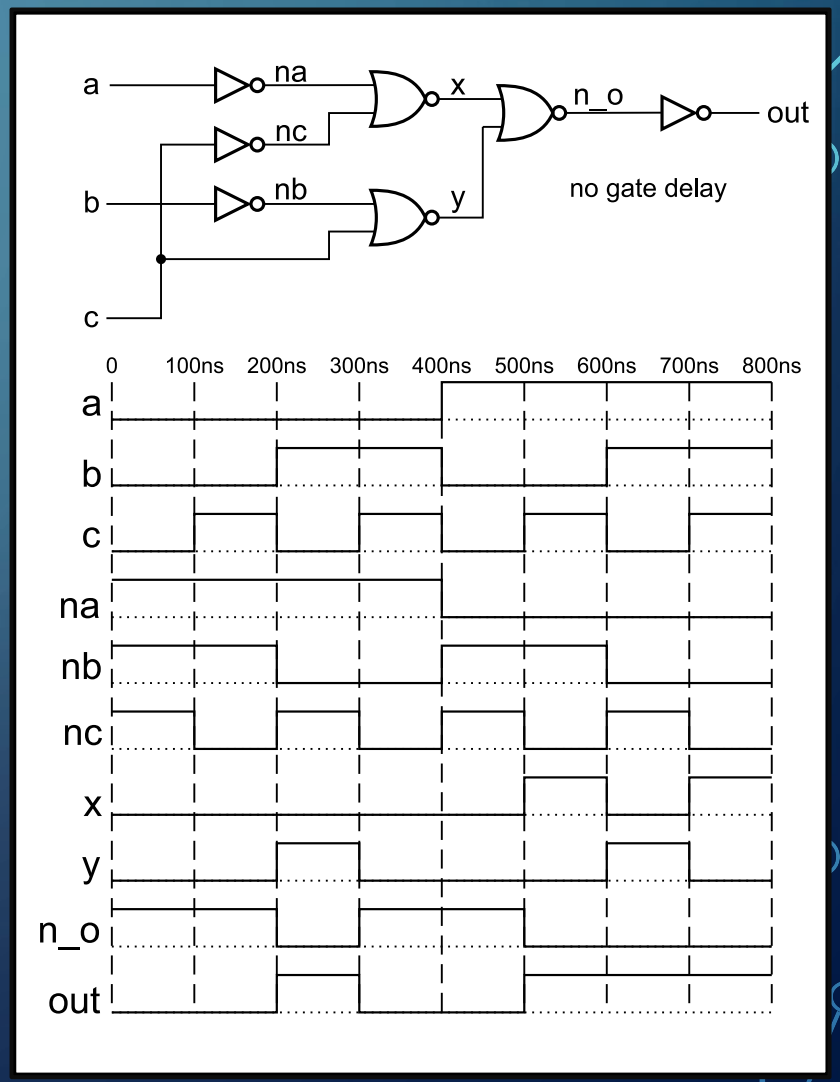
- They can use simple logic gates or canonical forms connected in cascade. The time the outputs take to settle when the inputs change can be longer and show less consistency.
- Multilevel networks are typical of modern programmable devices (CPLDs and FPGAs); they connect macrocells/LUTs (Look Up Tables, predefined logic blocks) to one another to build even very complex logic structures.



abc	y	w	x	z	k	v	out
000	1	0	1	0	1	0	1
001	0	1	1	0	0	1	0
010	1	0	0	0	0	1	0
011	0	1	0	1	0	0	1
100	0	1	1	0	0	1	0
101	0	1	1	0	0	1	0
110	0	1	0	1	0	0	1
111	0	1	0	1	0	0	1

HAZARDS

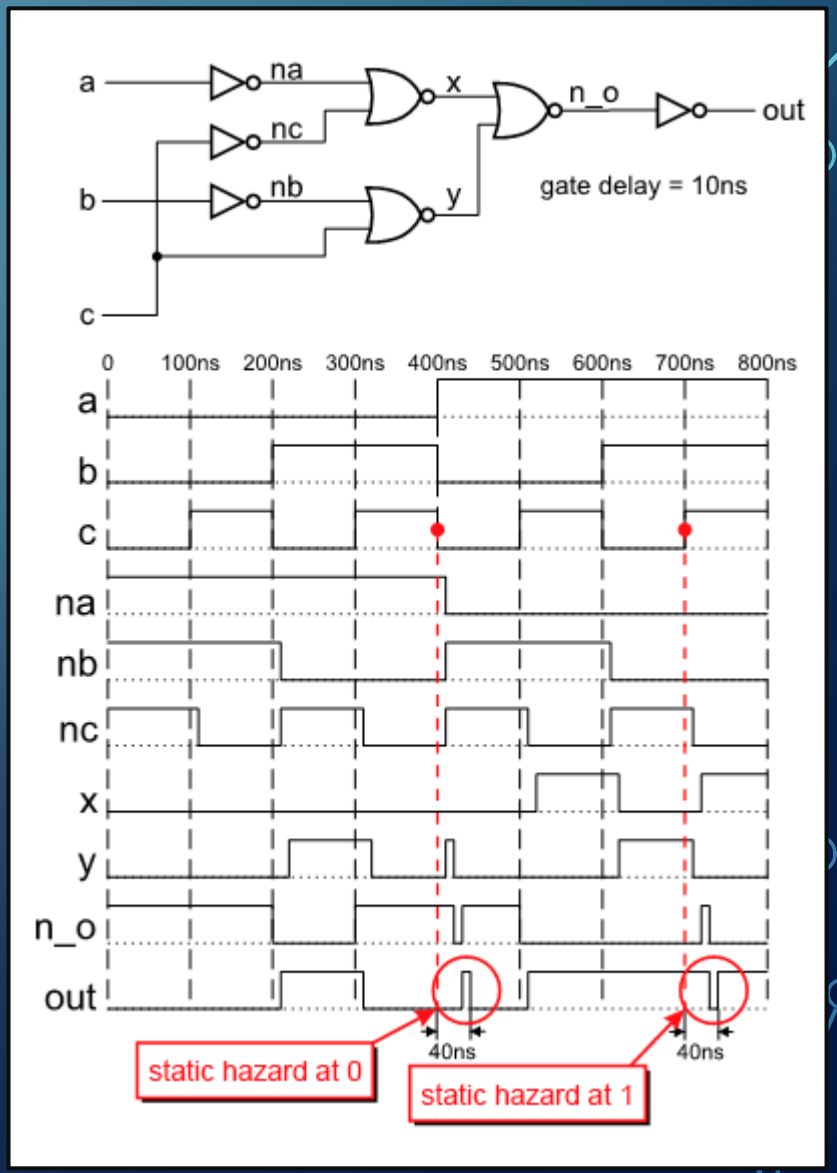
On the side you can see a multilevel network that produces the output *out* for the inputs *a*, *b* and *c*. Since the network is built with a single type of component, obtaining function out requires cascading more gates than in a canonical-form network, which in effect gives a multilevel network. Since the components exhibit no propagation delays, the behavior of this network is ideal. That is, there are no "uncertainties" in how the signals evolve over time.



STATIC HAZARDS

When the propagation times of the gates or of the physical paths (electrical connections) are not negligible, the circuit producing *out* can show uncertainty before settling on a nominal value. In the figure at the side you can see some very short pulses called hazards (glitches). These are transients that disturb the signal for a very short period. They must be taken into account at design time, since they can cause malfunctions in logic networks. Static hazards are not exclusive to multilevel networks. Canonical forms with non-ideal components or "slow" paths can also show uncertainty.

In the practical example at the side, the NOR gates of two SN74LS02 devices were used (for the inverters too), which exhibit a typical delay of 10 ns: note that the propagation delays are not really negligible compared with the speed of the applied signals (edges every 100 ns).

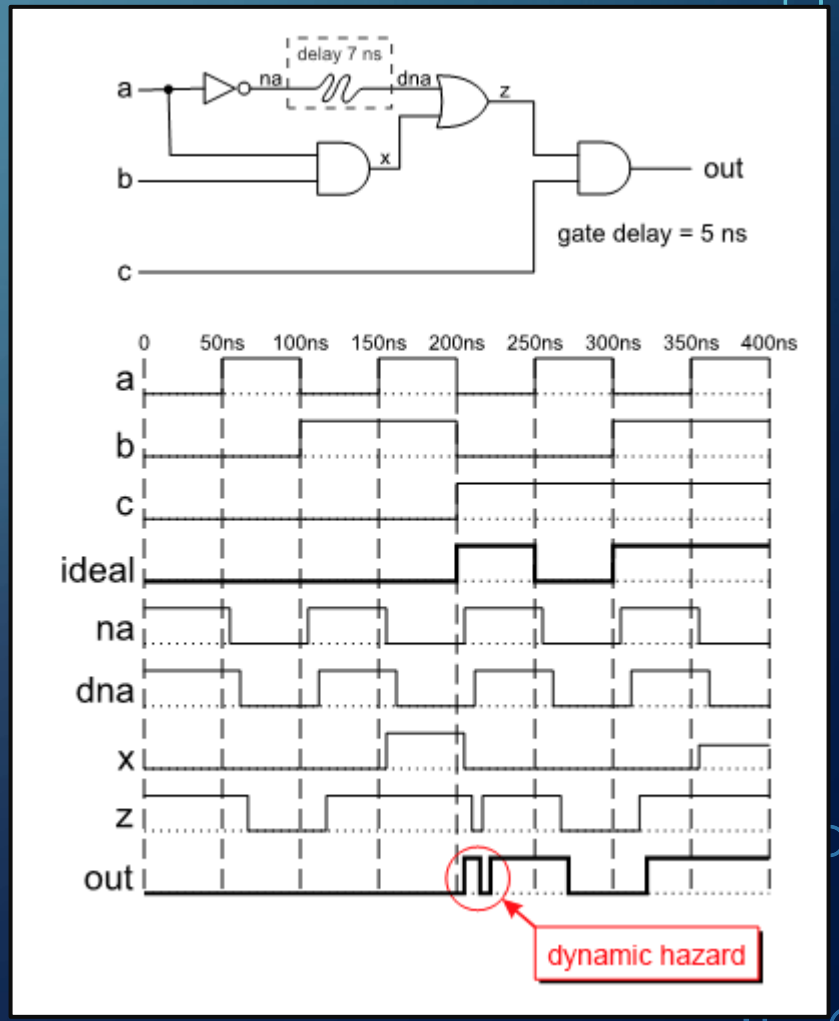


DYNAMIC HAZARDS

The circuit at the side exhibits a second type of hazard that produces an output which initially moves to the expected value (from 0 to 1) but does not remain stable: it is immediately followed by a pulse with the opposite value (zero), and only then finally settles on the desired value (one).

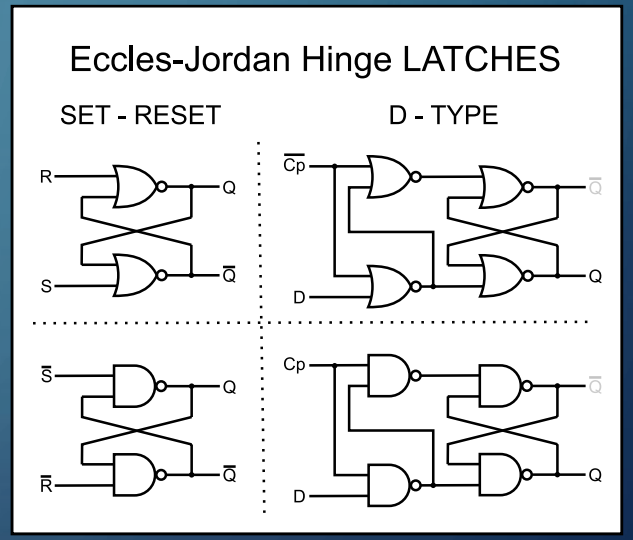
This circuit behavior is called a "dynamic hazard". Dynamic hazards appear when, following a change in one or more input signals, the output signal makes a transition to the opposite value, but instead of settling abruptly it may oscillate one or more times.

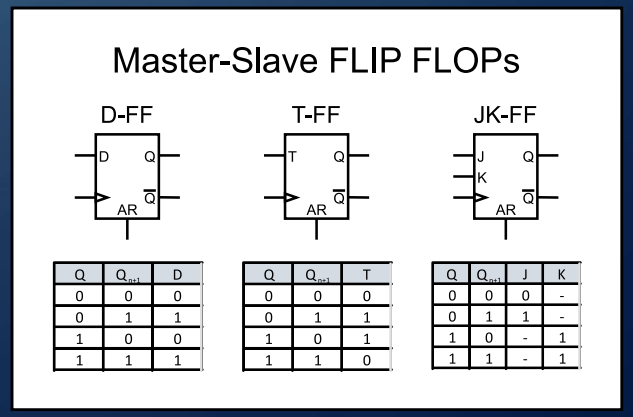
The presence of dynamic hazards causes considerable problems in the design and implementation of so-called "asynchronous sequential networks" with more than two states. This kind of network will not be covered in this presentation, although we will see an example of a very simple two-state asynchronous sequential network later on.



MEMORY ELEMENTS

- Certain circuit configurations constitute memory elements. These elements are the heart of sequential networks.
- Commonly used memory elements can be sensitive to "levels" (top box) or to "edges" (bottom box).
- "Edge"-sensitive memory elements are often used with a signal of specific cadence that sets the pace of processing (clock). They are mainly of three types, D, T and JK, and their operation is described by the so-called "excitation tables".
- They always have an internal feedback of one or more levels. They can be studied as combinational networks once the feedback loops are broken and treated as ordinary inputs.
- The combinational networks seen earlier "feed" the input variables of these memory elements in order to build larger and more complex sequential network systems.





TERNARY OR THREE-VALUED LOGIC (3VL)

- Three-valued logic is an extension of the binary propositional logic we have seen so far.
- It is useful in the analysis and simulation of binary digital circuits with *feedback*, that is, with the network's outputs fed back to its inputs.
- The values represented are three: 0, 1 and X.
- A function in three-valued logic $z = f(x_1, \ldots, x_n)$ is a rule $\{0,1,X\}^n \rightarrow \{0,1,X\}$ that maps z for every combination of $x_1, \ldots, x_n$.
- It appears as the representation of "undefined" in simulators.

NOT, OR AND AND IN THREE-VALUED LOGIC

NOT	
x	$f_1(x)$
0	1
1	0
X	X

OR	
$x_1\ x_2$	$f_2(x_1, x_2)$
0 0	0
0 1	1
0 X	X
1 0	1
1 1	1
1 X	1
X 0	X
X 1	1
X X	X

AND	
$x_1\ x_2$	$f_3(x_1, x_2)$
0 0	0
0 1	0
0 X	0
1 0	0
1 1	1
1 X	X
X 0	0
X 1	X
X X	X

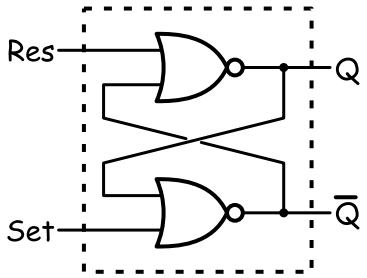
S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

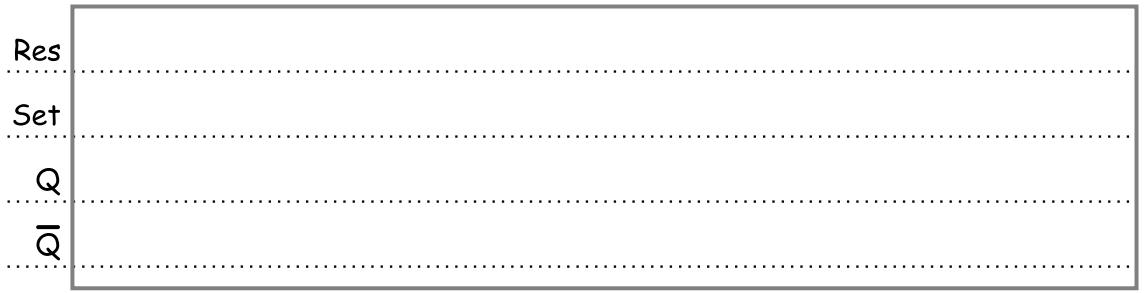
Three-valued logic is useful for studying the behavior of asynchronous logic networks.

As an example, let us look at the study of a *Set-Reset latch* built with NOR gates.

NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X



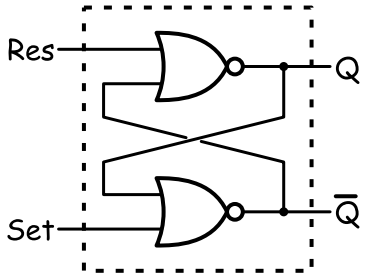


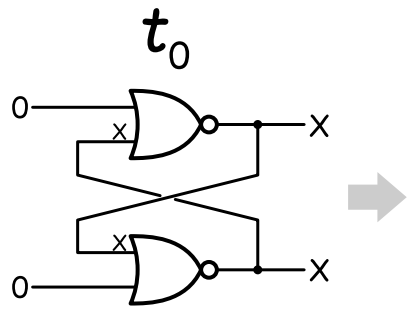
S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

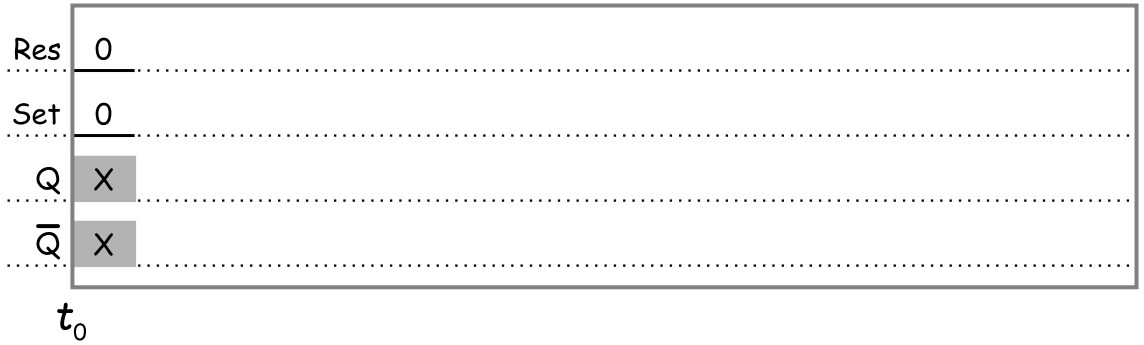
Initially (instant t_0) the output state of the NOR gates is unknown because inputs at 0 (zero) allow X (undefined) values as outputs. In practice, the state of the S-R Latch cannot be determined with certainty. It can be stated, however, that outputs Q and $\bar{Q}$ are the negation of each other under stable conditions.

NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X



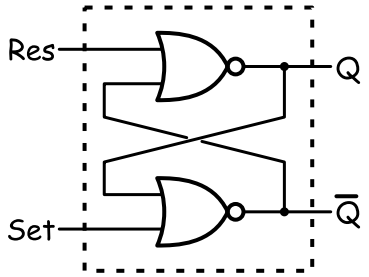


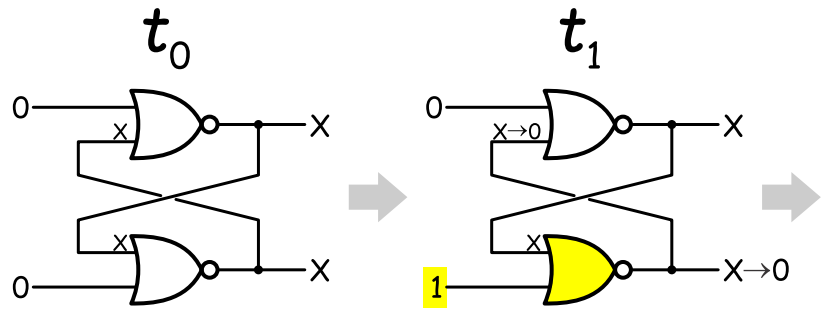


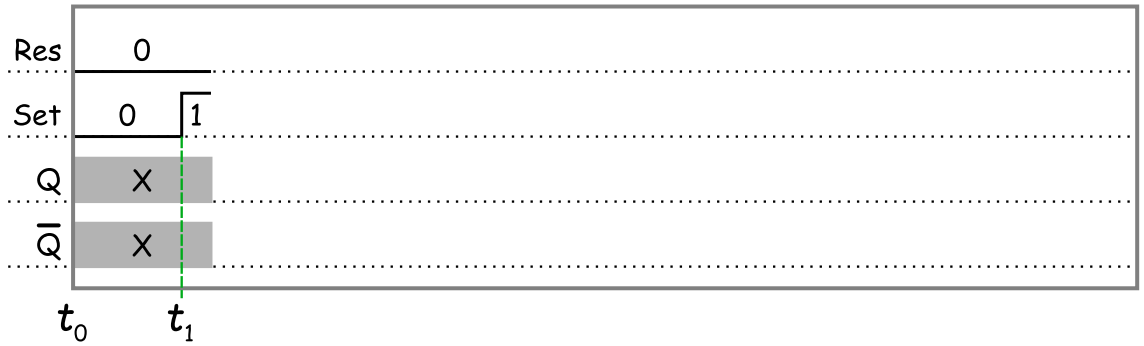
S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

As soon as the Set input is driven to 1 (instant t_1), the gate connected to it "reacts", getting ready to produce a 0 regardless of whether the second input sees X.

	NOR
$x_1\ x_2$	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X



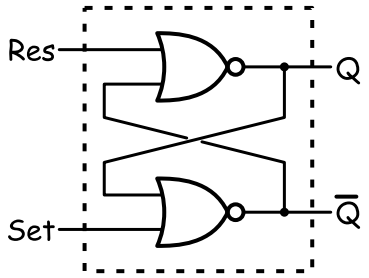


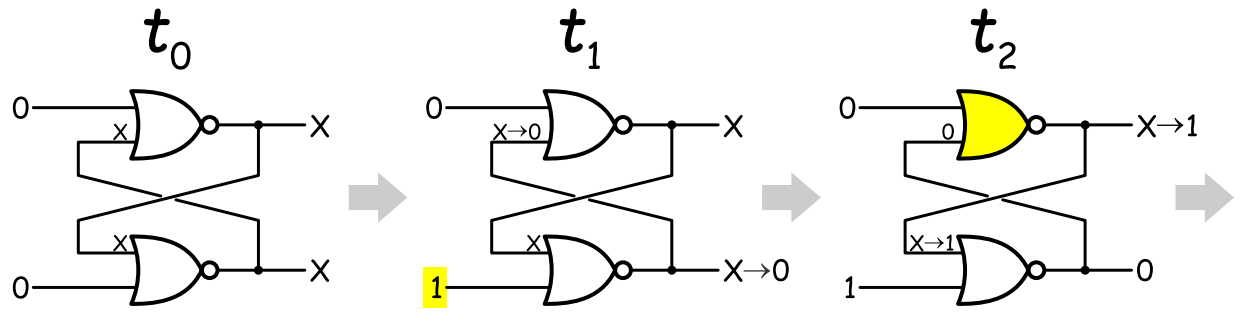


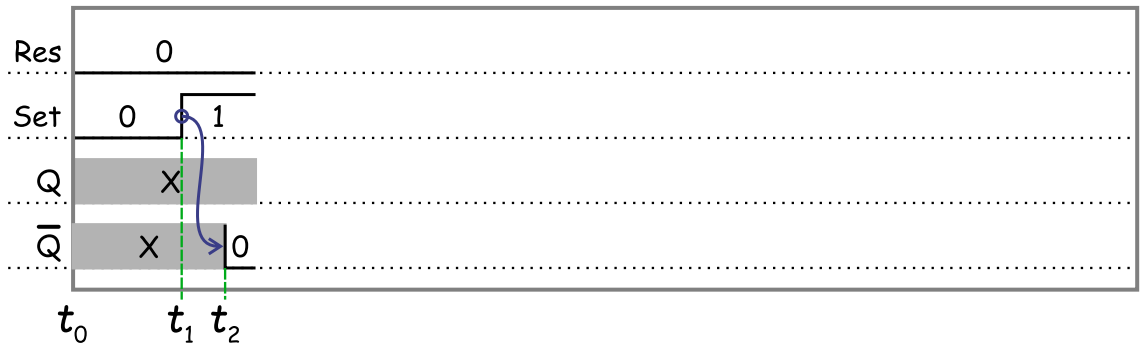
S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

At instant t_2, the NOR gate connected to output $\bar{Q}$ stably generates the value zero. However, the output change triggers the other NOR gate which, seeing the 00 configuration at its input, gets ready to generate the value 1 for output Q.

	NOR
$x_1\ x_2$	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X





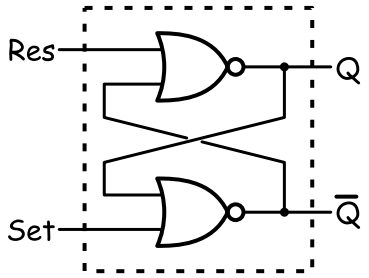


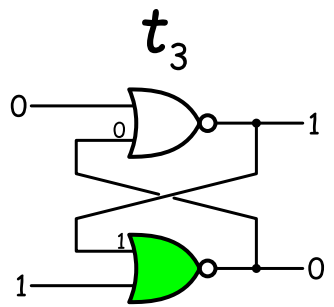
S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

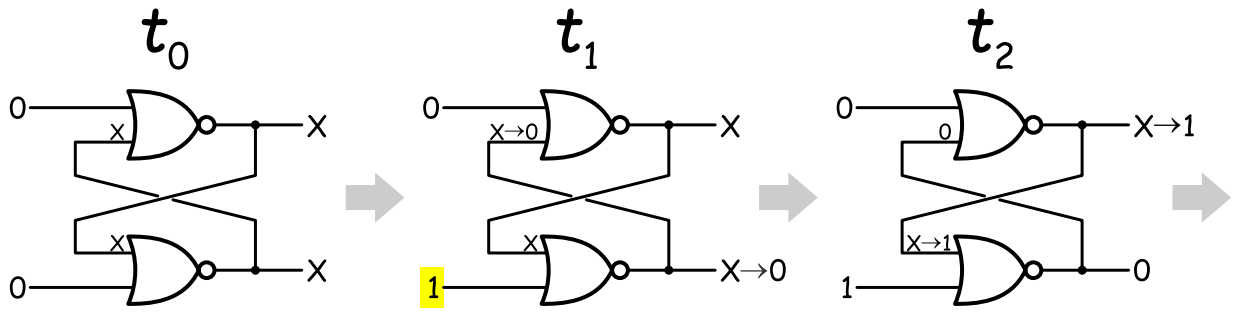
Output $\bar{Q}$ confirms the value 0 (instant t_3), reinforced by the 11 configuration at the input of the NOR gate concerned.

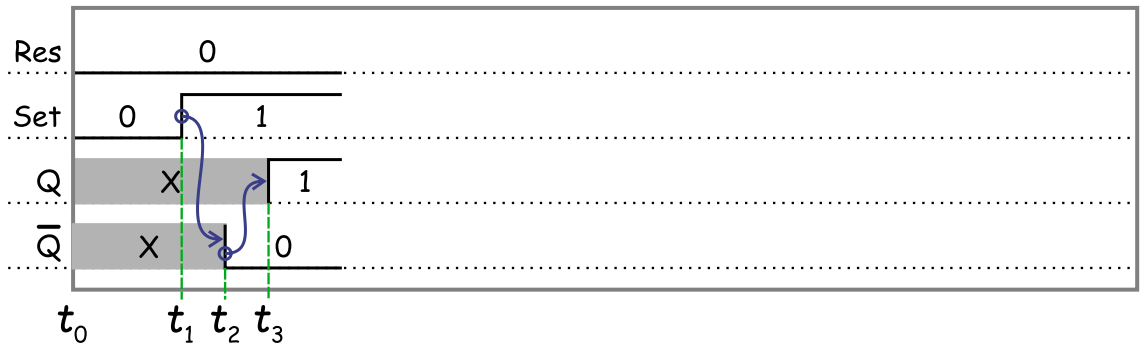
NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X







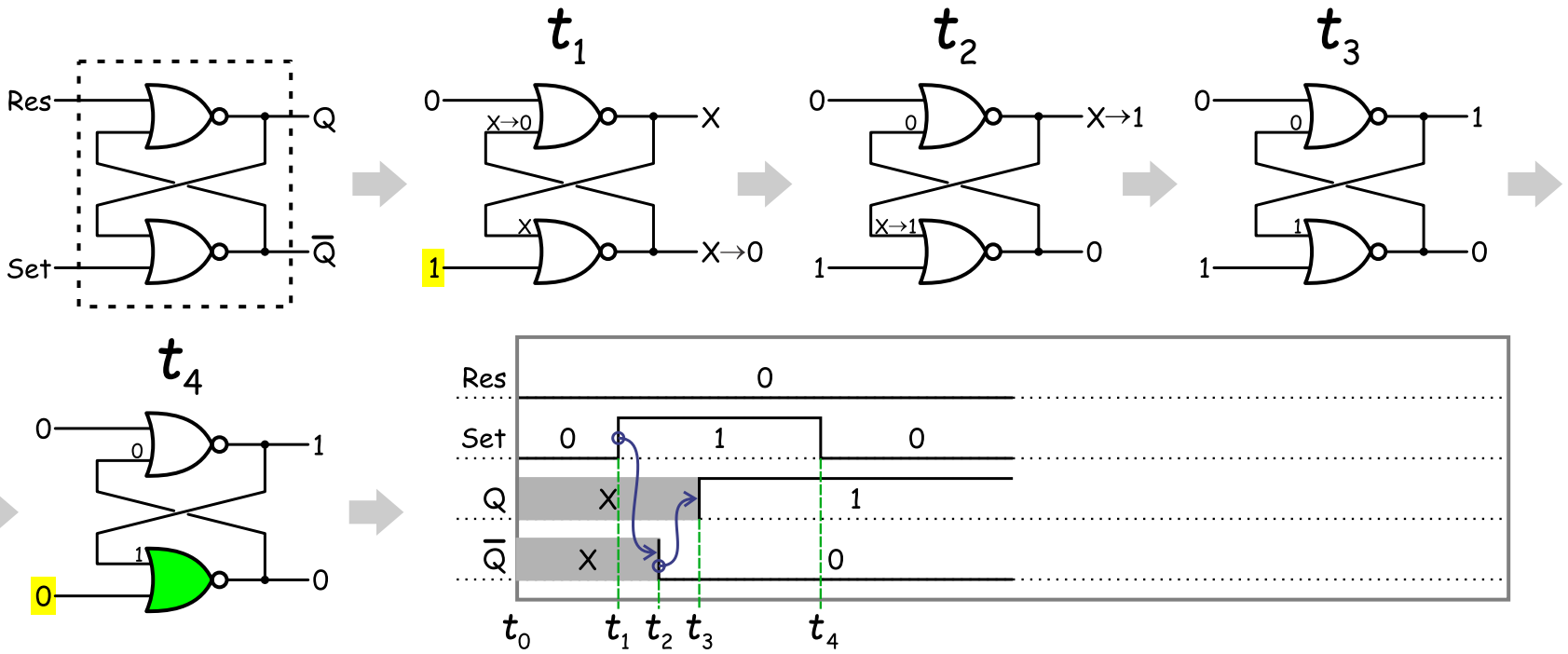


S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

When the Set command is de-asserted (set back to zero at instant t_4), the latch, as expected, keeps the previously imposed value.

NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X

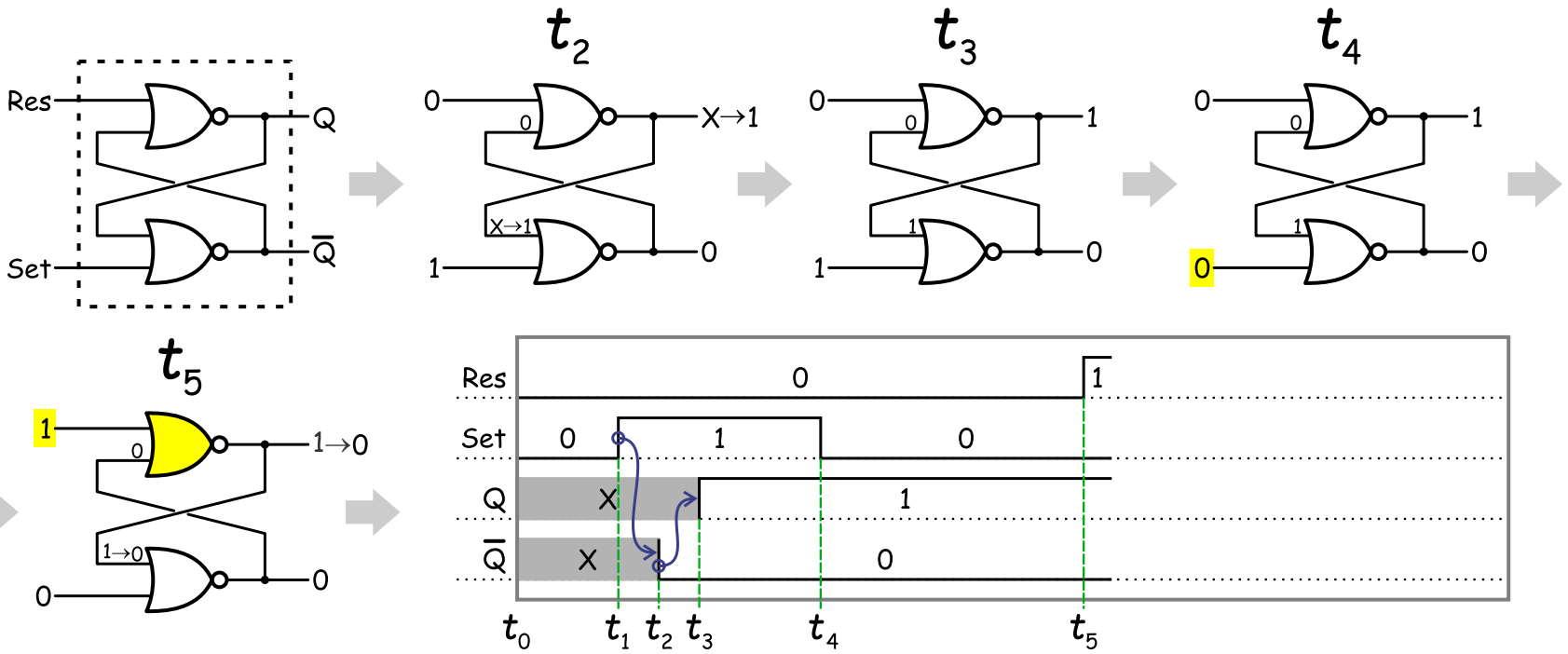


S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

At instant t_5 the Res (Reset) input of the latch is driven to 1. The gate connected to output Q immediately triggers an output change given by the 10 configuration present on its inputs.

NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X

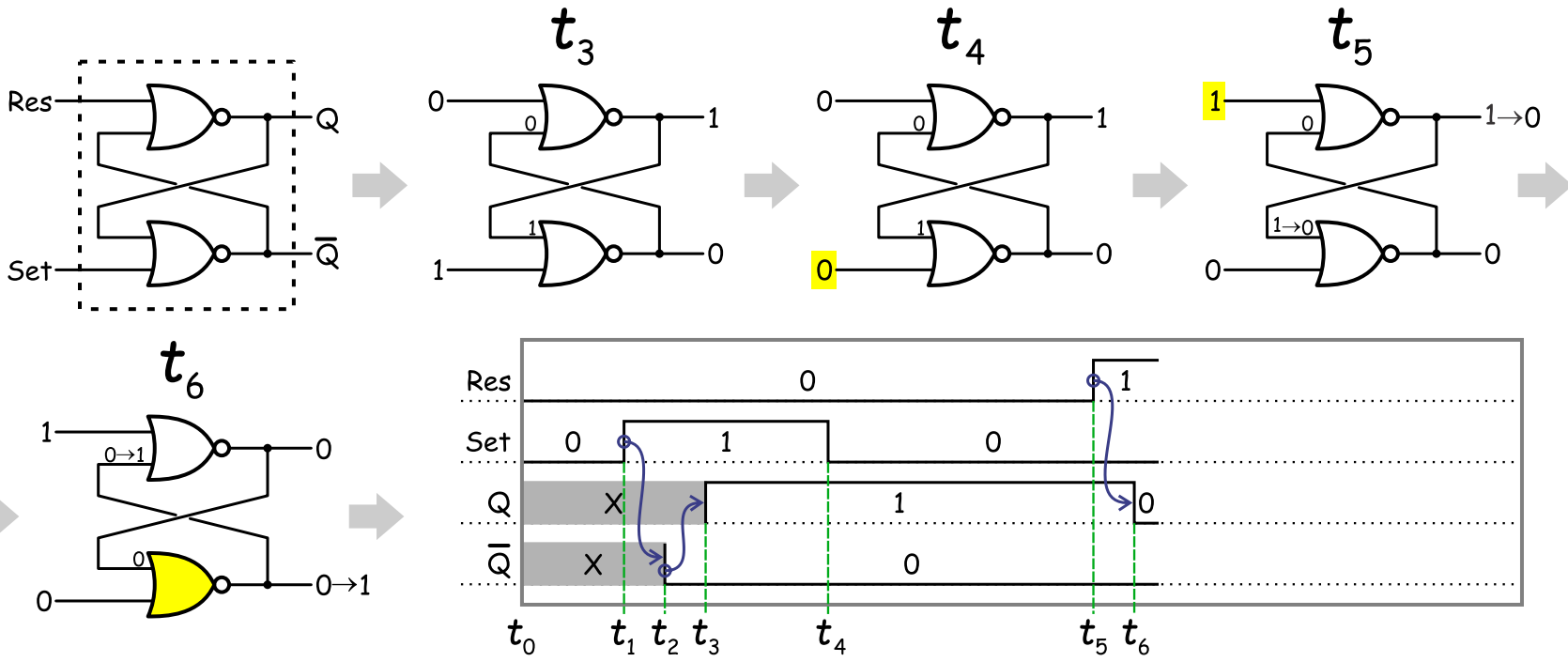


S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

At instant t_6 the latch output Q immediately goes to zero. This event triggers the next change to 1 of the output relating to $\bar{Q}$.

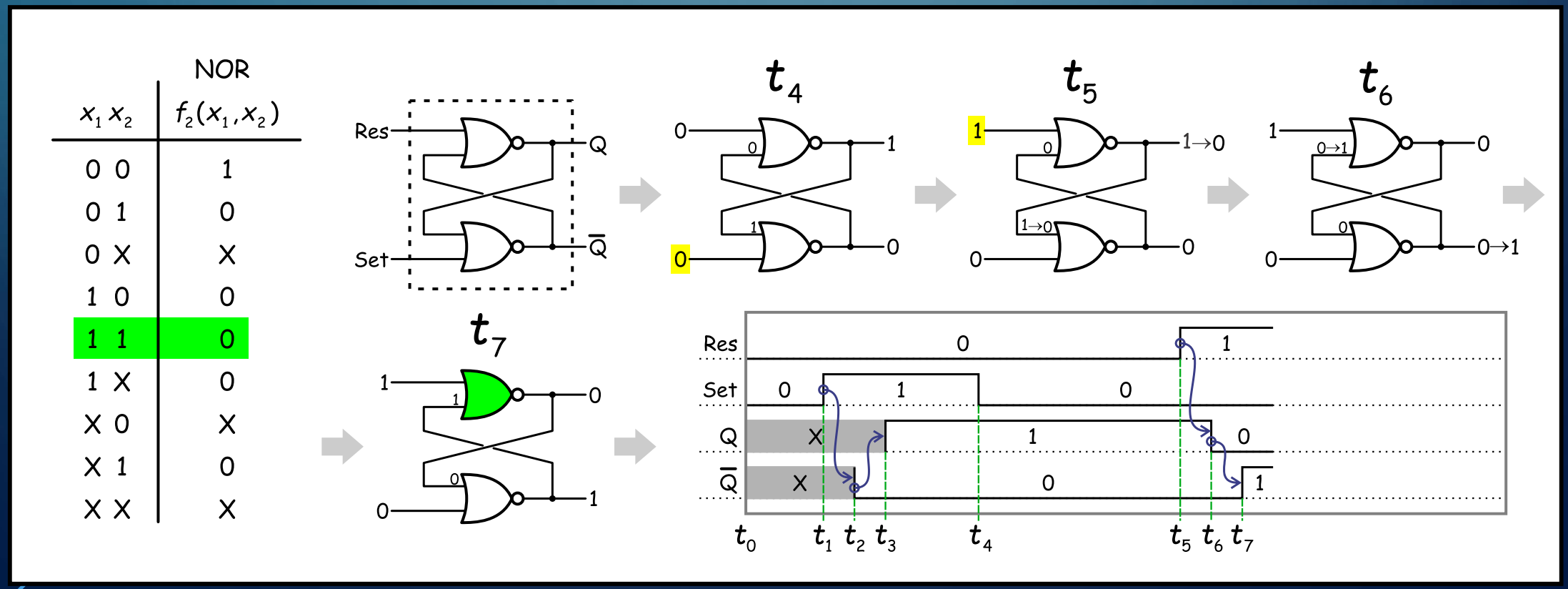
NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X



S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

As expected, shortly afterwards output $\bar{Q}$ settles at 1 at instant t_7.

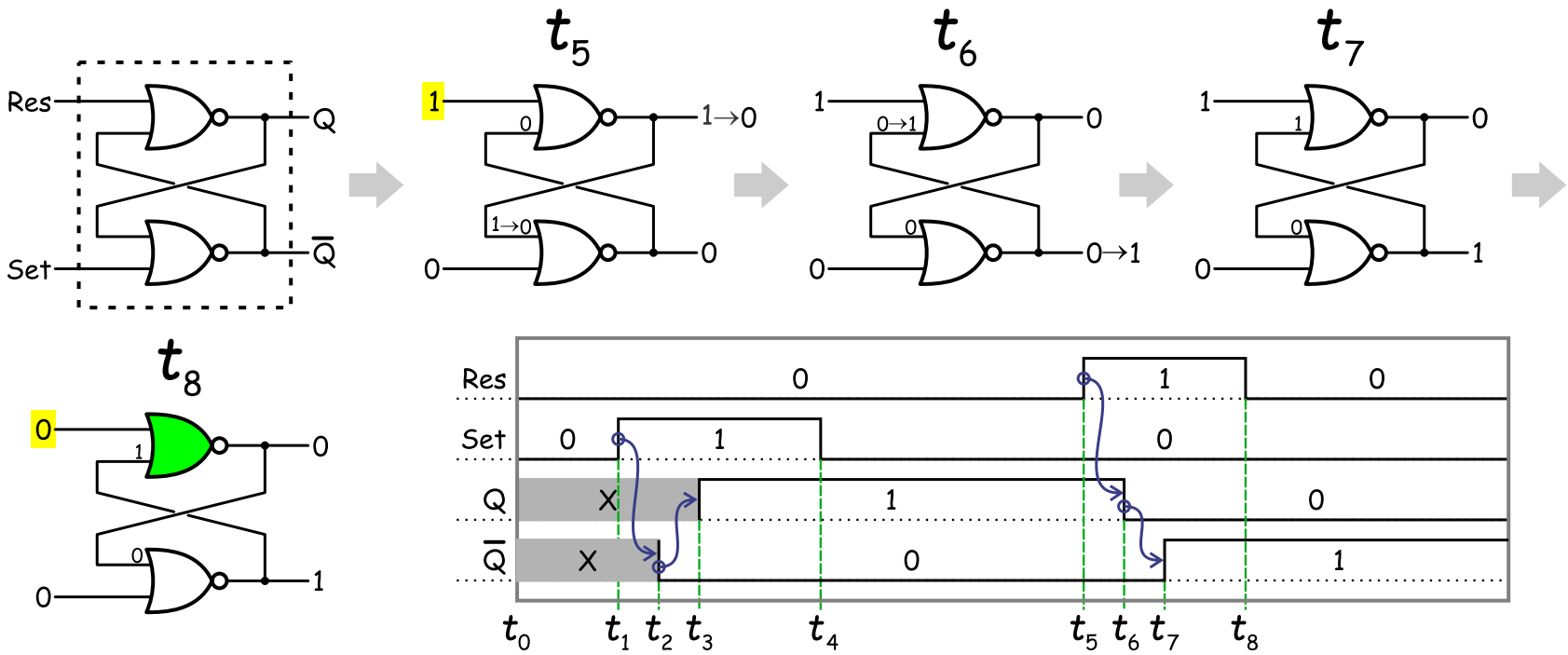


S-R LATCH SIMULATION WITH THREE-VALUED LOGIC

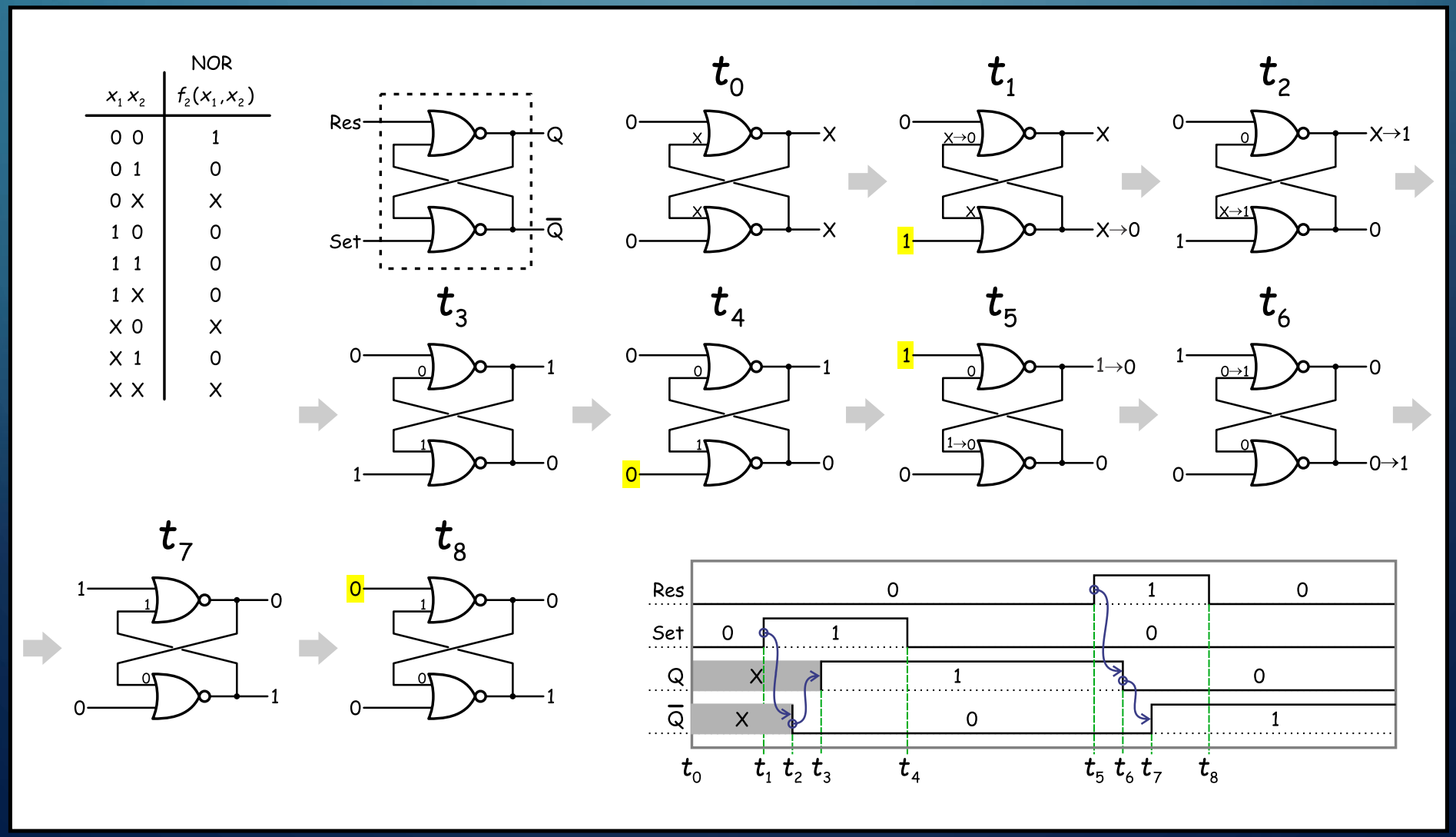
When the Res command is removed, the 01 configuration at the input of the NOR gate of output Q confirms the value zero. The latch will now remember the state until the next Set.

NOR

x_1 x_2	$f_2(x_1, x_2)$
0 0	1
0 1	0
0 X	X
1 0	0
1 1	0
1 X	0
X 0	X
X 1	0
X X	X

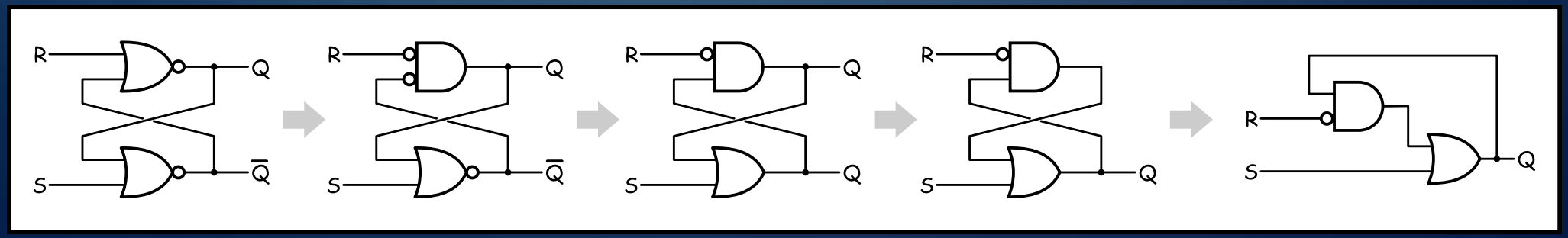


S-R LATCH SIMULATION WITH THREE-VALUED LOGIC



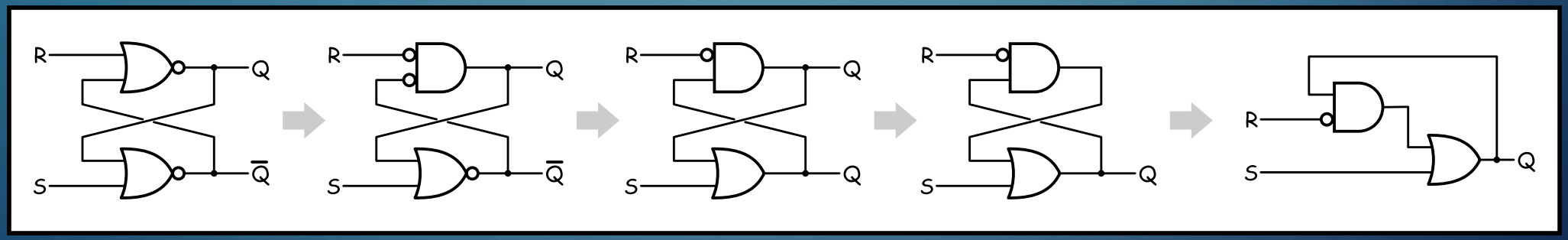
SEQUENTIAL LOGIC NETWORKS (1)

The S-R Latch simulation seen earlier suggests the existence of logic networks able to "remember" the processing phase, normally also known as internal state.
Still observing the previous experiment, we can deduce that feeding the output signals back to the input may be the key to giving a logic network the ability to have memory.
With a simple manipulation we can see that the NOR-based S-R Latch seen earlier can also be defined as a combinational network in SP canonical form that reads its own output (we consider only output Q).
Note that the form in the middle of the drawing below suggests that, depending on which of the two outputs (Q) is to be used as feedback, one can choose either of the two canonical forms (SP if the OR is chosen or PS if the AND is chosen).

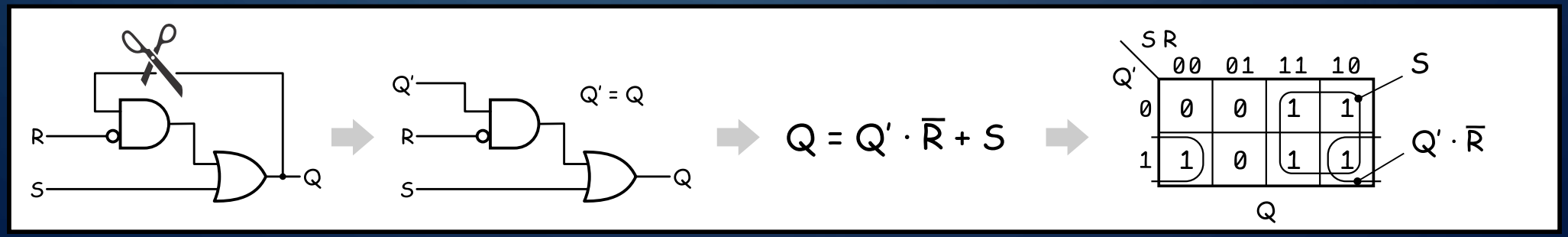


SEQUENTIAL LOGIC NETWORKS (2)

The last form on the right shows that output Q is fed back to the input.



By setting the relation $Q' = Q$, the loop can be opened to obtain a pure combinational network from which the Boolean expression $Q = Q' \cdot \bar{R} + S$ is derived.

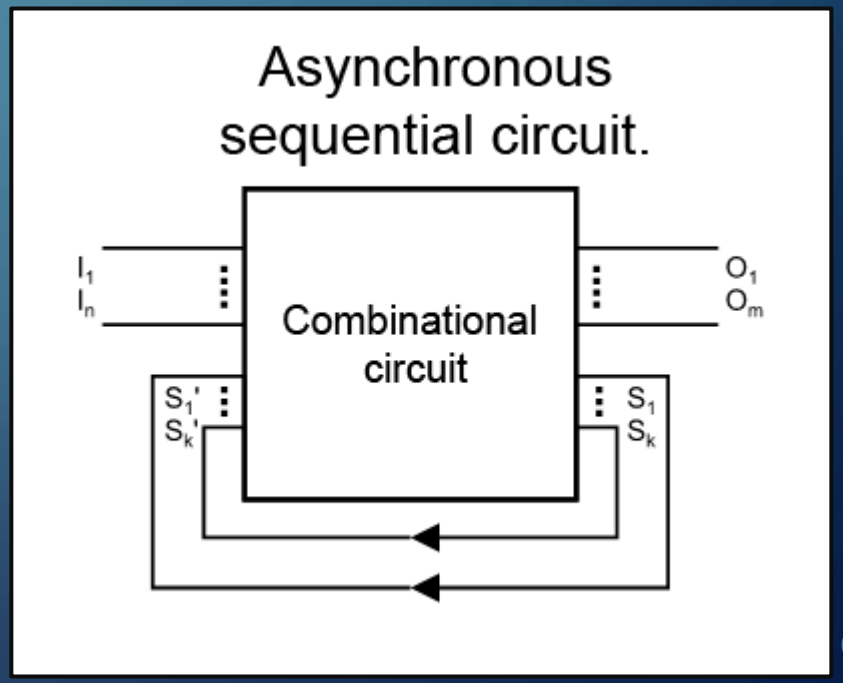


ASYNCHRONOUS SEQUENTIAL NETWORK MODEL

In the SP canonical form of the S-R Latch, we can say that output Q also represents the state (or rather, it is the state variable) of the sequential network.

The category of sequential networks that feed the state variable directly back to the input is called "asynchronous".

Changes in the outputs immediately affect the inputs; if the state variable consists of several expressions (i.e. it is a vector of several related binary variables), this category of logic networks becomes complicated to manage.

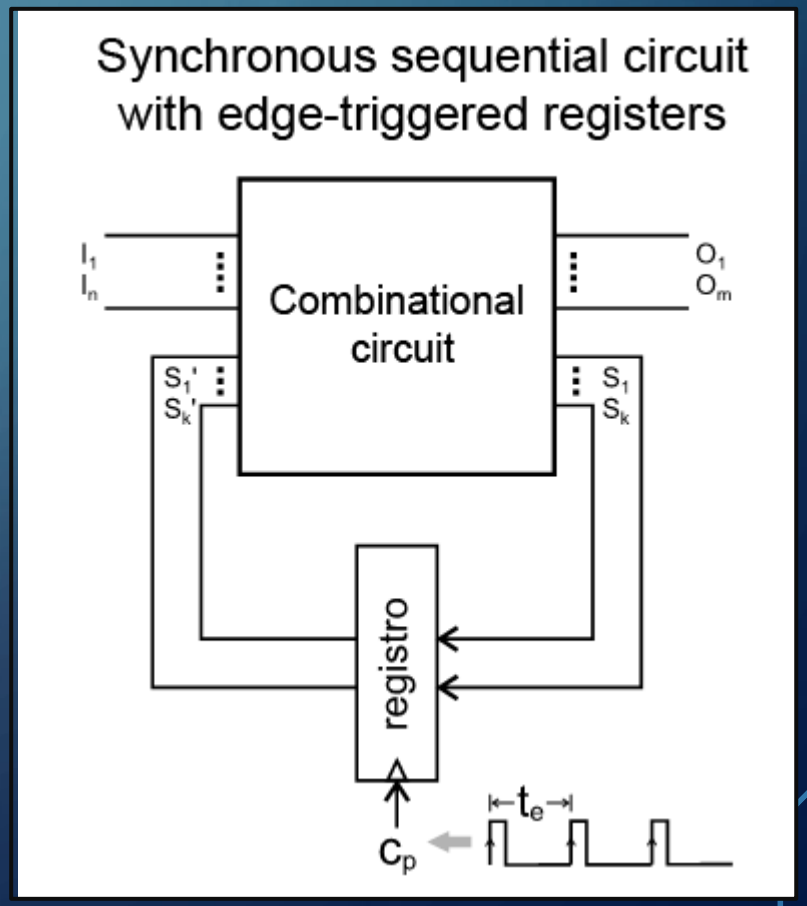


Because of the hazards (especially dynamic ones) mentioned earlier, the state variable may be encoded incorrectly, resulting in errors or instability of the system. Techniques for mitigating these problems are well beyond the scope of this presentation.

SYNCHRONOUS SEQUENTIAL NETWORKS (1)

To try to solve the problem and the typical critical issues of asynchronous sequential networks, one can try placing a "pulse-controlled register" on the loops. A pulse-controlled register can consist of an array of D-type Latches (already seen earlier) with their enable inputs C_p tied together. By applying a pulse to the latches' enable inputs, the register can be asked to "track", continuously storing it, the internal state of the sequential network. When the pulse is absent the register presents the last stored configuration. In this way the output state is not continuously fed back to the inputs. In effect the processing is "synchronized" with the cadence of the pulses.

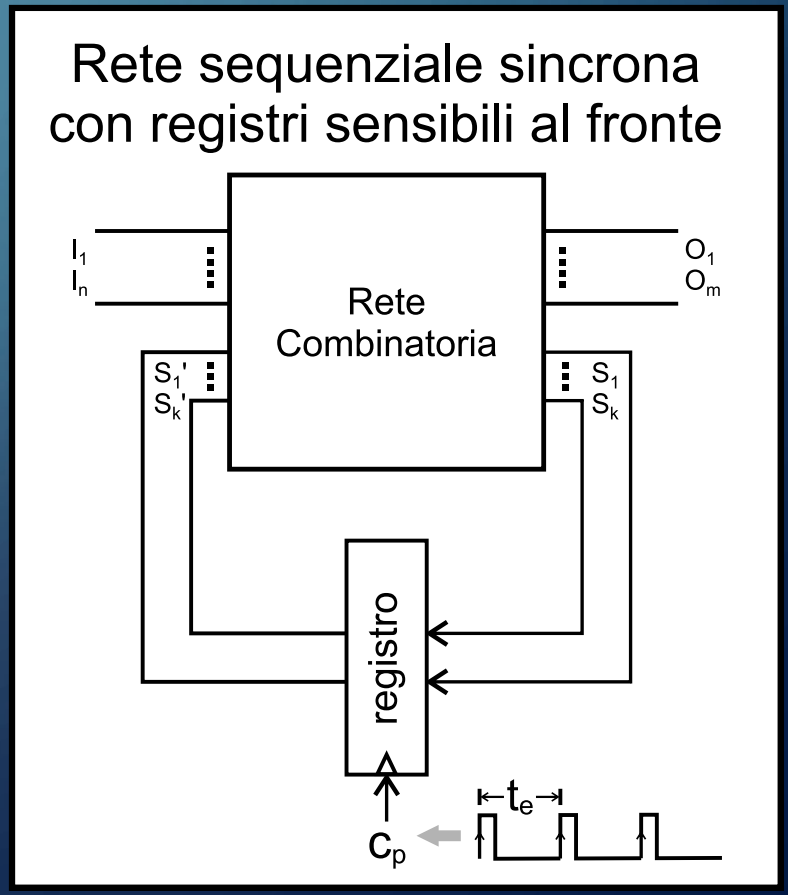
This synchronous model is nevertheless not free of problems: one must make sure that time t_e is long enough to guarantee settling of the combinational network; at the same time the pulse t_p must be long enough for the register to store the state, yet short enough not to create an asynchronous loop (a continuous connection).
The configuration described here, which uses so-called "Transparent Latches", is difficult to manage in terms of network reliability.



SYNCHRONOUS SEQUENTIAL NETWORKS (2)

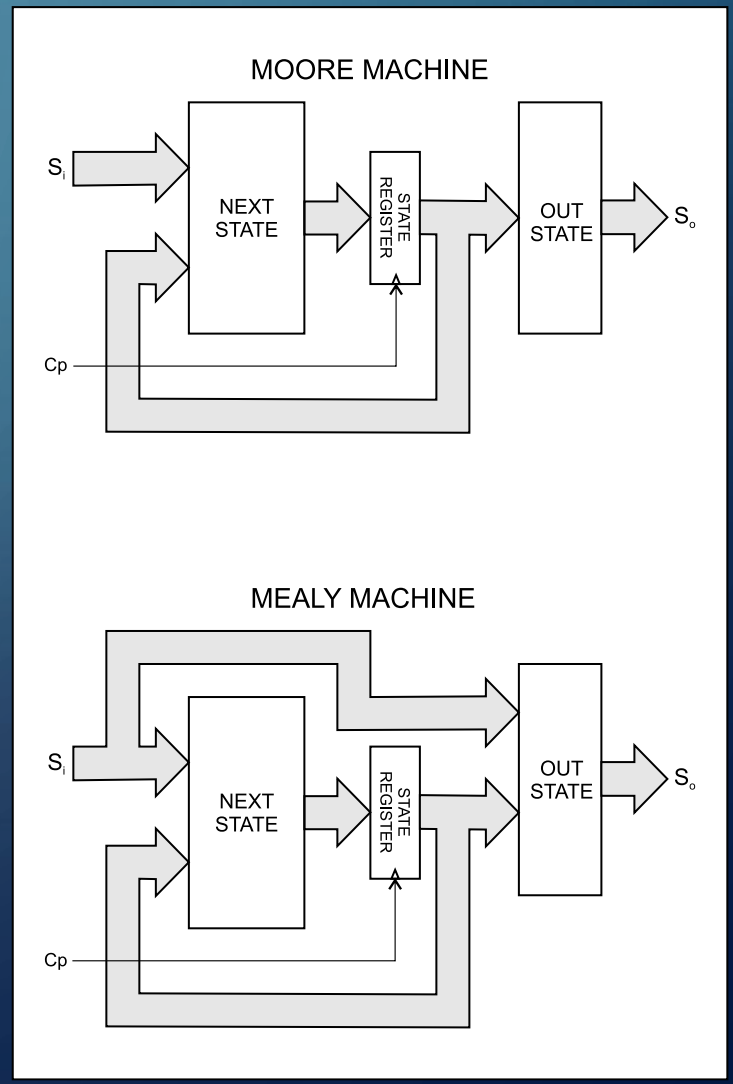
To arrive at a reliable synchronous network model, the state registers must be built using Flip-Flops. With this type of register there is no longer a need to control the width of the cadence pulse (also called "Clock"). The register built with Flip-Flops stores information based on the "changes" of signal C_p and not on its level. For a rising edge, i.e. from 0 to 1, we speak of a "positive edge" (posedge). For the transition from 1 to 0 we speak of a "negative edge" (negedge). A Flip-Flop reacts to only one of the two edges, depending on its nature.

The Flip-Flops for this type of application are called Master-Slave, since they consist of two latches in cascade working on opposite levels of the value of C_p. If the first latch stores at level 0, continuously "tracking" the input, the second latch presents the stored value at the output. As soon as signal C_p goes from 0 to 1, the first latch stops tracking, holding the last value, while the second latch "tracks" the stable value of the first. A small "triangle" at the register's input C_p denotes sensitivity to the positive edge. If a small empty circle is also present, the register is sensitive to the negative edge.

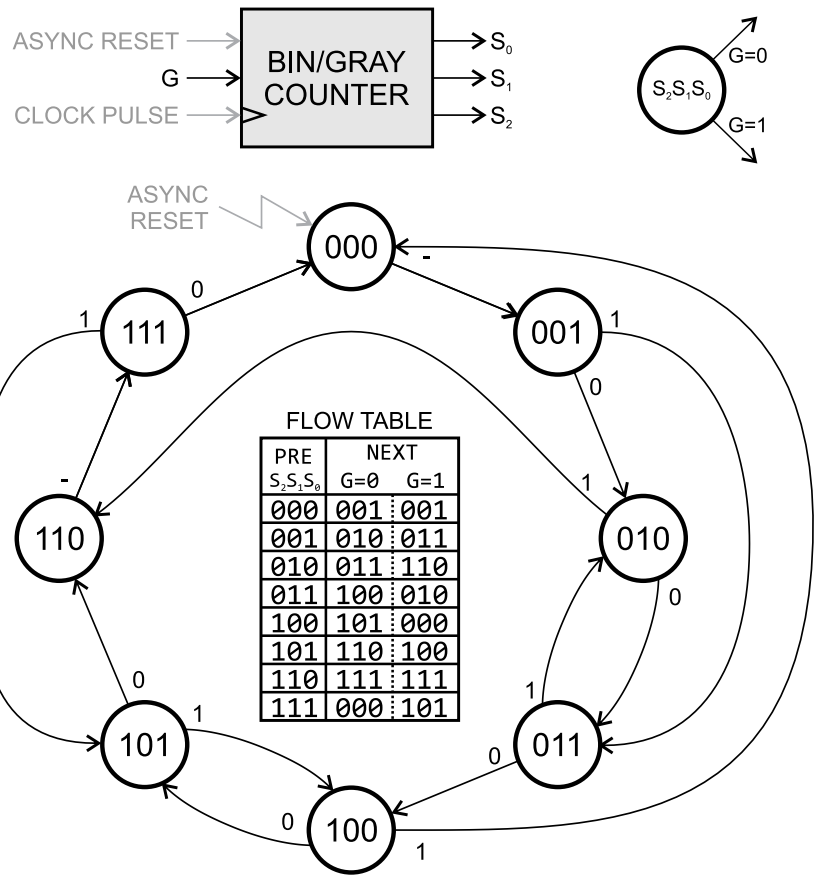


SYNCHRONOUS SEQUENTIAL NETWORKS (3)

- They use a cadence (clock) for processing.
- They need a register (a set of flip-flops) to store the internal state.
- They use a combinational network to compute the next state.
- They may have a combinational network to produce the output state.
- They are called Moore Machines if the output depends only on the internal state; those whose output state depends on both the internal state and the input state are called Mealy Machines.



SYNCHRONOUS SEQUENTIAL EXAMPLE (1)

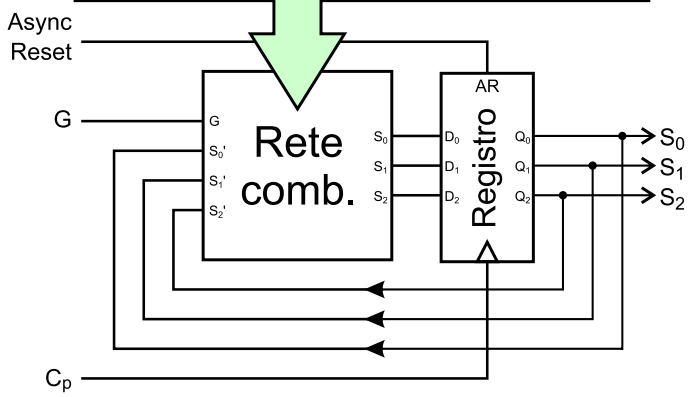



```
# File bg_cnt.pla
.i 4
.o 3
.p 16
.ilb g s2' s1' s0'
.ob s2 s1 s0
0 000 001
0 001 010
0 010 011
0 011 100
0 100 101
0 101 110
0 110 111
0 111 000
1 000 001
1 001 011
1 010 110
1 011 010
1 100 000
1 101 100
1 110 111
1 111 101
.e
```

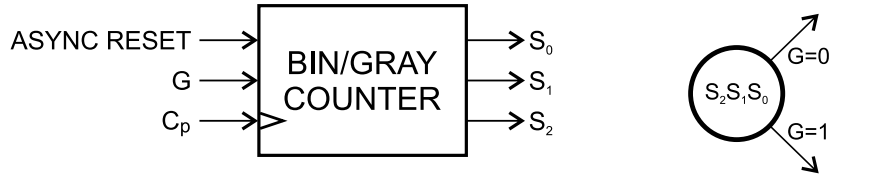
```
s2 = (!g & !s2' &  s1' &  s0')
   | (      s2' & !s1' &  s0')
   | (!g &  s2' &         !s0')
   | ( g &         s1' & !s0')
   | ( g &  s2' &  s1');

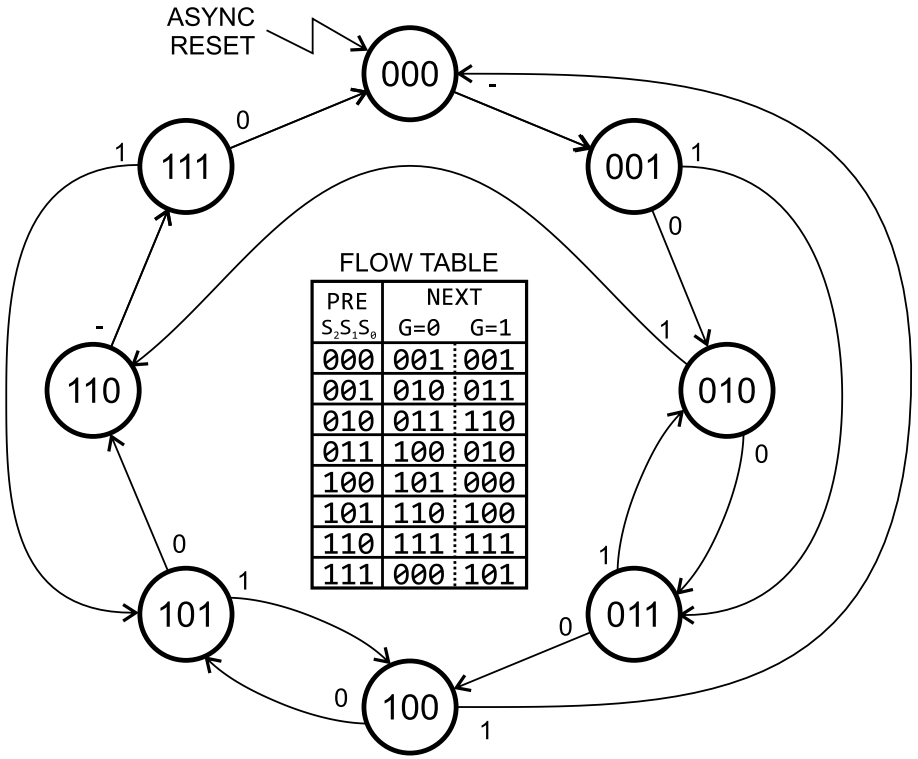
s1 = (!g &         !s1' &  s0')
   | ( g & !s2' &          s0')
   | (              s1' &  s0');

s0 = ( g & !s2' & !s1')
   | ( g &  s2' &  s1')
   | (!g &                !s0');
```



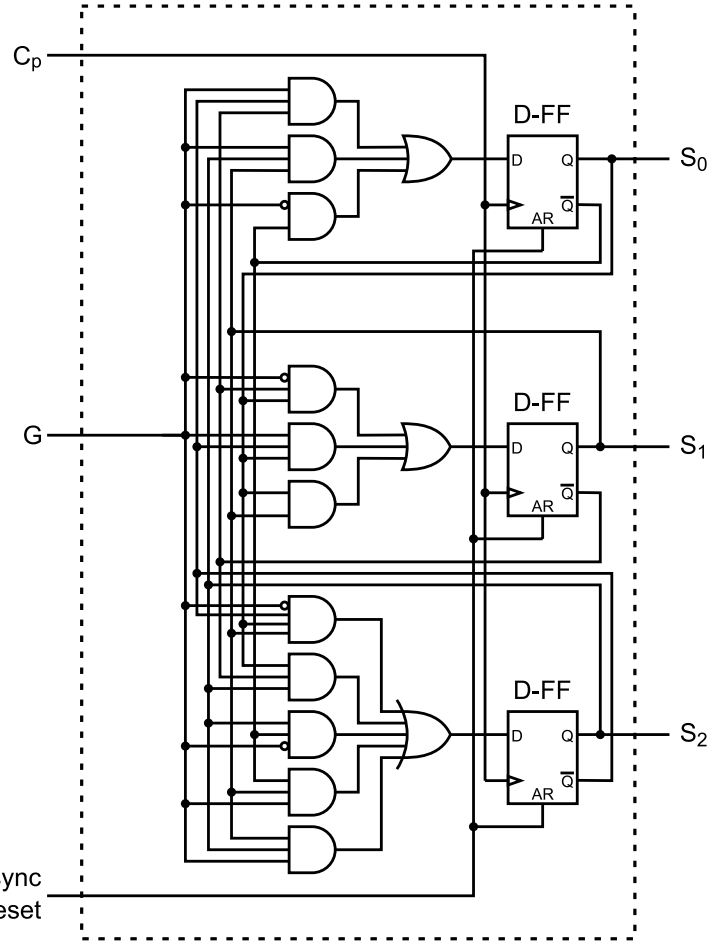
SYNCHRONOUS SEQUENTIAL EXAMPLE (2)





FLOW TABLE

PRE $S_2S_1S_0$	NEXT G=0	NEXT G=1
000	001	001
001	010	011
010	011	110
011	100	010
100	101	000
101	110	100
110	111	111
111	000	101



INCREASING CIRCUIT COMPLEXITY

- To reduce size, power consumption and cost, and to allow more complex and faster logic structures, the semiconductor industry has introduced ever more complex components and new hardware technologies, together with the related software development systems.
    i. The so-called "simple programmable logic" (SPLD) such as PROM, PAL, PLA, GAL and PALCE replaced parts of networks made with discrete components (logic gates and small SSI functions).
    ii. Introduction of the first rudimentary HDL software tools to manage the programming of SPLDs, with PALASM and later ABEL, ISP and CUPL.
    iii. Appearance on the market of the first "complex programmable logic" devices (CPLD).
    iv. Appearance of the first programmable Gate Array device (FPGA), namely the XC2064 from Xilinx Inc., introduced in 1985. The tool to program this device was called XACT (Xilinx Advanced CAD Technology), whose syntax was based on ABEL (Advanced Boolean Expression Language) from Data I/O Corporation.

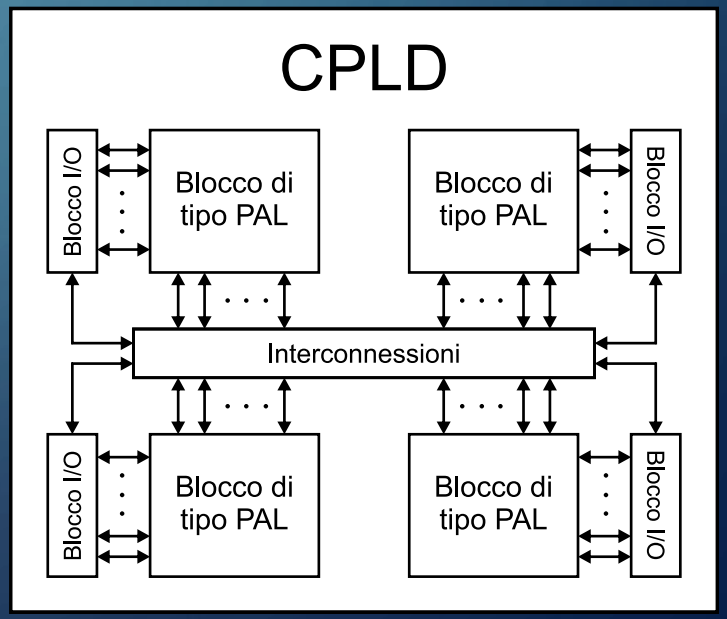
CPLD

CPLDs can be considered an evolution of PALs and consist of several PAL structures put together, known as "macrocells".

The macrocells are connected through a programmable interconnect, also called GIM (global interconnection matrix). By reconfiguring the GIM, the most diverse logic circuits can be built by interconnecting the macrocells.

CPLDs communicate with the outside world through digital I/Os. In the example at the side you can see that these are divided into groups closely and efficiently connected to their own macrocell. Nevertheless, a macrocell can reach the I/Os of other macrocells through the programmable interconnect matrix (GIM), albeit less efficiently.

CPLDs are complex enough to require CAE tools for their programming.



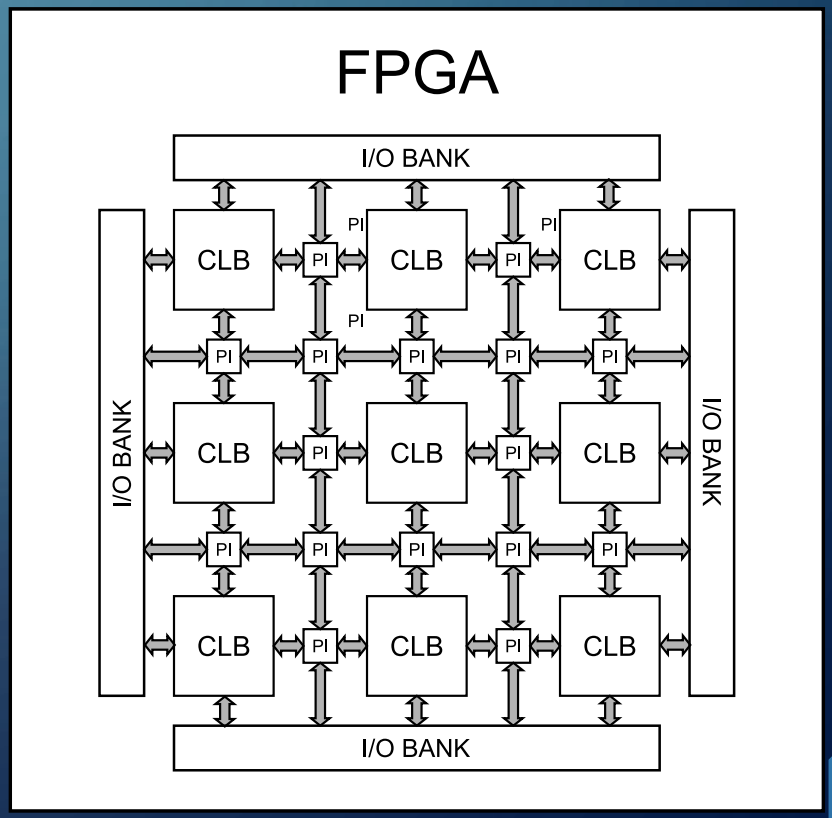
FPGA

FPGAs are, to all intents and purposes, programmable Gate Arrays. They are the user-programmable devices that come closest to ASICs. FPGAs are organized into logic blocks (CLB, Configurable Logic Block), somewhat like CPLDs, but these are simpler, more numerous and can be interconnected with one another through programmable interconnects (PI). The latter connect the I/Os, which are normally divided into banks.

In general, CLBs consist of a relatively small number of elements. A typical CLB consists of a 4-5 input LUT (Look Up Table), a Full-Adder and a D-type flip-flop. A multiplexer at the input of the Flip-Flop is often present as well. There is no standard scheme for LUTs, which always differ depending on the manufacturer.

Modern FPGAs also have many predefined macroblocks (Hard Blocks) dedicated to complex functions, specifically optimized in terms of speed and chip area. There are devices with memory blocks, parallel multipliers, entire DSP functions, processors and much more. FPGAs also implement support for clock generation (PLL) and programmable delays. Some are so rich in resources that they can implement entire systems inside them (SoC, System on Chip).

Programming a modern FPGA can only be tackled with sophisticated software tools that provide for the most efficient mapping of the cells and a careful routing of signals between the cells and to the I/Os.



MANAGING CIRCUIT COMPLEXITY

- Synthesis of a complexity that cannot be tackled manually or with primitive HDLs, in the context of modern FPGAs or complex PLDs.
- Simulation as the primary support for designing logic networks, to verify operation well before implementation.
- Need to synthesize the logic network, or parts of it, automatically.
- Appearance of the first sophisticated Electronic Design Automation (EDA) tools.

PART TWO

HARDWARE DESCRIPTION LANGUAGES

AUTOMATED DESIGN

- HDLs (Hardware Description Languages) are languages for modeling, simulating and, optionally, formally verifying hardware.
- If properly written, a model defined in an HDL can allow automatic synthesis.
- The main languages in use are Verilog (and SystemVerilog), VHDL and, more rarely, SystemC.
- They allow the model to be described at different levels of abstraction (behavioral, structural RTL, gate level or switch/transistor level).
- They allow the description of test vectors to stimulate the models, to be run on dedicated simulators. They can support stochastic functions (queues and random probability distributions) to support performance verification.
- They optionally allow formal verification of the model (SystemVerilog).

EDA – ELECTRONIC DESIGN AUTOMATION (1)

- A set of tools, specifically for digital systems, for designing ASICs or for development on CPLDs and FPGAs.
- EDA areas for the development of digital systems:
  1. Design Entry: definition of the logic network design (schematics and HDL languages).
  2. Verification of the logic network (pre-synthesis) by running the HDL code on a simulator.
  3. Synthesis of the logic network from the HDL code, generating netlists (structures of components/functions and nodes as interconnections) specific to the target hardware architecture.
  4. Simulation of the logic network at structural level (pre-layout): the technology blocks are considered ideal components.

EDA – ELECTRONIC DESIGN AUTOMATION (2)

- EDA areas for the development of digital systems (continued)
  5. Place & Route, i.e. mapping the network onto the various predefined blocks of the chosen technology and routing the signals.
  6. Calculation of the time the signals need to cross the interconnects and logic blocks (propagation times). Generation of a further netlist provided with elements that generate delays equivalent to the propagation times calculated above (back-annotation). The purpose of
     back-annotation is to have accurate post-synthesis simulations that also take into account the physical operating conditions (such as the estimated ambient temperature and the supply voltage(s)).
  7. Programming of the device (with a bitstream) or generation of the layouts (masks) for production.

PART TWO

THE VERILOG LANGUAGE®

A BRIEF HISTORY OF THE VERILOG LANGUAGE

- Verilog HDL was invented by Phil Moorby and Prabhu Goel around 1984 as a proprietary hardware modeling language of Gateway Design Automation Inc.
- It was stabilized as a specification in 1990 and made open by Cadence Design Systems in 1991, again by Cadence, which organized the Open Verilog International (OVI) consortium.
- It was subsequently submitted to the IEEE and became the IEEE 1364-1995 standard, commonly referred to as Verilog-95.
- To cover some shortcomings found in Verilog-95, a number of extensions were resubmitted to the IEEE and became the IEEE 1364-2001 standard, known as Verilog-2001.
- Verilog-2005 (IEEE Standard 1364-2005) was published with minor corrections and changes.
- Also in 2005, SystemVerilog was published, a superset of Verilog-2005, with many new features and capabilities to facilitate design verification.
- Starting in 2009, the SystemVerilog and Verilog standards were merged into SystemVerilog 2009 (IEEE Standard 1800-2009).

VERILOG AND COMPUTER LANGUAGES

- Differences from computer programming languages.
  - **Purpose:** Verilog is used to describe a circuit or an entire digital system; a computer programming language is used to produce executable code.
  - **Constructs:** Verilog describes the properties of wires, registers, logic gates, etc., whereas a programming language describes variables, functions and flow-control structures.
  - **Execution:** Verilog describes but does not directly execute: it can be used to generate configurations or masks for physical circuits; a computer language describes code that, once compiled, is executed on a CPU.

INTRODUCTION TO THE VERILOG LANGUAGE

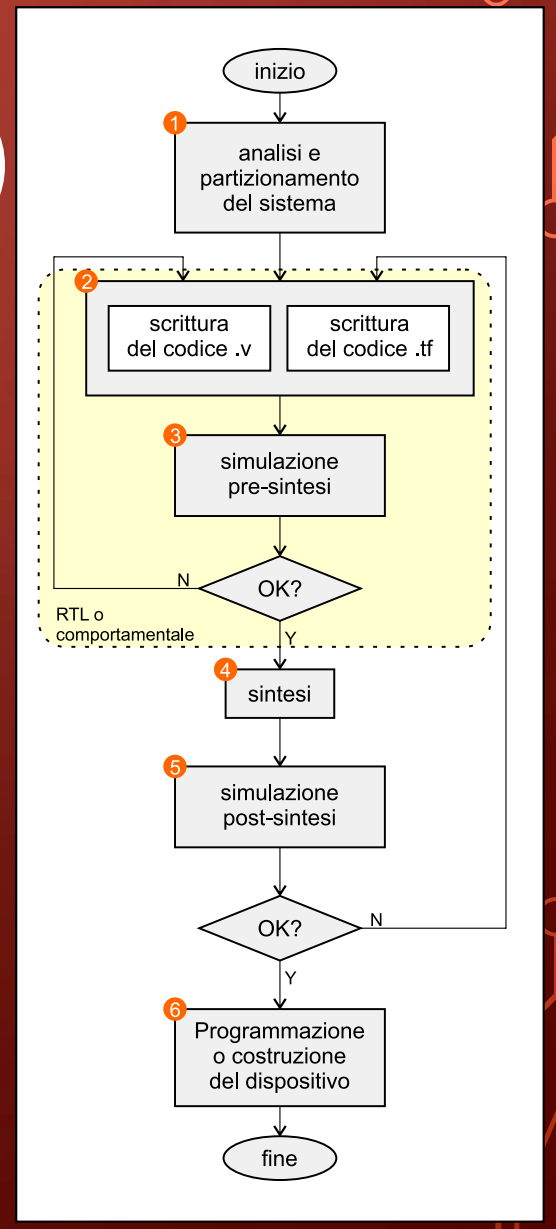
- A Verilog project consists of at least one text file; .v is usually used as the extension (.sv for SystemVerilog).
- Verilog is "case sensitive": var_X and var_x are two different objects.
- The main construct in Verilog is the module (lexical element 'module').
- A .v file can contain one or more modules and other commands, such as the so-called directives and tasks (which start with $).
- Some files may have the .tf extension (test fixtures): they are Verilog files (like .v files) that contain the stimulus and verification vectors (to be used with the simulator).

LEVELS OF ABSTRACTION

- Verilog supports abstract behavioral modeling, so it can be used to model the functionality of a logic network at a high level of abstraction. This is useful for verifying the analysis and system-partitioning phase (proof of concept).
- Verilog supports the RTL (Register Transfer Level), which is used for the detailed design of digital circuits. If the code is written appropriately, synthesis tools transform RTL descriptions down to the level of gates/logic blocks/switches.
- Verilog supports gate-level and switch-level descriptions, used for the verification of digital designs, including very low-level logic simulation, static and dynamic timing analysis, testability analysis and fault classification.
- The test vectors, which stimulate the network and contain the expected results to compare with the network's output, are also written in Verilog, usually in non-synthesizable form.
- At the highest level, Verilog contains stochastic functions (queues and random probability distributions) to support test fixtures for performance modeling.

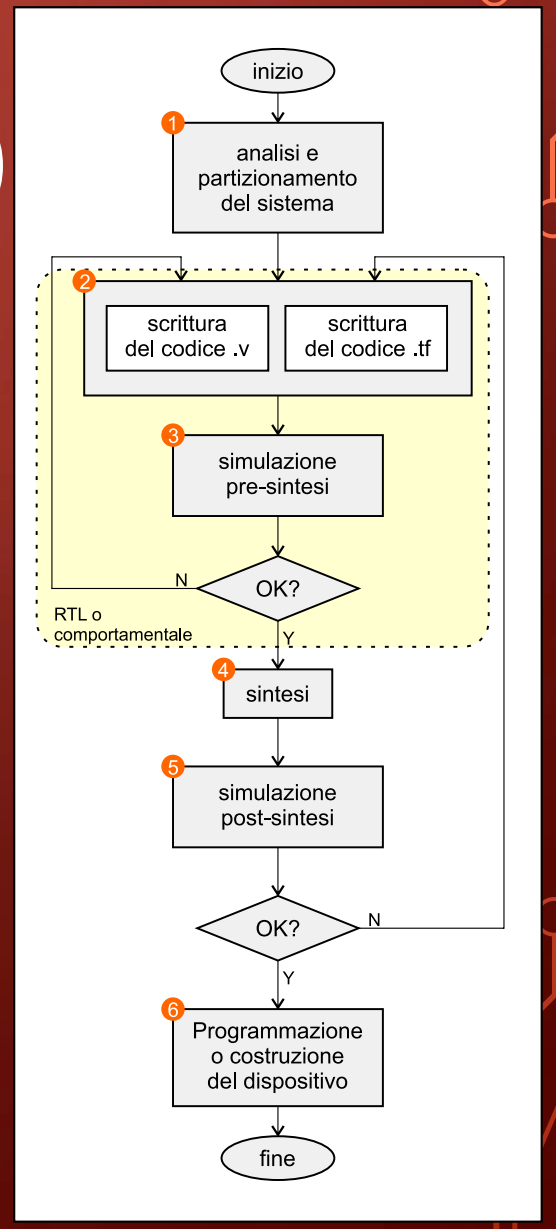
HDL-BASED ASIC DEVELOPMENT FLOW (1)

1. Analysis and partitioning of the problem.
2. Writing or editing the code blocks (modules) in behavioral or RTL style and the related test vectors.
3. Simulation (pre-synthesis) of the code: if the results are unexpected, go back to step 2.
4. Synthesis into blocks specific to the chosen technology.



HDL-BASED ASIC DEVELOPMENT FLOW (2)

5. Simulation (post-synthesis) of the code: in case of error, go back to step 2.
6. Place & route. Mapping of "resources" and interconnections.
7. Post-place & route simulation is to be considered an optional step and not very useful except in special circumstances. Instead of simulation, the technique based on so-called "constraints" applied to the critical signal paths is used.
8. Programming of the device or circuit implementation.

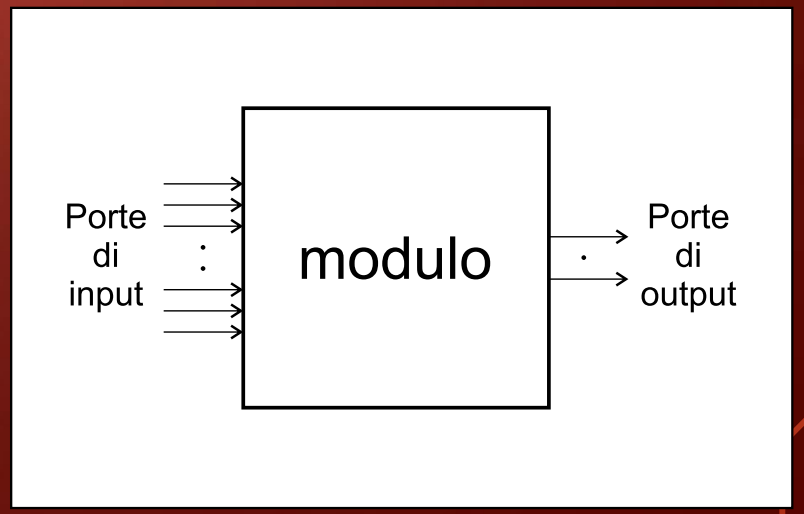


INTRO TUTORIAL (1)

Modules:

- They are blocks of code that implement functionality.
- They communicate with the outside through ports (input/output), which can be linked to electrical connections (wires).
- They contain code constructs that describe their internal structure.
- They are delimited by the keywords *module* and *endmodule*.

```
module name( [port list] );
  // Module contents
endmodule
```



INTRO TUTORIAL (2)

Modules:

- They may have no port list.

```
module name;
  // Module contents
endmodule
```

- The keywords *input* and *output* are used to define the respective input and output ports.

```
module fn_Z( input a, input b, input c, output z );
  // Module contents
endmodule
```

- Inside the module are the constructs that define the behavior. In this box we see a construct that represents a "continuous" combinational assignment.

```
module device_under_test( input a, input b, input c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (3)

Modules:

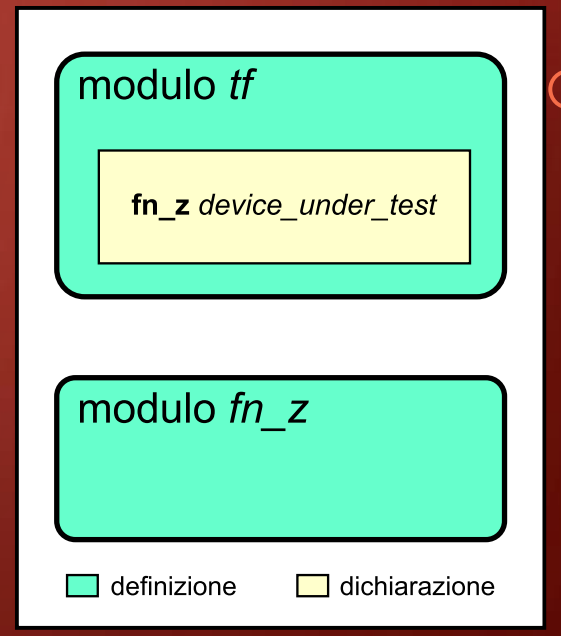
- To verify the correctness of what we have written so far, we will use an open-source tool called Icarus Verilog (iverilog). Icarus Verilog is an implementation of the IEEE-1364 Verilog hardware description language.
- Icarus Verilog is a compiler and simulator that reads a Verilog source file and produces an output file. The latter, on some platforms, can also be a pure executable. If compiling the module produces no errors, we are looking at code that is at least syntactically correct.

```
$ iverilog assign.v
$ ll
drwxr-xr-x  2 user user 4096 Apr 23 12:43 ./
drwxr-x--- 29 user user 4096 Apr 23 11:56 ../
-rwxr-xr-x  1 user user 2367 Apr 23 12:43 a.out*
-rw-r--r--  1 user user  109 Apr 23 12:32 assign.v
$ ./a.out
$ _
```

INTRO TUTORIAL (4/1)

Modules:

- Digital systems are structured hierarchically: that is, they consist of blocks interconnected with one another having more or less complex functions. Some blocks contain blocks that in turn may contain other blocks, and so on. Through modules, Verilog can model the hierarchical structure of such systems.
- Modules, however, cannot be defined inside other modules. Although this seems to contradict the previous point, it should be remembered that – like programming languages – Verilog also distinguishes between declarations and definitions.



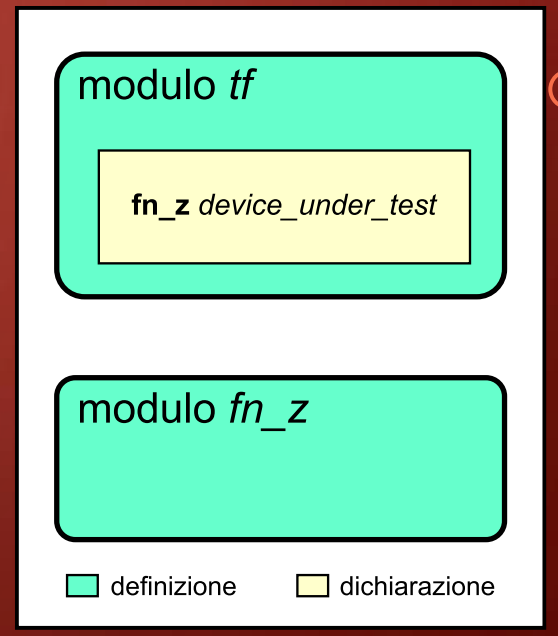

```
module tf;
  fn_z device_under_test(…);
endmodule

module fn_z(…);
  // Module contents
endmodule
```

INTRO TUTORIAL (4/2)

Modules:

- Modules can also contain "instances" (i.e. declarations) of other modules, but they cannot directly or indirectly contain instances of themselves.
- A module can also declare multiple instances of other modules.
- At the top of the hierarchy there will be a single module that contains all of them (directly or indirectly), identified as the "top" module (*tf* in the example alongside).




```
module tf;
  fn_z device_under_test(…);
endmodule

module fn_z(…);
  // Module contents
endmodule
```

INTRO TUTORIAL (5/1)

Modules:

- Some modules (test benches) can contain the stimuli for the simulation (test fixtures). The stimuli are usually grouped in suitable constructs.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (5/2)

Modules:

- Some modules (test benches) can contain the stimuli for the simulation (test fixtures). The stimuli are usually grouped in suitable constructs.
- In our example, the keyword *initial* is used to define a time sequence for applying the stimuli. During simulation, whatever is controlled by *initial* is executed only once. To incorporate more than one command inside *initial*, we must resort to another construct that represents the so-called block.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (5/3)

Modules:

- Some modules (test benches) can contain the stimuli for the simulation (test fixtures). The stimuli are usually grouped in suitable constructs.
- In our example, the keyword *initial* is used to define a time sequence for applying the stimuli. During simulation, whatever is controlled by *initial* is executed only once. To incorporate more than one command inside *initial*, we must resort to another construct that represents the so-called block.
- The block is syntactically defined by the pair of keywords *begin…end*. Inside a block, which is always controlled by another construct (in this case *initial*), the statements are considered to be a single unit. In the case of *initial* the commands of the block (excluding the directives that start with $) are executed sequentially. *initial* blocks are not synthesizable. For the moment the details of the simulation constructs will be ignored.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (6/1)

Modules:

- Inside the simulation modules there are the instances of the modules to be simulated (they generally form a hierarchy). In the example alongside, we see that module *fn_z* defines an instance (*device_under_test*) that receives the stimuli generated by the hosting module *tf*.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (6/2)

Modules:

- Inside the simulation modules there are the instances of the modules to be simulated (they generally form a hierarchy). In the example alongside, we see that module *fn_z* defines an instance (*device_under_test*) that receives the stimuli generated by the hosting module *tf*.
- There are other fundamental elements that we have already met. Besides *reg* we see the keyword *wire* highlighted at the side. Both elements allow interconnection between modules.
- *wires* can be likened to real electrical connections. They can be read or assigned. No value is stored inside them. They must be driven by an *assign* statement or by a module port.
- *regs* represent the memory elements (registers) seen earlier. They keep their value until a new one is given to them. Contrary to their name, *regs* do not necessarily correspond to physical registers. They can be synthesized as Flip-Flops, latches or combinational circuits.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (7/1)

Modules:

- The actual stimuli consist of sequences of values assigned to the registers.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

a	b	c	z
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

INTRO TUTORIAL (7/2)

Modules:

- The actual stimuli consist of sequences of values assigned to the registers.
- Verilog "test benches" should include a `timescale directive. The first value defines the default time unit and the second the simulation resolution. In this example, the default time unit is 10 ns and the simulation resolution is 100 ps.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (7/3)

Modules:

- The actual stimuli consist of sequences of values assigned to the registers.
- Verilog "test benches" should include a `timescale directive. The first value defines the default time unit and the second the simulation resolution. In this example, the default time unit is 10 ns and the simulation resolution is 100 ps.
- After each assignment to registers a, *b*, and c, the # operator is present. In our case, # suspends processing for a period of time computed by multiplying the time unit defined by the *timescale* directive by the value that follows the operator itself. A delay of #10 results in a step of 100 ns.

```
// File assign_sim.v
`timescale 10ns/100ps

module tf;
  reg a, b, c;;
  wire z;

  initial begin
    $monitor( a, b, c,, z );
    $dumpvars( 0, tf );

    a = 0; b = 0; c = 0; #10
    a = 0; b = 0; c = 1; #10
    a = 0; b = 1; c = 0; #10
    a = 0; b = 1; c = 1; #10
    a = 1; b = 0; c = 0; #10
    a = 1; b = 0; c = 1; #10
    a = 1; b = 1; c = 0; #10
    a = 1; b = 1; c = 1; #10

    $finish;
  end

  fn_z device_under_test( a, b, c, z );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (8/1)

- Verilog has nets and variables of scalar and vector type.
  - A declaration without specifying a so-called "range" is one bit wide and is a scalar.

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (8/2)

- Verilog has nets and variables of scalar and vector type.
  - A declaration without specifying a so-called "range" is one bit wide and is a scalar.
  - If a declaration specifies a range, we get a multi-bit entity also known as a "vector". The syntax for defining a range (and therefore a vector) is the following:

    [MSB bit index : LSB bit index]

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (8/3)

- Verilog has nets and variables of scalar and vector type.
  - A declaration without specifying a so-called "range" is one bit wide and is a scalar.
  - If a declaration specifies a range, we get a multi-bit entity also known as a "vector". The syntax for defining a range (and therefore a vector) is the following:

    [MSB bit index : LSB bit index]

- In Verilog the following syntax is used to specify values:

  [*width*]'[*radix*][*literal*]

  width: declares the width of the value in bits;

  literal: represents the actual value

  radix: specifies the nature of the value expressed by *literal* and can be: *d* (decimal), *h* (hexadecimal), o (octal), *b* (binary).

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (9/1)

- Vector indexing To

  select one or more bits belonging to a vector, the [] operator is used, with the following syntax: [msb-index:lsb-index] or [bit-index]

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (9/2)

- Vector indexing To
  select one or more bits belonging to a vector, the [] operator is used, with the following syntax: [msb-index:lsb-index] or [bit-index]
- The module can have parameters that describe the interface through which it communicates externally. The parameters are associated with the names needed to use them inside the module.

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (9/3)

- Vector indexing To
  select one or more bits belonging to a vector, the [] operator is used, with the following syntax: [msb-index:lsb-index] or [bit-index]
- The module can have parameters that describe the interface through which it communicates externally. The parameters are associated with the names needed to use them inside the module.
- The direction of the signals must be declared for all the module's parameters.

```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

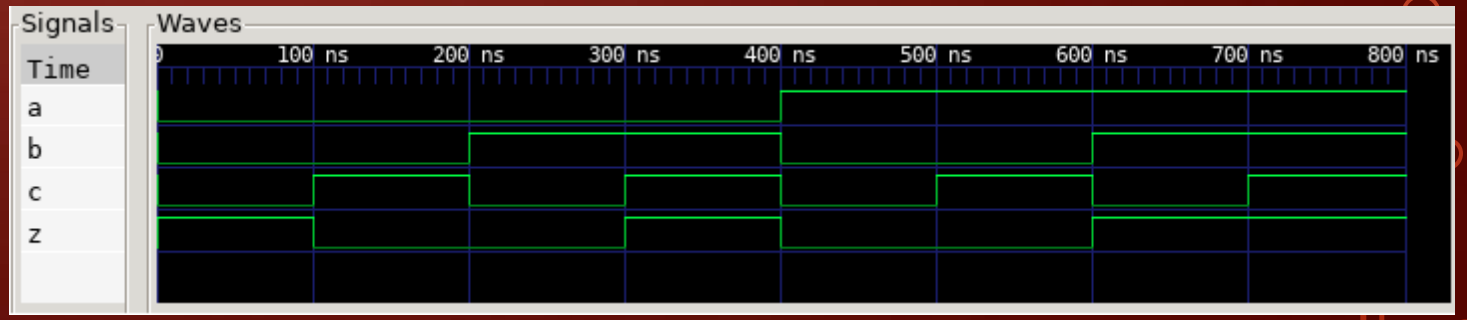
INTRO TUTORIAL (10)

Modules:

- With the stimuli supplied in module *tf* to the *device_under_test* instance of type *fn_z*, Icarus Verilog is run again.
- Running the command *vvp* (the simulator's run-time engine) and passing it *assign_sim.out* as a parameter, the file *dump.vcd is produced and* the simulation result is written to standard output. You can see that the last column matches what is expected from the truth table of function *z*.
- Finally, passing the dump.vcd file to the *gtkwave* software lets us display a diagram with signals *a*, *b*, *c* and *z* evolving over time.

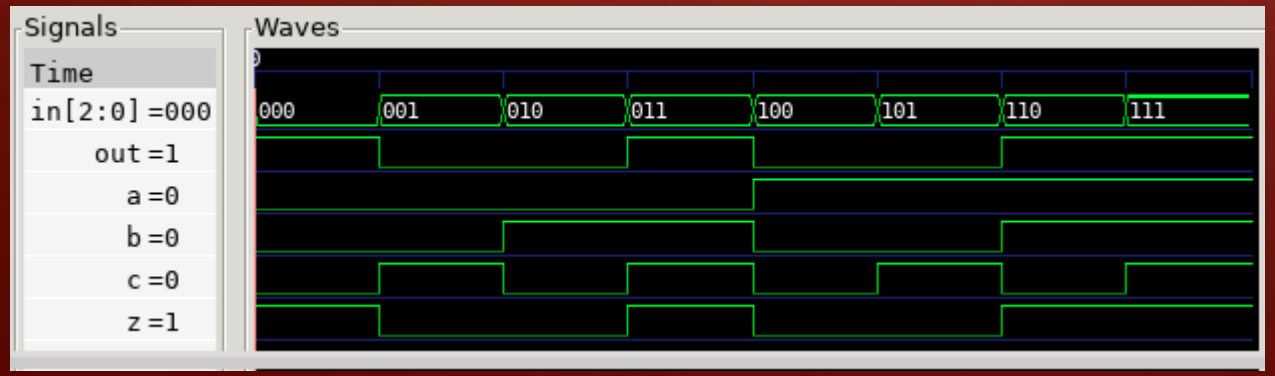
a	*b*	*c*	*z*
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

```
$ iverilog assign_sim.v -o assign_sim.out
$ vvp assign_sim.out
VCD info: dumpfile dump.vcd opened for output.
000 1
001 0
010 0
011 1
100 0
101 0
110 1
111 1
$ gtkwave dump.vcd &
$ _
```



INTRO TUTORIAL (11)

- As we have seen, data types are used to characterize the memory elements (*reg*) and the "nets", which in Verilog, remember, can be likened to *wires*. Several *wires* or *regs* can be grouped in an orderly way to represent a single data type.
- Groups of *wires* can be regarded as so-called "Buses", as they are classically represented in the context of digital systems.
- Memory elements can be likened to variables in programming languages; a single Flip-Flop (*reg*) is an example of a variable that stores one bit. A group of *regs*
  represents a typical *n*-bit register.




```
// File assign_sim_b.v
`timescale 10ns/100ps

module tf;
  reg [2:0] in;
  wire out;

  initial begin
    $monitor( in,, in[2], in[1], in[0],, out );
    $dumpvars( 0, tf );

    in = 3'b000; #10
    in = 3'b001; #10
    in = 3'b010; #10
    in = 3'b011; #10
    in = 3'b100; #10
    in = 3'b101; #10
    in = 3'b110; #10
    in = 3'b111; #10

    $finish;
  end

  fn_z device_under_test( in[2], in[1], in[0], out );
endmodule

module fn_z( input a, b, c, output z );
  assign z = ~a & ~b & ~c | a & b | b & c;
endmodule
```

INTRO TUTORIAL (12)

Operators

Verilog has several operators: alongside we see a summary table where we can also find the logical (bitwise) operators used so far. Below it, the precedence table.

Operatore	Precedenza
+ - ! ~ (unari)	alta
**	
* / %	
+ - (binari)	
<< >> <<< >>>	
< <= > >=	
== != === !==	
& ~&	
^ ~^ ^~	
| ~|	
&&	
||	
?:	bassa

Operatore	Operazione	Descrizione
{ , }	Concatenazione	Unisce i bit di due o più espressioni separate da virgole
+	Addizione	Somma due operandi
-	Sottrazione	Differenza tra due operandi
-	Meno unario	Cambia il segno dell'operando
*	Moltiplicazione	Moltiplica due operandi
/	Divisione	Divide due operandi
%	Modulo	Calcola il resto
**	Potenza	Elevazione a potenza
>	Maggiore di	Vero se maggiore
>=	Maggiore o uguale di	Vero se maggiore o uguale
<	Minore di	Vero se minore
<=	Minore o uguale di	Vero se minore o uguale
!	Negazione logica	Complemento logico unario
&&	AND logico	AND di due valori logici
||	OR logico	OR di due valori logici
==	Uguaglianza logica	Confronta due valori per l'uguaglianza
!=	Disuguaglianza logica	Confronta due valori per la disuguaglianza
===	Uguaglianza (case)	Confronta due valori per l'uguaglianza (case)
!==	Disuguaglianza (case)	Confronta due valori per la disuguaglianza (case)
~	Negazione bitwise	Completa ogni bit dell'operando
&	AND bit a bit	Produce l'AND bit a bit di due operandi
|	OR bit a bit	Produce l'OR inclusivo bit a bit di due operandi
^	XOR bit a bit	Produce lo XOR esclusivo bit a bit di due operandi
^~ or ~^	Equivalenza	Produce lo XNOR esclusivo bit a bit di due operandi
&	Riduzione unaria AND	Produce l'AND a singolo bit di tutti i bit dell'operando
~&	Riduzione unaria NAND	Produce il NAND a singolo bit di tutti i bit dell'operando
|	Riduzione unaria OR	Produce l'OR a singolo bit di tutti i bit dell'operando
~|	Riduzione unaria NOR	Produce il NOR a singolo bit di tutti i bit dell'operando
^	Riduzione unaria XOR	Produce lo XOR a singolo bit di tutti i bit dell'operando
^~ or ~^	Riduzione unaria XNOR	Produce lo XOR a singolo bit di tutti i bit dell'operando
<<	Scorrimento a sinistra	Sposta l'operando sinistro a sinistra (logicamente) del numero di posizioni di bit specificato dall'operando destro
>>	Scorrimento a destra	Sposta l'operando sinistro a destra (logicamente) del numero di posizioni di bit specificato dall'operando destro
<<<	Scorrimento aritmetico a sinistra	Sposta l'operando sinistro a sinistra (aritmeticamente) del numero di posizioni di bit specificato dall'operando destro
>>>	Scorrimento aritmetico a destra	Sposta l'operando sinistro a destra (aritmeticamente) del numero di posizioni di bit specificato dall'operando destro
? :	Condizionale	Assegnare uno dei due valori in base all'espressione

INTRO TUTORIAL (13/1)

Always Block

- Verilog allows more complex logic than the simple *assign* thanks to the use of *always* blocks. Combinational logic can be written using *always* @(*). The value inside the parentheses is called the "sensitivity list". Using * tells the tool to compute this "sensitivity list" automatically.

```
module fn_z( input a, b, c, output /*reg*/ z );
  reg z;  // required with always
  // assign z = ~a & ~b & ~c | a & b | b & c;
  // always @(a,b,c)
  always @(*)
    begin
      z = ~a & ~b & ~c | a & b | b & c;
    end
endmodule
```

INTRO TUTORIAL (13/2)

Always Block

```
module fn_z( input a, b, c, output /*reg*/ z );
  reg z;  // required with always
  // assign z = ~a & ~b & ~c | a & b | b & c;
  // always @(a,b,c)
  always @(*)
    begin
      z = ~a & ~b & ~c | a & b | b & c;
    end
endmodule
```

- Verilog allows more complex logic than the simple *assign* thanks to the use of *always* blocks. Combinational logic can be written using *always* @(*). The value inside the parentheses is called the "sensitivity list". Using * tells the tool to compute this "sensitivity list" automatically.
- If a signal is driven inside an *always* block, it must be defined as *reg*. Conversely, as we have seen, if a continuous assignment (*assign*) is used, it can be defined as *wire* which always represents a connection, whereas a *reg* is equivalent to a *wire* if it is used inside an *always* @(*) construct.

INTRO TUTORIAL (14)

The *always block*

- A *reg* represents a register if it is used inside an *always* block with synchronous sequential logic, which is generated by specifying the list of signals in the "sensitivity list" of the *always* block.

```
module counter( input Clk, input G );
  reg [2:0] S = 3'd0; // WARNING: not synthesizable

  always @( posedge Clk ) begin
    case( S )
      3'b000: S <= !G ? 3'b001 : 3'b001;
      3'b001: S <= !G ? 3'b010 : 3'b011;
      3'b010: S <= !G ? 3'b011 : 3'b110;
      3'b011: S <= !G ? 3'b100 : 3'b010;
      3'b100: S <= !G ? 3'b101 : 3'b000;
      3'b101: S <= !G ? 3'b110 : 3'b100;
      3'b110: S <= !G ? 3'b111 : 3'b111;
      3'b111: S <= !G ? 3'b000 : 3'b101;
    endcase
  end
endmodule
```

INTRO TUTORIAL (14/2)

The *always block*

- A *reg* represents a register if it is used inside an *always* block with synchronous sequential logic, which is generated by specifying the list of signals in the "sensitivity list" of the *always* block.
- Moreover, with the keywords *posedge* and *negedge* you can specify on which edge (signal change) the block is activated: *posedge 0→1*, *negedge 1→ 0*.

```
module counter( input Clk, input G );
  reg [2:0] S = 3'd0; // WARNING: not synthesizable

  always @( posedge Clk ) begin
    case( S )
      3'b000: S <= !G ? 3'b001 : 3'b001;
      3'b001: S <= !G ? 3'b010 : 3'b011;
      3'b010: S <= !G ? 3'b011 : 3'b110;
      3'b011: S <= !G ? 3'b100 : 3'b010;
      3'b100: S <= !G ? 3'b101 : 3'b000;
      3'b101: S <= !G ? 3'b110 : 3'b100;
      3'b110: S <= !G ? 3'b111 : 3'b111;
      3'b111: S <= !G ? 3'b000 : 3'b101;
    endcase
  end
endmodule
```

INTRO TUTORIAL (14/3)

The *always block*

- A *reg* represents a register if it is used inside an *always* block with synchronous sequential logic, which is generated by specifying the list of signals in the "sensitivity list" of the *always* block.
- Moreover, with the keywords *posedge* and *negedge* you can specify on which edge (signal change) the block is activated: *posedge 0→1*, *negedge 1→ 0*.
- Inside the *always* block there is a case statement that serves to synthesize the next-state function, taken from the Gray/Binary counter example seen earlier. As is known, the function that computes the next state is a combinational structure, and note that in this case no equations were used to define it. *always* blocks can make the code more readable and easier to understand.

```
module counter( input Clk, input G );
  reg [2:0] S = 3'd0; // WARNING: not synthesizable

  always @( posedge Clk ) begin
    case( S )
      3'b000: S <= !G ? 3'b001 : 3'b001;
      3'b001: S <= !G ? 3'b010 : 3'b011;
      3'b010: S <= !G ? 3'b011 : 3'b110;
      3'b011: S <= !G ? 3'b100 : 3'b010;
      3'b100: S <= !G ? 3'b101 : 3'b000;
      3'b101: S <= !G ? 3'b110 : 3'b100;
      3'b110: S <= !G ? 3'b111 : 3'b111;
      3'b111: S <= !G ? 3'b000 : 3'b101;
    endcase
  end
endmodule
```

FLOW TABLE

PRE $S_2S_1S_0$	NEXT G=0	NEXT G=1
000	001	001
001	010	011
010	011	110
011	100	010
100	101	000
101	110	100
110	111	111
111	000	101

INTRO TUTORIAL (14/4)

The *always block*

- A *reg* represents a register if it is used inside an *always* block with synchronous sequential logic, which is generated by specifying the list of signals in the "sensitivity list" of the *always* block.
- Moreover, with the keywords *posedge* and *negedge* you can specify on which edge (signal change) the block is activated: *posedge 0→1*, *negedge 1→ 0*.
- Inside the *always* block there is a *case* statement that serves to synthesize the next-state function, taken from the Gray/Binary counter example seen earlier. As is known, the function that computes the next state is a combinational structure, and note that in this case no equations were used to define it. *always* blocks can make the code more readable and easier to understand.
- Verilog has two assignment operators: blocking (=) and non-blocking (<=). The blocking operator (=) is used for combinational logic and the non-blocking one for synchronous sequential logic.

```
module counter( input Clk, input G );
  reg [2:0] S = 3'd0; // WARNING: not synthesizable

  always @( posedge Clk ) begin
    case( S )
      3'b000: S <= !G ? 3'b001 : 3'b001;
      3'b001: S <= !G ? 3'b010 : 3'b011;
      3'b010: S <= !G ? 3'b011 : 3'b110;
      3'b011: S <= !G ? 3'b100 : 3'b010;
      3'b100: S <= !G ? 3'b101 : 3'b000;
      3'b101: S <= !G ? 3'b110 : 3'b100;
      3'b110: S <= !G ? 3'b111 : 3'b111;
      3'b111: S <= !G ? 3'b000 : 3'b101;
    endcase
  end
endmodule
```

INTRO TUTORIAL (15/1)

"Test Bench" modules

- In addition to the actual counter module, an additional module (clock) is needed that generates the cadence (clk_gen). As written, this module is not synthesizable, but that is not a problem since it only serves to generate one of the two test vectors (Cp) for counter_1.

```
`timescale 10ns/100ps

module tf;
  reg Mode; // 0=Bin 1=Gray
  wire Cp;

  initial begin
    $dumpvars( 0, tf );
    Mode = 0; #102
    Mode = 1; #108
    $finish;
  end

  clock clk_gen( .Out( Cp ) );
  counter counter_1( .Clk( Cp ), .G( Mode ) );
endmodule

module clock( output reg Out );
  // WARNING: module not synthesizable
  initial
    Out = 1;

  always
    begin
      #5
      Out = ~Out;
    end
endmodule
```

INTRO TUTORIAL (15/2)

"Test Bench" modules

- In addition to the actual counter module, an additional module (clock) is needed that generates the cadence (clk_gen). As written, this module is not synthesizable, but that is not a problem since it only serves to generate one of the two test vectors (Cp) for counter_1.
- The two modules declared inside *tf*, namely clk_gen and counter_1, are interconnected through the net Cp (*wire*).

```
`timescale 10ns/100ps

module tf;
  reg Mode; // 0=Bin 1=Gray
  wire Cp;

  initial begin
    $dumpvars( 0, tf );
    Mode = 0; #102
    Mode = 1; #108
    $finish;
  end

  clock clk_gen( .Out( Cp ) );
  counter counter_1( .Clk( Cp ), .G( Mode ) );
endmodule

module clock( output reg Out );
  // WARNING: module not synthesizable
  initial
    Out = 1;

  always
    begin
      #5
      Out = ~Out;
    end
endmodule
```

INTRO TUTORIAL (15/3)

"Test Bench" modules

- In addition to the actual counter module, an additional module (clock) is needed that generates the cadence (clk_gen). As written, this module is not synthesizable, but that is not a problem since it only serves to generate one of the two test vectors (Cp) for counter_1.
- The two modules declared inside tf, namely clk_gen and counter_1, are interconnected through the net Cp (wire).
- In the example there are two constructs that start with $. The first ($dumpvars) instructs the simulator to consider the future observation of all the nets of the main module (the top module*tf*).

  The second construct is $finish. This "system task" tells the simulator to stop and terminate.

```
`timescale 10ns/100ps

module tf;
  reg Mode; // 0=Bin 1=Gray
  wire Cp;

  initial begin
    $dumpvars( 0, tf );
    Mode = 0; #102
    Mode = 1; #108
    $finish;
  end

  clock clk_gen( .Out( Cp ) );
  counter counter_1( .Clk( Cp ), .G( Mode ) );
endmodule

module clock( output reg Out );
  // WARNING: module not synthesizable
  initial
    Out = 1;

  always
    begin
      #5
      Out = ~Out;
    end
endmodule
```

INTRO TUTORIAL (16/1)

- To make the counter synthesizable, a mechanism must be added to clear the state from outside. For this purpose an *input* (net *Rst*) can be added to the module's list of inputs/outputs.

```
module counter( input Clk, input Rst, input G );
  reg [2:0] S;

  always @( posedge Clk or posedge Rst )
    begin
      if ( Rst )
        begin
          S <= 3'b000;
        end
      else
        begin
          case ( S )
            3'b000: S <= !G ? 3'b001 : 3'b001;
            3'b001: S <= !G ? 3'b010 : 3'b011;
            3'b010: S <= !G ? 3'b011 : 3'b110;
            3'b011: S <= !G ? 3'b100 : 3'b010;
            3'b100: S <= !G ? 3'b101 : 3'b000;
            3'b101: S <= !G ? 3'b110 : 3'b100;
            3'b110: S <= !G ? 3'b111 : 3'b111;
            3'b111: S <= !G ? 3'b000 : 3'b101;
          endcase
        end
    end
endmodule
```

INTRO TUTORIAL (16/2)

- To make the counter synthesizable, a mechanism must be added to clear the state from outside. For this purpose an *input* (net *Rst*) can be added to the module's list of inputs/outputs.
- The net *Rst* is therefore also added to the *sensitivity list* of the *always* statement, and every change of it (*posedge*) will therefore be taken into account.

```
module counter( input Clk, input Rst, input G );
  reg [2:0] S;

  always @( posedge Clk or posedge Rst )
    begin
      if ( Rst )
        begin
          S <= 3'b000;
        end
      else
        begin
          case ( S )
            3'b000: S <= !G ? 3'b001 : 3'b001;
            3'b001: S <= !G ? 3'b010 : 3'b011;
            3'b010: S <= !G ? 3'b011 : 3'b110;
            3'b011: S <= !G ? 3'b100 : 3'b010;
            3'b100: S <= !G ? 3'b101 : 3'b000;
            3'b101: S <= !G ? 3'b110 : 3'b100;
            3'b110: S <= !G ? 3'b111 : 3'b111;
            3'b111: S <= !G ? 3'b000 : 3'b101;
          endcase
        end
    end
endmodule
```

INTRO TUTORIAL (16/3)

- To make the counter synthesizable, a mechanism must be added to clear the state from outside. For this purpose an *input* (net *Rst*) can be added to the module's list of inputs/outputs.
- The net *Rst* is therefore also added to the *sensitivity list* of the *always* statement, and every change of it (*posedge*) will therefore be taken into account.
- If, when the *always* block is triggered, the value of *Rst* is equal to 1, we are facing a request to clear the register that holds the state.

```
module counter( input Clk, input Rst, input G );
  reg [2:0] S;

  always @( posedge Clk or posedge Rst )
    begin
      if ( Rst )
        begin
          S <= 3'b000;
        end
      else
        begin
          case ( S )
            3'b000: S <= !G ? 3'b001 : 3'b001;
            3'b001: S <= !G ? 3'b010 : 3'b011;
            3'b010: S <= !G ? 3'b011 : 3'b110;
            3'b011: S <= !G ? 3'b100 : 3'b010;
            3'b100: S <= !G ? 3'b101 : 3'b000;
            3'b101: S <= !G ? 3'b110 : 3'b100;
            3'b110: S <= !G ? 3'b111 : 3'b111;
            3'b111: S <= !G ? 3'b000 : 3'b101;
          endcase
        end
    end
endmodule
```

INTRO TUTORIAL (16/4)

- To make the counter synthesizable, a mechanism must be added to clear the state from outside. For this purpose an *input* (net *Rst*) can be added to the module's list of inputs/outputs.
- The net *Rst* is therefore also added to the *sensitivity list* of the *always* statement, and every change of it (*posedge*) will therefore be taken into account.
- If, when the *always* block is triggered, the value of *Rst* is equal to 1, we are facing a request to clear the register that holds the state.
- If it was not *Rst* that triggered the *always* block, then it was certainly the *Clk* input. The register will therefore be updated with the rule seen earlier, contained in the *case* statement.

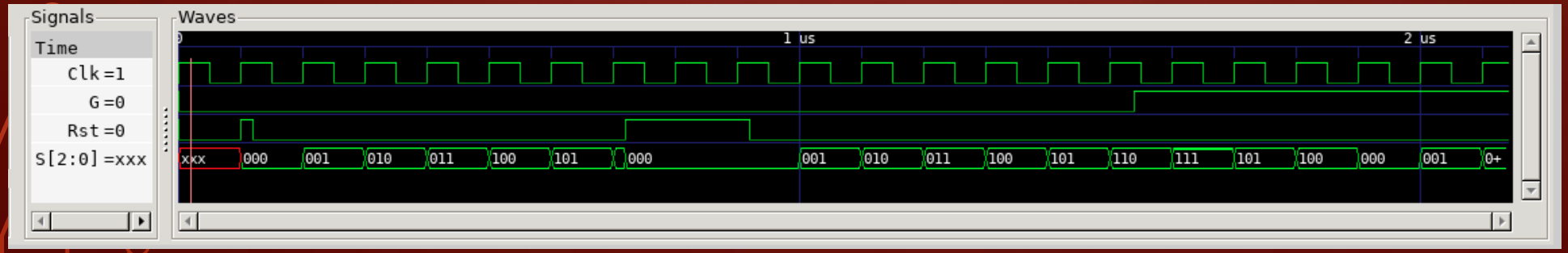
```
module counter( input Clk, input Rst, input G );
  reg [2:0] S;

  always @( posedge Clk or posedge Rst )
    begin
      if ( Rst )
        begin
          S <= 3'b000;
        end
      else
        begin
          case ( S )
            3'b000: S <= !G ? 3'b001 : 3'b001;
            3'b001: S <= !G ? 3'b010 : 3'b011;
            3'b010: S <= !G ? 3'b011 : 3'b110;
            3'b011: S <= !G ? 3'b100 : 3'b010;
            3'b100: S <= !G ? 3'b101 : 3'b000;
            3'b101: S <= !G ? 3'b110 : 3'b100;
            3'b110: S <= !G ? 3'b111 : 3'b111;
            3'b111: S <= !G ? 3'b000 : 3'b101;
          endcase
        end
    end
endmodule
```

INTRO TUTORIAL (17)

Once the test fixtures have been updated to support the new Reset input, the simulation can be run to verify that the counter works correctly. As can be seen in the diagrams below, the Rst pulse initially clears the register, which started with an undefined value (xxx). When Rst is released, the subsequent clock pulses (Clk) advance the count. An Rst level of 1 held long enough to exceed the period of several clock cycles shows that the Rst input has priority, like an asynchronous reset applied directly to the registers.

Lastly, you can see that a change from zero to one of the G input modifies the counting criterion, switching from Binary to Gray code.




```
`timescale 10ns/100ps

module tf;
  reg Mode;
  wire Cp;
  reg Reset;

  initial begin
    $dumpvars( 0, tf );
    Mode = 0; #154
    Mode = 1; #60
    $finish;
  end

  initial begin
    Reset = 0; #10
    Reset = 1; #2
    Reset = 0; #60
    Reset = 1; #20
    Reset = 0;
  end

  clock Clk( .Out( Cp ) );
  counter counter_1 (
    .Clk( Cp ),
    .Rst( Reset ),
    .G( Mode )
  );
endmodule
```

Q&A

- Speakers:

Emanuel Conti (e.conti@embedded.sm)

Giuliano Cardinali (giulicard@gmail.com)

BIBLIOGRAPHY

- Reti logiche e
  calcolatori F. Luccio, L. Pagli –
  ISBN: 978-8833954870
- FPGAs: Instant Access
  Clive Maxfield - ISBN: 978-0750689748
- Bebop to the Boolean Boogie: An Unconventional Guide to Electronics
  Clive Maxfield - ISBN: 978-1856175074
- Logic Circuit Design: Selected Topics and Methods, Second Ed.
  Shimon P. Vingron - ISBN: 978-3031406720
- The Verilog® Hardware Description Language, 5th Ed.
  Donald E. Thomas, Philip R. Moorby - ISBN: 978-1402070896
- Digital design : with an introduction to the verilog hdl
  M. Morris Mano, Michael D. Ciletti.—5th ed. - ISBN-13: 978-0132774208
- Digital Design Principles and Practices
  John F. Wakerly - ISBN: 978-0134460093